

A HANDS-ON JAVASCRIPT INTERVIEW-PREP COURSE

Design Patterns

23 classic (Gang of Four) patterns, the 5 SOLID principles behind them, 6 LLD machine-coding problems that apply them, and 6 System Design (HLD) topics for scaling beyond one machine.

Compiled for reading — run the code from the original project to experiment.

Design Patterns Course (JavaScript) — run locally

Har pattern ko **chala kar** samajhne ke liye. Koi npm install ki zaroorat nahi — sab kuch pure Node hai.

Requirement

- Node.js 18+ installed. Check karo:

```
node --version
```

Agar nahi hai: <https://nodejs.org> se LTS install karo.

Kaise use karein (har lesson ke liye same flow)

1. Lesson folder ka `README.md` padho.
2. `1-naive.js` chalao — dekho problem kaisi dikhti hai:

```
node 01-strategy/1-naive.js
```

1. Pattern wali files chalao — dekho kaise clean hota hai.
2. `exercise.js` khol ke khud solve karo. Chala kar test karo.

Structure — saare 23 classic (GoF) patterns

Creational — object *kaise banta hai*

```
design-patterns-course/  
├─ 04-singleton/      ← sirf ek instance guarantee karo  
├─ 05-factory-method/ ← object banane ka kaam ek function ko do  
├─ 06-abstract-factory/ ← related objects ka poora consistent family banao  
├─ 07-builder/        ← complex object ko step-by-step banao  
├─ 08-prototype/      ← naye instance existing object ko CLONE karke banao
```

Structural — objects *kaise jude/compose hote hain*

└─ 09-adapter/	← mismatched interface ko apne shape mein convert karo
└─ 10-bridge/	← do independent dimensions ko alag hierarchies mein todo
└─ 11-composite/	← tree (leaf + branch) ko uniform interface do
└─ 12-decorator/	← runtime pe stackable behavior jodo
└─ 13-facade/	← complex subsystem ke aage ek simple entry point
└─ 14-flyweight/	← shared data reuse karo, memory bachao
└─ 15-proxy/	← real object ke aage access-control/lazy-load stand-in

Behavioral — objects *kaise interact/communicate karte hain*

└─ 01-strategy/	← runtime pe interchangeable algorithms (yahin se shuru karo)
└─ 02-observer/	← ek-se-anek: state badle, sab subscribers ko pata chale
└─ 03-command/	← action ko object banao (undo/queue/replay)
└─ 16-chain-of-responsibility/	← handlers ki chain, jo handle kar sake wahi kare
└─ 17-iterator/	← kisi bhi collection ko uniform tarike se traverse karo
└─ 18-mediator/	← components sirf ek central mediator se baat karein
└─ 19-memento/	← object khud apna snapshot save/restore kare
└─ 20-state/	← behavior current status pe depend kare, state khud transition kare
└─ 21-template-method/	← shared skeleton, sirf steps subclass mein override
└─ 22-visitor/	← classes ko chhue bina naye operations add karo
└─ 23-interpreter/	← chhoti grammar ko composable nodes se evaluate karo
└─ package.json	← shortcut scripts (optional)

SOLID Principles — the WHY behind the patterns

5 principles jo batate hain ki achha OO design **kyun** achha hota hai — har principle ek

✗ violation aur ✓ fix ke saath, aur kaunse patterns usi principle ko satisfy karne ke liye

exist karte hain. Poora index: [solid-principles/README.md](#) .

LLD Practice — interview machine-coding problems

Patterns seekhne ke baad **use** karna practice karo — 6 classic "design X" interview

questions (Parking Lot, Splitwise, Elevator, Rate Limiter, Movie Booking, Notification

Service), har ek mein patterns real problem pe applied dikhte hain, plus "interviewer kya

dekhta hai" notes. Poora index: [lld-practice/README.md](#) .

System Design (HLD) — scaling beyond one machine

6 topics jo senior/staff interviews mein sabse zyada aate hain — Load Balancing, Caching (LRU), Consistent Hashing, DB Sharding & Replication, CAP Theorem, Message Queues. Har topic

ek runnable simulation ke saath (in mein se kai concept "it depends" hote hain, isliye zyada zor tradeoffs samajhne pe hai, sirf code pe nahi). Poora index:

[system-design/README.md](#) .

Shortcut scripts (optional)

`package.json` mein scripts hain, toh yeh bhi kaam karta hai:

```
npm run strategy:registry
npm run observer:emitter
npm run singleton:class
npm run visitor:pattern
```

Poori list ke liye `package.json` kholo — har lesson ke `naive` /pattern

files ke liye ek script hai.

Golden rule

Sirf padhna nahi — **chalao, tood-fod karo, dobara chalao**. Jo pattern tumne khud break karke wapas jode ho, wahi yaad rehta hai.

Table of Contents

Part I — Design Patterns

CREATIONAL PATTERNS

1. Singleton
2. Factory Method
3. Abstract Factory
4. Builder
5. Prototype

STRUCTURAL PATTERNS

6. Adapter
7. Bridge
8. Composite
9. Decorator
10. Facade
11. Flyweight
12. Proxy

BEHAVIORAL PATTERNS

13. Strategy
14. Observer
15. Command
16. Chain of Responsibility
17. Iterator
18. Mediator
19. Memento
20. State
21. Template Method
22. Visitor
23. Interpreter

Part II — SOLID Principles

SOLID PRINCIPLES

- 24. Single Responsibility Principle (SRP)
 - 25. Open/Closed Principle (OCP)
 - 26. Liskov Substitution Principle (LSP)
 - 27. Interface Segregation Principle (ISP)
 - 28. Dependency Inversion Principle (DIP)
-

Part III — LLD Practice

LLD PRACTICE

- 29. Parking Lot System
 - 30. Splitwise (Expense Sharing)
 - 31. Elevator System
 - 32. Rate Limiter
 - 33. Movie Ticket Booking (BookMyShow-lite)
 - 34. Notification Service
-

Part IV — System Design (HLD)

SYSTEM DESIGN (HLD)

- 35. Load Balancing
- 36. Caching Strategies
- 37. Consistent Hashing
- 38. Database Sharding & Replication
- 39. CAP Theorem & Consistency Models
- 40. Message Queues & Async Processing

1. Singleton

Problem: kuch cheezein app mein **sirf ek hi** honi chahiye — config, connection pool, logger. Agar har jagah `new` karte raho to alag-alag state ban jaata hai aur sab out of sync ho jaate hain.

Idea: class khud control kare ki uska sirf ek instance bane. Pehli baar `new` / `getInstance()` call hone par instance banta hai, uske baad wahi wapas milta hai.

Files — is order mein chalao

```
node 1-naive.js          # ❌ har `new Config()` alag object
node 2-singleton.js      # ✅ class + static instance
node 3-singleton-module.js # ✅ idiomatic JS: module scope khud singleton hai
node exercise.js         # 🔄 tumhara turn
```

Principles yahan dikh rahe hain

1. **Single source of truth** — ek hi state, jahan se bhi access karo.
2. **Controlled instantiation** — constructor khud check karta hai ki instance already ban chuka hai ya nahi.

When to skip

Singleton **global mutable state** hai — testing mushkil kar deta hai (state tests ke beech leak hota hai) aur hidden dependencies bana deta hai. Agar dependency injection se kaam chal sakta hai (function/constructor ko object pass karo), woh usually behtar hai. JS mein simple config/constants ke liye module export (`3-singleton-module.js`) aksar kaafi hai — poora class/pattern zaroori nahi.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — har jagah naya config object bana rahe hain.
// Chalao: node 1-naive.js
```

```

class Config {
  constructor() {
    this.settings = { theme: 'light', retries: 3 };
  }
}

const configA = new Config();
const configB = new Config();

configA.settings.theme = 'dark';
console.log('configA.theme:', configA.settings.theme);
console.log('configB.theme:', configB.settings.theme); // still 'light' - inconsistent!

// PROBLEM: Config app-wide ek hi hona chahiye (ek hi source of truth). Har
// `new Config()` alag state banata hai - kahin change karo, kahin nahi dikhta.

```

✓ 2-singleton.js — the pattern

```

// ✓ SINGLETON PATTERN - sirf ek instance, jahan se bhi maango wahi milega.
// Chalao: node 2-singleton.js

class Config {
  static #instance;

  constructor() {
    if (Config.#instance) return Config.#instance; // already bana hai to wahi do
    this.settings = { theme: 'light', retries: 3 };
    Config.#instance = this;
  }

  static getInstance() {
    return Config.#instance ?? new Config();
  }
}

const configA = Config.getInstance();
const configB = Config.getInstance();

configA.settings.theme = 'dark';
console.log('configA.theme:', configA.settings.theme);
console.log('configB.theme:', configB.settings.theme); // 'dark' - same object hai
console.log('same instance?', configA === configB);

```

✓ 3-singleton-module.js — the pattern

```

// ✓ IDIOMATIC JS: ES module khud hi singleton hai - module-level object
// export karo, JS engine use sirf EK baar evaluate karta hai (import cache).
// Chalao: node 3-singleton-module.js

const settings = { theme: 'light', retries: 3 }; // module scope = automatically singleton
export default settings;

```

```
// Isko do jagah "import" karke dikhate hain (simulate):
const configA = settings;
const configB = settings;
configA.theme = 'dark';
console.log('configA.theme:', configA.theme);
console.log('configB.theme:', configB.theme); // 'dark' – same reference

// Yehi tarika JS mein sabse zyada use hota hai: class + getInstance() ki
// zaroorat kam padti hai, module system khud singleton de deta hai.
```

exercise.js — your turn

```
// 🐛 EXERCISE – Logger ko Singleton banao taaki poore app mein ek hi log
// history ho.
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
class Logger {
  constructor() { this.logs = []; }
  log(msg) { this.logs.push(msg); console.log('LOG:', msg); }
}

const loggerA = new Logger();
const loggerB = new Logger();
loggerA.log('user signed up');
loggerB.log('payment received');
console.log('loggerA has', loggerA.logs.length, 'logs'); // 1 – but chahiye 2!

// =====
// TODO 1: Logger class mein static #instance field + getInstance() add karo
//         (2-singleton.js jaisa), taaki loggerA aur loggerB same object hon.
//
// TODO 2 (bonus): ek module-level `export default new Logger()` variant bhi
//                 try karo (3-singleton-module.js jaisa) – dono approach ka tradeoff
//                 socho: class singleton lazy hai (pehli baar use hone par banta
//                 hai), module singleton eager hai (import hote hi ban jaata hai).
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

2. Factory Method

Problem: object banane ka logic (`new X()` , kaunsi class chuni) caller ke code mein bikhra hua hai. Naya type add karna matlab har jagah if/else dhoondh ke edit karna.

Idea: object banane ka kaam ek dedicated factory function/method ko do.

Caller sirf "mujhe X type chahiye" bolta hai, konsi class instantiate hui — uski chinta nahi karta.

Files — is order mein chalao

node 1-naive.js	# ❌ caller khud class chunta hai
node 2-factory-method.js	# ✅ factory function + registry
node exercise.js	# 🎯 tumhara turn

Principles yahan dikh rahe hain

1. **Separate creation from use** — "kya banaya" aur "kaise use kiya" alag.
2. **Open/closed** — naya type = registry mein entry, caller code same rehta.

When to skip

Sirf ek hi class hai jo kabhi variant nahi legi? Seedha `new X()` karo — factory ki indirection tabhi kamaati hai jab **multiple related types** ho jinke beech runtime pe chunna ho.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — caller khud decide karta hai KAUNSI class banani hai.  
// Chalao: node 1-naive.js  
  
class EmailNotification {  
  send(msg) { console.log(`📧 Email: ${msg}`); }  
}  
  
class SmsNotification {  
  send(msg) { console.log(`📱 SMS: ${msg}`); }  
}  
  
function notify(type, msg) {
```

```

let notification;
if (type === 'email') notification = new EmailNotification();
else if (type === 'sms') notification = new SmsNotification();
else throw new Error(`Unknown type: ${type}`);
notification.send(msg);
}

notify('email', 'Order shipped');
notify('sms', 'Order shipped');

// PROBLEM: naya notification type (push) add karne ke liye notify() function
// dobara edit karna padega. Creation Logic aur usage Logic mixed hain.

```

✓ 2-factory-method.js — the pattern

```

// ✓ FACTORY METHOD – object banane ka kaam ek dedicated function/method ko
// do, caller sirf "type" maangta hai, "how to build" nahi jaanta.
// Chalao: node 2-factory-method.js

class EmailNotification {
  send(msg) { console.log(`✉ Email: ${msg}`); }
}
class SmsNotification {
  send(msg) { console.log(`📱 SMS: ${msg}`); }
}
class PushNotification {
  send(msg) { console.log(`🔔 Push: ${msg}`); }
}

// Factory – ye "kaunsi class banani hai" ka decision yahin rakhta hai
function createNotification(type) {
  const notifications = {
    email: EmailNotification,
    sms: SmsNotification,
    push: PushNotification,
  };
  const NotificationClass = notifications[type];
  if (!NotificationClass) throw new Error(`Unknown type: ${type}`);
  return new NotificationClass();
}

function notify(type, msg) {
  const notification = createNotification(type); // caller ko class ka naam pata hi nahi
  notification.send(msg);
}

notify('email', 'Order shipped');
notify('push', 'Order shipped');

// Naya type add karna = registry mein ek line, notify() ko CHHUE BINA.

```

```
// 🐛 EXERCISE – shape banane ka if/else ko Factory Method mein refactor karo.
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
function makeShape(type, size) {
  if (type === 'circle') return { type, area: () => Math.PI * size * size };
  if (type === 'square') return { type, area: () => size * size };
  throw new Error(`Unknown shape: ${type}`);
}

console.log('circle area:', makeShape('circle', 3).area().toFixed(2));
console.log('square area:', makeShape('square', 3).area());

// =====
// TODO 1: `shapeFactories` registry object banao – { circle: (size) => {...},
//         square: (size) => {...} } – jismein har entry ek factory function
//         hai jo shape object return karta hai.
//
// TODO 2: `makeShape(type, size)` ko registry lookup karne wala banao (jaisa
//         2-factory-method.js mein hai).
//
// TODO 3 (bonus): 'triangle' add karo (area = base*height/2, size ko base aur
//         height dono maan lo ya function signature adjust karo) – makeShape
//         ko chhue bina.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

3. Abstract Factory

Problem: kuch objects **families** mein aate hain jinka saath mein consistent rehna zaroori hai (theme ke button+checkbox, DB ke connection+query-builder). Agar har object alag jagah se banta hai, mismatch (dark button + light checkbox) possible ho jaata hai.

Idea: ek factory poori family banaye. Caller ek hi factory pakadta hai aur usi se saare related objects nikalta hai — mismatch structurally impossible ho jaata hai.

Files — is order mein chalao

```
node 1-naive.js           # ❌ har object alag factory function se — mismatch possible
node 2-abstract-factory.js # ✅ ek factory = poori consistent family
node exercise.js          # 🎯 tumhara turn
```

Principles yahan dikh rahe hain

1. **Factory Method ka bada bhai** — ek object ke bajaye, related objects ka poora **set** banata hai.
2. **Consistency by construction** — galat combination banana structurally mushkil ho jaata hai, runtime check ki zaroorat nahi.

When to skip

Agar objects ke beech koi "family" relationship nahi hai (wo independently change ho sakte hain), plain Factory Method (Lesson 5) kaafi hai — Abstract Factory ka extra layer overkill hoga.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — button aur checkbox alag-alag banaye ja rahe hain,
// theme consistency guarantee nahi hoti.
// Chalao: node 1-naive.js

function createButton(theme) {
  return theme === 'dark'
    ? { render: () => console.log('🌑 [dark button]')}
    : { render: () => console.log('🌞 [light button]')}
```

```

    : { render: () => console.log('🟢 [light button]') };
  }
  function createCheckbox(theme) {
    return theme === 'dark'
      ? { render: () => console.log('✅ [dark checkbox]') }
      : { render: () => console.log('❌ [light checkbox]') };
  }

  // Bug: kisi ne galti se mismatched theme pass kar diya
  const button = createButton('dark');
  const checkbox = createCheckbox('light'); // 😬 mismatch – koi compile-time check nahi
  button.render();
  checkbox.render();

  // PROBLEM: button aur checkbox alag-alag jagah se banaye jaate hain – kuch bhi
  // unhe "same family" (same theme) rehne ki guarantee nahi deta.

```

✅ 2-abstract-factory.js — the pattern

```

// ✅ ABSTRACT FACTORY – related objects ka poora "family" ek hi factory se
// banta hai, taaki wo hamesha consistent rahein (e.g. sab dark, ya sab light).
// Chalao: node 2-abstract-factory.js

const lightFactory = {
  createButton: () => ({ render: () => console.log('🟢 [light button]') }),
  createCheckbox: () => ({ render: () => console.log('❌ [light checkbox]') }),
};

const darkFactory = {
  createButton: () => ({ render: () => console.log('🟡 [dark button]') }),
  createCheckbox: () => ({ render: () => console.log('✅ [dark checkbox]') }),
};

function getFactory(theme) {
  const factories = { light: lightFactory, dark: darkFactory };
  return factories[theme] ?? (() => { throw new Error(`Unknown theme: ${theme}`); })();
}

// UI code ko sirf EK factory milta hai – mismatch ab possible hi nahi
function renderForm(factory) {
  factory.createButton().render();
  factory.createCheckbox().render();
}

console.log('-- dark theme form --');
renderForm(getFactory('dark'));
console.log('-- light theme form --');
renderForm(getFactory('light'));

// Naya theme (e.g. "high-contrast") add karna = naya factory object,
// renderForm() ko CHHUE BINA.

```

```
// 🐛 EXERCISE – postgres aur mysql "families" (connection + query builder)
// ko Abstract Factory se consistent banao.
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
function createConnection(db) {
  return db === 'mysql' ? { name: 'MySQL conn' } : { name: 'Postgres conn' };
}
function createQueryBuilder(db) {
  return db === 'mysql' ? { name: 'MySQL builder' } : { name: 'Postgres builder' };
}

const conn = createConnection('mysql');
const builder = createQueryBuilder('postgres'); // 😬 mismatch possible
console.log(conn.name, '+', builder.name);

// =====
// TODO 1: `mysqlFactory` aur `postgresFactory` objects banao, dono mein
//         `createConnection()` aur `createQueryBuilder()` methods hon.
//
// TODO 2: `getFactory(db)` function banao jo sahi factory return kare.
//
// TODO 3: `setup(factory)` function likho jo EK factory leke connection +
//         builder dono banaye – ab mismatch structurally impossible hona
//         chahiye.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

4. Builder

Problem: ek object banane ke liye bohot saare (zyadatar optional) params chahiye. Constructor mein sabko positional pass karna unreadable hai aur order galat hone par silent bug ban jaata hai.

Idea: object ko ek baar mein nahi, **step-by-step methods** se banao. Har method ek field set karke `this` return kare (chaining), aakhir mein `build()` final object de.

Files — is order mein chalao

```
node 1-naive.js      # ❌ 6 positional params, order-sensitive
node 2-builder.js    # ✅ fluent chain, sirf jo chahiye set karo
node exercise.js     # 🔄 tumhara turn
```

Principles yahan dikh rahe hain

1. **Step-by-step construction** — complex object ek chain of readable calls se banta hai.
2. **Sensible defaults** — sirf jo customize karna hai woh set karo, baaki default rahe.

When to skip

Agar object mein 2-3 hi (mostly required) fields hain, plain object literal

`{ ... }` ya normal constructor kaafi hai — Builder tabhi kamaata hai jab fields zyada hon aur zyada optional hon.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — constructor mein bohot saare positional params,
// order yaad rakhna mushkil, optional cheezon ke liye undefined pass karna
// padta hai.
// Chalao: node 1-naive.js

class HttpRequest {
  constructor(method, url, headers, body, timeout, retries) {
    this.method = method; this.url = url; this.headers = headers;
    this.body = body; this.timeout = timeout; this.retries = retries;
  }
}
```

```
// Headers ya body nahi chahiye? undefined pass karna padega, order galat
// hui to bug – pata bhi nahi chalega (retries ki jagah timeout chala jaayega)
const req = new HttpRequest('POST', '/api/orders', undefined, { id: 1 }, 5000, undefined);
console.log(req);

// PROBLEM: 6 positional args yaad rakhna, optional wale skip karne ke liye
// undefined chain karna, aur order-mistake ka silently wrong behavior.
```

✓ 2-builder.js — the pattern

```
// ✓ BUILDER PATTERN – object ko step-by-step, readable chaining se banao.
// Chalao: node 2-builder.js
```

```
class HttpRequest {
  constructor() {
    this.method = 'GET'; this.headers = {}; this.retries = 0;
  }
}

class HttpRequestBuilder {
  #request = new HttpRequest();

  setMethod(method) { this.#request.method = method; return this; }
  setUrl(url) { this.#request.url = url; return this; }
  setHeader(key, val) { this.#request.headers[key] = val; return this; }
  setBody(body) { this.#request.body = body; return this; }
  setTimeout(ms) { this.#request.timeout = ms; return this; }
  setRetries(n) { this.#request.retries = n; return this; }
  build() { return this.#request; }
}
```

```
// Fluent chain – sirf jo chahiye woh set karo, order matter nahi karta
```

```
const req = new HttpRequestBuilder()
  .setMethod('POST')
  .setUrl('/api/orders')
  .setHeader('Content-Type', 'application/json')
  .setBody({ id: 1 })
  .setTimeout(5000)
  .build();
```

```
console.log(req);
```

```
// Naya optional field add karna = ek naya `set...()` method, purane
// callers ko CHHUE BINA (woh method call hi nahi karenge to default rahega).
```

🔗 exercise.js — your turn

```
// 🔗 EXERCISE – pizza order ko Builder pattern se refactor karo.
// Chalao: node exercise.js
```

```
// ----- naive version (isse refactor karna hai) -----
```

```
function makePizza(size, crust, toppings, extraCheese) {
  return { size, crust, toppings: toppings ?? [], extraCheese: !!extraCheese };
}
console.log(makePizza('large', 'thin', ['mushroom', 'olives'], true));

// =====
// TODO 1: `PizzaBuilder` class banao with methods: setSize(size),
//          setCrust(crust), addTopping(name), setExtraCheese(bool), build().
//          Har set/add method `this` return kare (chaining ke liye).
//
// TODO 2: default size 'medium', default crust 'regular' rakho constructor
//          mein, toppings [] se start karo.
//
// TODO 3 (bonus): agar `build()` call hone tak size set nahi hua (still
//                  default) to warn console.log karo – "using default size".
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// new PizzaBuilder().setSize('large').addTopping('mushroom').build()
// jaisa use hona chahiye.
// =====
```

5. Prototype

Problem: naya object banana expensive hai (heavy setup, nested config), lekin zyadatar naye objects kisi existing object se **milte-julte** hote hain — sirf 1-2 field alag.

Idea: ek "template" (prototype) object rakho. Naya instance scratch se banane ke bajaye prototype ko **clone** karo, phir jo chahiye woh change karo.

Files — is order mein chalao

```
node 1-naive.js      # ❌ har baar scratch se setup
node 2-prototype.js  # ✅ clone() se template reuse
node exercise.js     # 🔄 tumhara turn
```

Principles yahan dikh rahe hain

1. **Clone instead of construct** — setup cost ek hi baar, uske baad copy.
2. **Deep clone matters** — `structuredClone` (ya deep clone) use karo, warna nested objects/arrays accidentally share ho jaate hain (shallow `{...obj}` sirf top-level copy karta hai).

When to skip

Agar object banana already cheap hai (simple literal, no heavy setup), plain

`{ ... }` ya factory function likhna simpler hai — cloning ki extra indirection ki zaroorat nahi.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — similar object har baar scratch se, expensive setup
// duplicate ho raha hai.
// Chalao: node 1-naive.js

function createGoblin() {
  // pretend yeh expensive hai (asset load, stat calculation, etc.)
  return { type: 'goblin', hp: 30, speed: 5, loot: ['dagger'] };
}

const goblin1 = createGoblin();
```

```

const goblin2 = createGoblin();
goblin2.hp = 45; // is goblin ko thoda strong banana hai

console.log(goblin1);
console.log(goblin2);

// PROBLEM: har goblin poori setup logic dobara chalata hai, sirf ek field
// tweak karne ke liye bhi. Agar setup expensive ho (e.g. deep nested config,
// file load) to yeh wasteful hai.

```

✓ 2-prototype.js — the pattern

```

// ✓ PROTOTYPE PATTERN — ek baar "template" object banao, naye instances
// usse CLONE karke banao (expensive setup dobara nahi chalta).
// Chalao: node 2-prototype.js

// Class use karte hain taaki clone() method PROTOTYPE chain pe rahe, instance
// ki apni "own" data properties mein nahi — structuredClone(this) sirf data
// clone karta hai, functions clone nahi kar sakta (agar clone() khud ek own
// property hoti, jaisa plain object literal mein hota, to yeh crash karta).
class Goblin {
  constructor() {
    this.type = 'goblin'; this.hp = 30; this.speed = 5; this.loot = ['dagger'];
  }
  clone() {
    return structuredClone(this); // deep copy, prototype ke reference se independent
  }
}

const goblinPrototype = new Goblin(); // ek baar "expensive" setup

const goblin1 = goblinPrototype.clone();
const goblin2 = goblinPrototype.clone();
goblin2.hp = 45; // sirf clone mein change, prototype untouched

console.log(goblin1);
console.log(goblin2);
console.log('prototype still:', goblinPrototype.hp);

// Naya enemy type (orc) add karna = ek nayi class jo yehi clone() pattern
// follow kare, goblinPrototype ko CHHUE BINA.
class Orc {
  constructor() {
    this.type = 'orc'; this.hp = 60; this.speed = 3; this.loot = ['axe'];
  }
  clone() { return structuredClone(this); }
}

const orcPrototype = new Orc();
console.log(orcPrototype.clone());

```

🔗 exercise.js — your turn

```
// 🐛 EXERCISE – invoice template ko Prototype pattern se clone karo.
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
function createInvoice() {
  return {
    header: { company: 'Acme Inc', address: '123 Main St' },
    items: [],
    tax: 0.18,
  };
}

const invoiceA = createInvoice();
invoiceA.items.push({ name: 'Widget', price: 100 });
console.log(invoiceA);

// =====
// TODO 1: `InvoicePrototype` CLASS banao (constructor mein header + empty
//       items + tax set kare) with ek `clone()` method jo
//       `structuredClone(this)` return kare.
//       (Class use karo, plain object literal nahi – warna clone() khud
//       ek "own property" ban jaayegi aur structuredClone use clone nahi
//       kar payega kyunki functions clone-able nahi hote. Class method
//       prototype chain pe rehta hai, isliye safe hai – 2-prototype.js
//       mein yehi tarika dikhaya gaya hai.)
//
// TODO 2: `const invoicePrototype = new InvoicePrototype()` banao, phir do
//       invoices clone karo (`invoiceA`, `invoiceB`), dono mein alag items
//       push karo, dikhao ki ek doosre ko affect nahi karte (structuredClone
//       deep copy hai, shallow spread {...this} nahi).
//
// TODO 3 (bonus): socho – agar tumne `clone()` mein `{...this}` (shallow
//       copy) use kiya hota, to `items` array kya problem create karta?
//       (Hint: dono clones same array reference share karte.)
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

6. Adapter

Problem: tumhare app ko ek interface chahiye, lekin jo library/API mil rahi hai uska shape alag hai (different param order, different field names). Tum usko change nahi kar sakte (third-party/legacy).

Idea: ek thin wrapper (adapter) banao jo purani/mismatched interface ko apne app ke expected shape mein convert kare. App sirf apne expected interface se baat karta hai, adapter ke peeche kya hai — usse fark nahi padta.

Files — is order mein chalao

```
node 1-naive.js      # ❌ har jagah mismatch-handling duplicate
node 2-adapter.js    # ✅ ek wrapper, clean interface
node exercise.js     # 🎯 tumhara turn
```

Principles yahan dikh rahe hain

1. **Isolate the mismatch** — conversion logic ek hi jagah rehta hai.
2. **App depends on ITS interface, not theirs** — third-party SDK badle to sirf adapter badalta hai.

When to skip

Agar tum khud us API/library ke owner ho (usko directly badal sakte ho), adapter mat banao — seedha shape fix karo. Adapter tabhi zaroori hai jab tum kisi cheez ko control nahi kar sakte.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — third-party lib ka "alag shape" wala response
// direct use ho raha hai, jahan bhi use hota hai wahi conversion duplicate.
// Chalao: node 1-naive.js

// Pretend yeh ek third-party payment SDK hai jiska shape hum control nahi
// kar sakte:
const legacyPaymentSdk = {
  makeCharge: (cents) => ({ status_code: 200, charged_amount_cents: cents }),
};
```

```
function checkout(amountRupees) {
  const result = legacyPaymentSdk.makeCharge(amountRupees * 100);
  // har jagah jahan yeh SDK use hoga, yeh conversion + shape-reading duplicate hogi
  if (result.status_code === 200) {
    console.log(`✅ Charged ₹${result.charged_amount_cents / 100}`);
  }
}

checkout(499);

// PROBLEM: baaki poora app "amount in rupees, { success, amount }" shape
// expect karta hai, lekin yeh SDK cents + status_code deta hai. Yeh
// mismatch-handling code har call-site pe copy-paste hoga.
```

✅ 2-adapter.js — the pattern

```
// ✅ ADAPTER PATTERN — third-party ke ajeeb shape ko apne app ke expected
// interface mein convert karne wala ek wrapper banao.
// Chalao: node 2-adapter.js

const legacyPaymentSdk = {
  makeCharge: (cents) => ({ status_code: 200, charged_amount_cents: cents }),
};

// Adapter — apne app ke liye "clean" interface: pay(amountRupees) -> {success, amount}
class PaymentAdapter {
  pay(amountRupees) {
    const result = legacyPaymentSdk.makeCharge(amountRupees * 100);
    return {
      success: result.status_code === 200,
      amount: result.charged_amount_cents / 100,
    };
  }
}

function checkout(amountRupees, paymentGateway) {
  const result = paymentGateway.pay(amountRupees); // apna clean interface, SDK ka pata nahi
  if (result.success) console.log(`✅ Charged ₹${result.amount}`);
}

checkout(499, new PaymentAdapter());

// Kal ko SDK badal jaaye (Stripe, Razorpay) — sirf Adapter badlega,
// checkout() ka baaki poora app CHHUE BINA rahega.
```

🔗 exercise.js — your turn

```
// 🔗 EXERCISE — purani `oldLogger` (positional args) ko naye app ke expected
// `Log({level, message})` interface mein Adapter se convert karo.
// Chalao: node exercise.js

// ----- third-party jise hum control nahi kar sakte -----
```

```

const oldLogger = {
  write: (level, msg) => console.log(`[${level.toUpperCase()}] ${msg}`),
};

// ----- naive usage (isse refactor karna hai) -----
function reportError(msg) {
  oldLogger.write('error', msg); // direct coupling – har jagah 'level, msg' order yaad rakhna
}
reportError('Payment failed');

// =====
// TODO 1: `LoggerAdapter` class banao with ek method `log({ level, message })`
//          jo internally `oldLogger.write(level, message)` call kare.
//
// TODO 2: `reportError(msg, logger)` ko refactor karo taaki woh
//          `logger.log({ level: 'error', message: msg })` call kare –
//          `oldLogger` ka naam ab reportError() ke andar kahin na aaye.
//
// TODO 3 (bonus): ek doosra adapter socho jo console.log ko hi seedha
//                  `log({level, message})` interface de (no real "old" system) –
//                  dikhao ki reportError() dono adapters ke saath kaam karta hai.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====

```

7. Bridge

Problem: ek cheez do independent dimensions mein badalti hai (e.g. urgency × channel, shape × renderer). Har combination ke liye alag class banao to combinations ke saath class count **exponentially** badhta hai.

Idea: do dimensions ko DO alag hierarchies mein tod do — ek "abstraction" (kya) aur ek "implementation" (kaise) — aur abstraction implementation ko *hold* (reference) kare, inherit nahi kare. Ab dono independently badhte hain.

Files — is order mein chalao

```
node 1-naive.js    # ❌ (urgency × channel) har combo ke liye class
node 2-bridge.js   # ✅ do hierarchies, ek doosre ko hold karti hain
node exercise.js   # 🎯 tumhara turn
```

Principles yahan dikh rahe hain

1. **Composition over inheritance** — dusri dimension ko *inherit* nahi, *hold* karo.
2. **Independent extensibility** — naya channel ya nayi urgency level add karna doosri dimension ko touch nahi karta.

When to skip

Agar sirf EK dimension hai jo badalti hai (channel ya urgency, dono nahi), Strategy pattern (Lesson 1) kaafi hai — Bridge tabhi chahiye jab do dimensions ho jo independently combine hoti hon.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — har (notification-type × channel) combination ke
// liye alag class — class explosion.
// Chalao: node 1-naive.js

class UrgentEmailAlert {
  send(msg) { console.log(`🔔🔥 URGENT EMAIL: ${msg}`); }
}

class NormalEmailAlert {
  send(msg) { console.log(`🔔📧 email: ${msg}`); }
}
```

```

class UrgentSmsAlert {
  send(msg) { console.log(` 📠 🚨 URGENT SMS: ${msg}`); }
}
class NormalSmsAlert {
  send(msg) { console.log(` 📠 sms: ${msg}`); }
}

new UrgentEmailAlert().send('Server down');
new NormalSmsAlert().send('Deploy finished');

// PROBLEM: 2 urgency levels × 2 channels = 4 classes. Teesra channel (push)
// add karo -> 6 classes. Teesri urgency level add karo -> 9 classes. Yeh
// (urgency × channel) combinations ke saath EXPONENTIALLY badhta hai.

```

✅ 2-bridge.js — the pattern

```

// ✅ BRIDGE PATTERN – do dimensions (urgency × channel) ko DO alag
// hierarchies mein tod do, ek doosre ko "hold" karke connect karo.
// Chalao: node 2-bridge.js

// Implementation hierarchy – "kaise bheja jaaye" (channel)
const emailChannel = { deliver: (text) => console.log(` 📧 ${text}`) };
const smsChannel    = { deliver: (text) => console.log(` 📠 ${text}`) };

// Abstraction hierarchy – "kya bheja jaaye" (urgency), channel ko bas HOLD karta hai
class Alert {
  constructor(channel) { this.channel = channel; }
  send(msg) { this.channel.deliver(msg); }
}
class UrgentAlert extends Alert {
  send(msg) { this.channel.deliver(` 🚨 URGENT: ${msg}`); }
}

// Ab dono dimensions INDEPENDENTLY combine hote hain – class explosion nahi
new Alert(emailChannel).send('Deploy finished');
new UrgentAlert(smsChannel).send('Server down');
new UrgentAlert(emailChannel).send('Payment failed');

// Naya channel (push) add karna = 1 naya object. Nayi urgency level add
// karna = 1 nayi class. Dono independently badhte hain, combinations nahi.

```

🔧 exercise.js — your turn

```

// 🔧 EXERCISE – Shape (circle/square) × Renderer (vector/raster) ko Bridge
// se refactor karo taaki class explosion na ho.
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
class VectorCircle { draw() { console.log('vector circle ○'); } }
class RasterCircle { draw() { console.log('raster circle (pixels) ●'); } }
class VectorSquare { draw() { console.log('vector square □'); } }
class RasterSquare { draw() { console.log('raster square (pixels) ■'); } }

```

```
new VectorCircle().draw();
new RasterSquare().draw();

// =====
// TODO 1: do "renderer" objects banao – `vectorRenderer`, `rasterRenderer` –
//         dono mein ek `renderShape(shapeName)` method ho jo shape ka naam
//         leke apne style mein console.log kare.
//
// TODO 2: `Shape` base class banao jo constructor mein renderer le, aur
//         `Circle extends Shape`, `Square extends Shape` banao jo apna naam
//         renderer ko pass karein (draw() method se).
//
// TODO 3 (bonus): naya renderer 'asciiRenderer' add karo – dikhao ki Circle
//         aur Square dono classes ko CHHUE BINA naya renderer kaam karta hai.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

8. Composite

Problem: ek tree structure hai (files/folders, menu/combo, org chart) — kuch nodes "leaf" hain (single item), kuch "branch" hain (group jisme aur nodes hain). Har operation mein "leaf ya branch?" check karna padta hai.

Idea: leaf aur branch dono ko EK hi interface do (e.g. `getSize()`, `getPrice()`). Branch apna operation apne children pe recursively call karke implement karta hai. Caller ko fark hi nahi karna padta — woh bas `node.getSize()` call karta hai.

Files — is order mein chalao

```
node 1-naive.js      # ❌ har operation mein type-check duplicate
node 2-composite.js  # ✅ File aur Folder same interface follow karte hain
node exercise.js     # 🌀 tumhara turn
```

Principles yahan dikh rahe hain

1. **Uniform treatment** — single object aur group ko same tarique se treat karo.
2. **Recursive composition** — branch apna kaam apne children ko delegate karke karta hai.

When to skip

Agar structure kabhi nested nahi hoga (sirf flat list), plain array +

`.reduce()` kaafi hai — Composite ki recursion tabhi chahiye jab genuinely tree-shaped data ho.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — file aur folder ko har jagah alag treat karna padta
// hai, har naye operation ke liye type-check duplicate hota hai.
// Chalao: node 1-naive.js

function getSize(node) {
  if (node.type === 'file') return node.size;
  if (node.type === 'folder') {
    let total = 0;
    for (const child of node.children) total += getSize(child); // recursion, par type-check
```

```

saath
    return total;
}
throw new Error('unknown node type');
}

const tree = {
  type: 'folder', name: 'src',
  children: [
    { type: 'file', name: 'index.js', size: 10 },
    { type: 'folder', name: 'utils', children: [{ type: 'file', name: 'math.js', size: 5 } ] },
  ],
};

console.log('total size:', getSize(tree));

// PROBLEM: har naya operation (print, count files, search) getSize() jaisa
// har jagah "file vs folder" if/else dobara likhega. Caller ko bhi hamesha
// yaad rakhna padta hai ki node file hai ya folder.

```

✅ 2-composite.js — the pattern

```

// ✅ COMPOSITE PATTERN — File aur Folder dono EK hi interface follow karte
// hain (getSize()), caller ko fark nahi karna padta "leaf" hai ya "branch".
// Chalao: node 2-composite.js

class File {
  constructor(name, size) { this.name = name; this.size = size; }
  getSize() { return this.size; }
  print(indent = '') { console.log(`${indent} 📄 ${this.name} (${this.size})`); }
}

class Folder {
  constructor(name) { this.name = name; this.children = []; }
  add(node) { this.children.push(node); return this; }
  getSize() { return this.children.reduce((sum, c) => sum + c.getSize(), 0); } // same method
  name!
  print(indent = '') {
    console.log(`${indent} 📁 ${this.name}/`);
    this.children.forEach((c) => c.print(indent + '  '));
  }
}

const utils = new Folder('utils').add(new File('math.js', 5));
const tree = new Folder('src')
  .add(new File('index.js', 10))
  .add(utils);

tree.print();
console.log('total size:', tree.getSize());

// Naya operation (print) add karna = File aur Folder dono mein ek method,
// caller code (`tree.print()`) file/folder ka fark kiye bina kaam karta hai.

```

exercise.js — your turn

```
// 🐛 EXERCISE – restaurant menu (single item vs combo/submenu) ko Composite
// se refactor karo taaki total price recursively, uniformly nikle.
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
function getPrice(entry) {
  if (entry.kind === 'item') return entry.price;
  if (entry.kind === 'combo') return entry.items.reduce((sum, e) => sum + getPrice(e), 0);
  throw new Error('unknown kind');
}

const menu = {
  kind: 'combo', name: 'Family Meal',
  items: [
    { kind: 'item', name: 'Burger', price: 150 },
    { kind: 'combo', name: 'Sides', items: [{ kind: 'item', name: 'Fries', price: 50 }] },
  ],
};
console.log('total price:', getPrice(menu));

// =====
// TODO 1: `MenuItem` class banao (name, price) with `getPrice()` method
//         jo price return kare.
//
//
// TODO 2: `ComboMeal` class banao (name, empty children array) with
//         `add(entry)` aur `getPrice()` (children ka sum, recursively –
//         chahe entry MenuItem ho ya nested ComboMeal, same method call hota
//         hai) methods.
//
//
// TODO 3 (bonus): dono classes mein `print(indent)` method add karo jo tree
//                 ko nicely console.log kare (2-composite.js jaisa).
//
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

9. Decorator

Problem: ek base object mein runtime pe extra behavior/features jodne hain (coffee add-ons, text formatting, middleware), lekin har combination ke liye subclass banana class-explosion ban jaata hai.

Idea: har add-on ko ek wrapper object banao jo asli object jaisa hi interface follow kare, aur wrapped object ko call karke apna kaam upar se jode. Decorators ko **kisi bhi order mein, kitne bhi stack** kiya ja sakta hai.

Files — is order mein chalao

```
node 1-naive.js      # ❌ har combination ke liye subclass
node 2-decorator.js  # ✅ stackable wrapper objects
node exercise.js     # 🎯 tumhara turn
```

Principles yahan dikh rahe hain

1. **Same interface as wrapped object** — decorator aur base dono `cost()` / `describe()` follow karte hain, caller ko fark nahi padta.
2. **Composable at runtime** — combinations **code** mein nahi, **chaining** se banti hain.

When to skip

Agar sirf 1-2 fixed variants hain jo kabhi combine nahi hongy, simple subclass ya if/else kaafi hai — Decorator tabhi kamaata hai jab **independent add-ons ko mix-and-match** karna ho.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — har add-on combination ke liye alag subclass —
// class explosion (Bridge Lesson jaisa hi problem, dusre context mein).
// Chalao: node 1-naive.js

class Coffee { cost() { return 100; } describe() { return 'Coffee'; } }
class CoffeeWithMilk extends Coffee {
  cost() { return super.cost() + 20; }
  describe() { return super.describe() + ' + Milk'; }
}
class CoffeeWithMilkAndSugar extends CoffeeWithMilk {
```

```

    cost() { return super.cost() + 10; }
    describe() { return super.describe() + ' + Sugar'; }
  }

const order = new CoffeeWithMilkAndSugar();
console.log(order.describe(), '=', order.cost());

// PROBLEM: Milk, Sugar, Whip, ExtraShot – 4 add-ons ke 2^4 = 16 possible
// combinations, har combo ke liye subclass nahi bana sakte.

```

✓ 2-decorator.js — the pattern

```

// ✓ DECORATOR PATTERN – add-ons ko wrapper objects banao jo SAME interface
// (cost/describe) follow karte hain aur asli object ko "wrap" karte hain.
// Chalao: node 2-decorator.js

class Coffee {
  cost() { return 100; }
  describe() { return 'Coffee'; }
}

// Har decorator wahi interface (cost/describe) follow karta hai, aur jo
// wrap kiya hai uske result mein apna add-on jodta hai
class MilkDecorator {
  constructor(coffee) { this.coffee = coffee; }
  cost() { return this.coffee.cost() + 20; }
  describe() { return this.coffee.describe() + ' + Milk'; }
}
class SugarDecorator {
  constructor(coffee) { this.coffee = coffee; }
  cost() { return this.coffee.cost() + 10; }
  describe() { return this.coffee.describe() + ' + Sugar'; }
}

// Ab koi bhi combination RUNTIME pe chain ki jaa sakti hai, subclass nahi chahiye
let order = new Coffee();
order = new MilkDecorator(order);
order = new SugarDecorator(order);

console.log(order.describe(), '=', order.cost());

// Naya add-on (whip) = ek naya decorator class, existing decorators ko
// CHHUE BINA, aur kisi bhi order mein stack ho sakta hai.

```

🔗 exercise.js — your turn

```

// 🔗 EXERCISE – text formatters (bold/italic) ko Decorator pattern se
// refactor karo taaki koi bhi combination stack ho sake.
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
function boldItalic(text) { return `**_${text}_**`; }

```

```
function boldOnly(text) { return `**${text}**`; }
// Naya combo (underline+bold) chahiye? Naya function likhna padega...

console.log(boldItalic('Hello'));

// =====
// TODO 1: `PlainText` class banao with constructor(text) aur
//         `render()` method jo bas text return kare.
//
// TODO 2: `BoldDecorator` aur `ItalicDecorator` classes banao, dono
//         constructor mein "wrapped" text-object lein aur `render()` mein
//         `wrapped.render()` ke around apna marker (** ya _) laga dein.
//
// TODO 3 (bonus): `UnderlineDecorator` add karo aur teeno ko kisi bhi order
//         mein stack karke dikhao – dikhao ki order badalne se output ka
//         "nesting order" badalta hai.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

10. Facade

Problem: ek kaam poora karne ke liye kai subsystems ko sahi order mein call karna padta hai (decode → normalize → encode; reserve → charge → ship).

Har caller ko yeh poora orchestration aur order yaad rakhna padta hai.

Idea: subsystem ke aage ek simple object/function rakho jo saare steps internally sahi order mein chalayen. Caller sirf ek method call karta hai, subsystem ke internals se bekhabar.

Files — is order mein chalao

```
node 1-naive.js    # ❌ caller khud subsystem orchestrate karta hai
node 2-facade.js   # ✅ ek simple entry point, internals chhupe hue
node exercise.js   # 🎯 tumhara turn
```

Principles yahan dikh rahe hain

1. **Hide complexity, don't remove it** — subsystem waisa hi rehta hai, bas uske use karne ka tarika simple ho jaata hai.
2. **Single entry point** — orchestration logic ek hi jagah, duplicate nahi hoti har caller mein.

When to skip

Agar subsystem already simple hai (1-2 calls), facade ek extra layer hi hai jo kuch nahi kamaata — sirf tab banao jab **orchestration genuinely complex** ho aur multiple callers ho.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — caller ko poore subsystem (codec, audio, subtitle)
// ke saare steps aur unka sahi order khud yaad rakhna padta hai.
// Chalao: node 1-naive.js

const codec = { decode: (file) => `decoded(${file})`, encode: (data, fmt) => `${data}.${fmt}` };
const audioMixer = { normalize: (data) => `normalized(${data})` };
const subtitleEngine = { burnIn: (data, subs) => (subs ? `${data}+subs` : data) };

function convertVideo(file, format, subs) {
  const decoded = codec.decode(file);
```

```

    const normalized = audioMixer.normalize(decoded);
    const withSubs = subtitleEngine.burnIn(normalized, subs);
    return codec.encode(withSubs, format);
  }

  console.log(convertVideo('movie.mkv', 'mp4', true));

  // PROBLEM: caller (ya iske UI/CLI) ko subsystem ke saare internal steps
  // (decode -> normalize -> subs -> encode) aur unka sahi order pata hona
  // chahiye. Har naya caller yeh poora orchestration dobara likhega.

```

✅ 2-facade.js — the pattern

```

// ✅ FACADE PATTERN – poore complex subsystem ke aage ek SIMPLE, single
// entry-point object rakho. Caller ko internal steps ka pata hi nahi hona
// chahiye.
// Chalao: node 2-facade.js

const codec = { decode: (file) => `decoded(${file})`, encode: (data, fmt) => `${data}.${fmt}` };
const audioMixer = { normalize: (data) => `normalized(${data})` };
const subtitleEngine = { burnIn: (data, subs) => (subs ? `${data}+subs` : data) };

// Facade – subsystem ke saare steps ko ek simple method ke peeche chhupata hai
class VideoConverterFacade {
  convert(file, format, subs = false) {
    const decoded = codec.decode(file);
    const normalized = audioMixer.normalize(decoded);
    const withSubs = subtitleEngine.burnIn(normalized, subs);
    return codec.encode(withSubs, format);
  }
}

const converter = new VideoConverterFacade();
console.log(converter.convert('movie.mkv', 'mp4', true));
console.log(converter.convert('clip.avi', 'webm'));

// Caller ko codec/audioMixer/subtitleEngine ke naam tak pata nahi – sirf
// EK simple method call karta hai. Subsystem internally reorganize ho sakta
// hai, facade ka interface same rehta hai.

```

🔧 exercise.js — your turn

```

// 🔧 EXERCISE – order placement (inventory check + payment + shipping) ko
// Facade se ek simple `placeOrder()` ke peeche chhupao.
// Chalao: node exercise.js

// ----- subsystems (already given, inhe badalna nahi hai) -----
const inventory = { reserve: (item) => console.log(`📦 reserved: ${item}`) };
const payment = { charge: (amount) => console.log(`💵 charged ₹${amount}`) };
const shipping = { schedule: (item) => console.log(`🚚 shipping scheduled: ${item}`) };

// ----- naive usage (isse refactor karna hai) -----

```

```
function buyNow(item, amount) {
  inventory.reserve(item);
  payment.charge(amount);
  shipping.schedule(item);
}
buyNow('Headphones', 1999);

// =====
// TODO 1: `OrderFacade` class banao with ek method `placeOrder(item, amount)`
//         jo internally inventory.reserve -> payment.charge -> shipping.schedule
//         (yahi order mein) call kare.
//
// TODO 2: `buyNow` function ko hata kar `new OrderFacade().placeOrder(...)`
//         use karo.
//
// TODO 3 (bonus): agar `payment.charge` fail ho jaaye (throw kare) to socho -
//                 kya inventory.reserve() undo hona chahiye? Ek naya method
//                 `inventory.release(item)` maan lo aur facade mein try/catch se
//                 rollback add karo.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

11. Flyweight

Problem: bohot saare (hazaron/lakhon) similar objects banane hain, aur har object apna heavy/shared data (texture, font, config) alag se duplicate karta hai — memory waste hota hai.

Idea: jo data **shared** ho sakta hai (same type ke sab objects ke liye same) usse alag nikaal ke ek jagah reuse karo. Har instance mein sirf **unique** data (position, jaisi cheezein) rakho.

Files — is order mein chalao

```
node 1-naive.js      # ❌ har object apna shared data duplicate karta hai
node 2-flyweight.js  # ✅ shared data ek factory se reuse hota hai
node exercise.js     # 🎯 tumhara turn
```

Principles yahan dikh rahe hain

1. **Intrinsic vs extrinsic state** — intrinsic (shared, type-level) alag, extrinsic (unique, instance-level) alag.
2. **Factory ensures reuse** — naya object banane se pehle check karo ki shared version already exist karta hai kya.

When to skip

Agar objects ki count chhoti hai (sainkdo, hazaron nahi) ya shared data heavy nahi hai, yeh optimization ki zaroorat nahi — premature optimization se bachna behtar hai.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — har tree object apna poora heavy "texture" data
// duplicate karta hai, hazaron trees banate hi memory phat jaayegi.
// Chalao: node 1-naive.js

class Tree {
  constructor(x, y, type) {
    this.x = x; this.y = y;
    this.type = type;
    // pretend yeh heavy shared data hai (mesh, texture...) — har tree isko duplicate karta hai
  }
}
```

```

    this.texture = `${type}-texture-data-(pretend this is large)`;
    this.mesh = `${type}-mesh-data-(pretend this is large)`;
  }
  render() { console.log(`🌳 ${this.type} at (${this.x},${this.y}) using ${this.texture}`); }
}

const forest = [];
for (let i = 0; i < 5; i++) forest.push(new Tree(i, i * 2, 'oak'));
forest.forEach((t) => t.render());
console.log('trees created:', forest.length, '- har ek apna texture+mesh copy leke');

// PROBLEM: 'oak' texture/mesh SAME hai sab trees ke liye, phir bhi har Tree
// apna alag copy rakhta hai. 1 Lakh trees banao to woh data 1 Lakh baar
// duplicate hoga - memory waste.

```

✅ 2-flyweight.js — the pattern

```

// ✅ FLYWEIGHT PATTERN - shared/heavy data (texture, mesh) ko ALAG rakho aur
// same-type objects ke beech REUSE karo. Sirf unique data (x, y) har
// instance mein rehta hai.
// Chalao: node 2-flyweight.js

// Flyweight factory - same type ke liye same shared object reuse karta hai
class TreeTypeFactory {
  static #types = new Map();
  static getType(type) {
    if (!TreeTypeFactory.#types.has(type)) {
      console.log(` (creating shared texture+mesh for '${type}' - sirf EK baar)`);
      TreeTypeFactory.#types.set(type, {
        texture: `${type}-texture-data`,
        mesh: `${type}-mesh-data`,
      });
    }
    return TreeTypeFactory.#types.get(type); // same object wapas milta hai
  }
  static count() { return TreeTypeFactory.#types.size; }
}

// Tree ab sirf UNIQUE data (x, y) rakhta hai, shared data ko reference karta hai
class Tree {
  constructor(x, y, type) {
    this.x = x; this.y = y;
    this.sharedData = TreeTypeFactory.getType(type); // reused, duplicate nahi
  }
  render() { console.log(`🌳 tree at (${this.x},${this.y}) using ${this.sharedData.texture}`); }
}

const forest = [];
for (let i = 0; i < 5; i++) forest.push(new Tree(i, i * 2, 'oak'));
forest.forEach((t) => t.render());

console.log('unique shared objects created:', TreeTypeFactory.count(), '(sirf 1, chahe 5 trees hon)');

```

exercise.js — your turn

```
// 🐛 EXERCISE – document mein har character ka "font style" (heavy, shared)
// ko Flyweight se reuse karo, sirf position+char unique rakho.
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
class Character {
  constructor(char, x, y, font) {
    this.char = char; this.x = x; this.y = y;
    this.fontData = { family: font, size: 12, weight: 'normal' }; // har character apna copy
  }
}

const text = 'Hi';
const chars = [...text].map((c, i) => new Character(c, i, 0, 'Arial'));
console.log(chars.length, 'characters, har ek apna fontData copy leke');

// =====
// TODO 1: `FontFactory` class banao (static Map + `getFont(family)` method)
//         jo same family ke liye same shared `{family, size, weight}` object
//         return kare (2-flyweight.js jaisa).
//
// TODO 2: `Character` class ko refactor karo taaki woh apna `fontData`
//         khud na banaye, balki `FontFactory.getFont(font)` se le.
//
// TODO 3 (bonus): dikhao ki 1000 characters banane par bhi sirf 1 shared
//         font object banta hai (console.log se count verify karo).
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

12. Proxy

Problem: asli object expensive hai (load slow, network call), ya usmein access-control chahiye — lekin caller ko asli object jaisa hi interface chahiye, sirf "kab aur kaise access ho" pe control chahiye.

Idea: ek "stand-in" object banao jo asli object jaisa hi interface follow kare. Proxy decide karta hai — lazily load kare, cache kare, ya access allow/deny kare — phir kaam asli object ko delegate kare.

Files — is order mein chalao

```
node 1-naive.js    # ❌ eager load, no access control
node 2-proxy.js   # ✅ lazy load + access control, same interface
node exercise.js  # 🎯 tumhara turn
```

Principles yahan dikh rahe hain

1. **Same interface as real object** — caller ko fark nahi padta proxy hai ya asli object.
2. **Control before delegate** — proxy pehle decide karta hai (load? cache? allow?), phir kaam asli object ko sonpta hai.

When to skip

Agar object cheap hai aur access-control/caching/lazy-loading ki zaroorat nahi, seedha real object use karo — Proxy sirf tab kamaata hai jab "access ko control karna" genuinely valuable ho.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — heavy image har baar directly load hoti hai, chahe
// use ho ya na ho, aur koi access-control nahi.
// Chalao: node 1-naive.js

class HighResImage {
  constructor(file) {
    this.file = file;
    console.log(`⌚ loading heavy image: ${file} (pretend this is slow)`);
  }
  display() { console.log(`🖼️ showing ${this.file}`); }
}
```

```

}

console.log('gallery created (but image already loaded even before display!');
const image = new HighResImage('photo.png'); // Load turant ho gaya
image.display();
image.display(); // dobara Load nahi hota, but pehle hi ho chuka tha bina zaroorat

// PROBLEM: image object banate hi expensive load ho jaata hai, chahe display()
// kabhi call ho ya na ho. Aur koi bhi is object ko directly access kar sakta
// hai – permission check ka koi jagah nahi.

```

✓ 2-proxy.js — the pattern

```

// ✓ PROXY PATTERN – asli object ke aage ek "stand-in" rakho jo access
// control kare, aur loading ko TAB tak taale jab tak genuinely zaroorat na
// ho (lazy loading).
// Chalao: node 2-proxy.js

class HighResImage {
  constructor(file) {
    this.file = file;
    console.log(`⌚ loading heavy image: ${file} (pretend this is slow)`);
  }
  display() { console.log(`🖼️ showing ${this.file}`); }
}

// Proxy – same interface (display()) but real object ko lazily banata hai
class ImageProxy {
  #realImage = null;
  constructor(file, userRole) { this.file = file; this.userRole = userRole; }
  display() {
    if (this.userRole !== 'admin') {
      console.log('🚫 access denied – admin only');
      return;
    }
    if (!this.#realImage) this.#realImage = new HighResImage(this.file); // pehli baar hi load
    this.#realImage.display();
  }
}

console.log('gallery created (image NOT loaded yet)');
const image = new ImageProxy('photo.png', 'admin');
image.display(); // ab load hota hai
image.display(); // cached – dobara load nahi hota

const guestImage = new ImageProxy('secret.png', 'guest');
guestImage.display(); // access denied, load hi nahi hota

```

🔧 exercise.js — your turn

```

// 🔧 EXERCISE – repeated API calls ko Proxy se cache karo taaki same request
// dobara network na kare.

```

```
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
let callCount = 0;
function fetchWeather(city) {
  callCount++;
  console.log(`🌐 network call #${callCount} for ${city}`);
  return { city, tempC: 28 };
}
fetchWeather('Delhi');
fetchWeather('Delhi'); // same city – phir bhi naya network call

// =====
// TODO 1: `WeatherProxy` class banao with internal `#cache` (Map).
//       `get(city)` method: agar cache mein hai to wahi return karo, warna
//       `fetchWeather(city)` call karke result cache mein daalo aur return
//       karo.
//
// TODO 2: proxy use karke dikhao ki 'Delhi' do baar maangne par network call
//       sirf EK baar ho.
//
// TODO 3 (bonus): `invalidate(city)` method add karo jo cache se ek city
//       hata de – dikhao ki uske baad fresh network call hota hai.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

13. Strategy

Problem: ek kaam kai tarikon se ho sakta hai, aur kaunsa tarika use karna hai woh runtime pe decide hota hai (payment method, export format, shipping cost...).

Idea: har variant ko apne function/object mein nikaalo jo ek **common contract** follow kare. Caller ko sirf woh reference pakdao jo use chahiye. Naya variant add karna = naya object likhna, purana code chhue bina.

Files — is order mein chalao

```
node 1-naive.js          # ❌ if/else wali problem
node 2-strategy-class.js # ✅ class + strategy objects
node 3-strategy-registry.js # ✅ idiomatic JS: object as registry (best for JS)
node exercise.js         # 🎯 tumhara turn
```

Teen principles yahan dikh rahe hain

1. **Encapsulate what varies** — har payment method ka logic alag.
2. **Program to an interface** — sab `pay(amount)` contract follow karte hain.
3. **Composition over inheritance** — Checkout strategy ko **hold** karta hai, inherit nahi karta.

When to skip

Sirf 2 variants hain jo kabhi badhenge nahi? Plain `if` behtar — pattern ki indirection ki cost mat uthao.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — sab kuch ek function mein if/else ke saath.
// Chalao: node 1-naive.js

// Mock payment services (pretend ye real APIs call karte hain)
const processStripe = (amt) => `✅ Charged ₹${amt} via Stripe`;
const processPaypal = (amt) => `✅ Charged ₹${amt} via PayPal`;
const processCrypto = (amt) => `✅ Charged ₹${amt} via Crypto`;

function checkout(cart, method) {
  if (method === 'card') return processStripe(cart.total);
  if (method === 'paypal') return processPaypal(cart.total);
```

```

    if (method === 'crypto') return processCrypto(cart.total);
    throw new Error(`Unknown method: ${method}`);
  }

  const cart = { total: 4999 };
  console.log(checkout(cart, 'card'));
  console.log(checkout(cart, 'paypal'));
  console.log(checkout(cart, 'crypto'));

  // PROBLEM: naya method (Apple Pay) add karne ke liye is function ko DOBARA
  // kholna padega. Jitne payment ho sakte hain, utni jagah ye if/else copy hota
  // hai. Ek path ko akele test karna bhi mushkil. => Strategy pattern isse theek karta hai.

```

✓ 2-strategy-class.js — the pattern

```

// ✓ STRATEGY PATTERN (class form)
// Chalao: node 2-strategy-class.js

const processStripe = (amt) => `✓ Charged ₹${amt} via Stripe`;
const processPaypal = (amt) => `✓ Charged ₹${amt} via PayPal`;
const processCrypto = (amt) => `✓ Charged ₹${amt} via Crypto`;

// Har strategy ek hi contract follow karti hai: pay(amount)
const cardStrategy = { pay: (amount) => processStripe(amount) };
const paypalStrategy = { pay: (amount) => processPaypal(amount) };
const cryptoStrategy = { pay: (amount) => processCrypto(amount) };

// "Context" sirf delegate karta hai — use pata nahi kaunsi strategy hai
class Checkout {
  constructor(strategy) { this.strategy = strategy; }
  setStrategy(strategy) { this.strategy = strategy; }
  complete(amount) { return this.strategy.pay(amount); }
}

const checkout = new Checkout(cardStrategy);
console.log(checkout.complete(4999));

// Runtime pe behavior swap:
checkout.setStrategy(cryptoStrategy);
console.log(checkout.complete(4999));

// Naya method add karna = naya object, Checkout ko CHHUE BINA:
const applePayStrategy = { pay: (amount) => `✓ Charged ₹${amount} via Apple Pay` };
checkout.setStrategy(applePayStrategy);
console.log(checkout.complete(4999));

```

✓ 3-strategy-registry.js — the pattern

```

// ✓ IDIOMATIC JS: object-as-registry. JS mein aksar YEHI best hai.
// Chalao: node 3-strategy-registry.js

const processStripe = (amt) => `✓ Charged ₹${amt} via Stripe`;

```

```

const processPaypal = (amt) => `✅ Charged ₹${amt} via PayPal`;
const processCrypto = (amt) => `✅ Charged ₹${amt} via Crypto`;

// Functions first-class hain => strategy bas ek function, registry bas ek object.
const strategies = {
  card: (amount) => processStripe(amount),
  paypal: (amount) => processPaypal(amount),
  crypto: (amount) => processCrypto(amount),
};

function checkout(method, amount) {
  const pay = strategies[method];
  if (!pay) throw new Error(`Unknown method: ${method}`);
  return pay(amount);
}

console.log(checkout('card', 4999));
console.log(checkout('paypal', 4999));

// Apple Pay add karna = EK line. if/else ko chhue bina:
strategies.applePay = (amount) => `✅ Charged ₹${amount} via Apple Pay`;
console.log(checkout('applePay', 4999));

// Yahi tarika hai jisse experienced JS dev naturally if/else chain khatam karta hai.

```

exercise.js — your turn

```

// 🛠️ EXERCISE — ise Strategy pattern (registry form) mein refactor karo.
// Chalao: node exercise.js
//
// Rules:
//   'standard' -> flat ₹50
//   'express' -> ₹50 + ₹10 per kg
//   'overnight' -> ₹200 flat

// ----- naive version (isse refactor karna hai) -----
function shippingCost(order, method) {
  if (method === 'standard') return 50;
  if (method === 'express') return 50 + 10 * order.weightKg;
  if (method === 'overnight') return 200;
  throw new Error(`Unknown method: ${method}`);
}

const order = { weightKg: 3, total: 1200 };
console.log('standard:', shippingCost(order, 'standard'));
console.log('express:', shippingCost(order, 'express'));
console.log('overnight:', shippingCost(order, 'overnight'));

// =====
// TODO 1: upar wale ko `const strategies = { ... }` + chhote dispatch
//         function mein badlo. Har strategy ko (order) milega.
//
// TODO 2 (bonus): 'express-free-above-1000' add karo — agar order.total > 1000

```

```
//      to express free (₹0), warna normal express charge.
//      Sawaal: naya rule add karte waqt kya PURANI strategies ko chhuna pada?
//      (Sahi jawab: nahi. Yehi open/closed ka fayda hai.)
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====

// const strategies = {
//   ...
// };
// function shipping(method, order) { ... }
// console.log(shipping('express', order));
```

14. Observer

Problem: ek object ki state badalti hai, aur **kai doosre** objects ko react karna hai — par source ko yeh nahi jaanna chahiye ki kaun sun raha hai.
(order shipped → email + SMS + analytics + inventory...)

Idea: Subject sirf ek list rakhta hai interested logon ki (Observers), aur change pe unhe **notify** kar deta hai — bina jaane woh karte kya hain.

Files — is order mein chalao

```
node 1-naive.js      # ❌ Order har service se coupled
node 2-observer.js   # ✅ subscribe / notify / unsubscribe khud banaya
node 3-eventemitter.js # ✅ Node ka built-in EventEmitter = Observer pattern
node exercise.js     # 🎯 tumhara turn
```

Ye JS mein built-in hai

- Node: `EventEmitter` — `on` = subscribe, `emit` = notify, `off` = unsubscribe
- Browser: `addEventListener` / `dispatchEvent`

2 khatre (exercise mein dhyaan do)

1. **Execution order guarantee nahi** — agar B ko A ke baad chalna zaroori hai, Observer galat tool hai.
2. **Memory leak** — `unsubscribe` bhoole to subject observer ko pakde rehta hai aur woh garbage-collect nahi hota.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — Order khud sabko directly call karta hai.
// Chalao: node 1-naive.js

const sendShippedEmail = (o) => console.log(`📧 Email: order ${o.id} shipped`);
const smsNotify         = (o) => console.log(`📱 SMS: order ${o.id} shipped`);
const analyticsTrack    = (evt, o) => console.log(`📊 Analytics: ${evt} (${o.id})`);
const inventoryDecrement = (n) => console.log(`📦 Inventory down by ${n} items`);

class Order {
  constructor(id, items) { this.id = id; this.items = items; }
  ship() {
    this.status = 'shipped';
```

```

    // Order ab in SAB services ke baare mein jaanta hai – tight coupling
    sendShippedEmail(this);
    smsNotify(this);
    analyticsTrack('order_shipped', this);
    inventoryDecrement(this.items);
  }
}

new Order('A1', 2).ship();

// PROBLEM: naya reaction (loyalty points) add karna = Order.ship() ko dobara
// kholna. Order test karne ke liye ye sab mock karne padenge. Dependency ulti hai:
// core business object apne peripheral consumers ko jaanta hai.

```

✓ 2-observer.js — the pattern

```

// ✓ OBSERVER PATTERN (khud banaya)
// Chalao: node 2-observer.js

class Subject {
  #observers = new Set();

  // subscribe ek "unsubscribe" function return karta hai – cleanup aasan
  subscribe(observer) {
    this.#observers.add(observer);
    return () => this.unsubscribe(observer);
  }
  unsubscribe(observer) { this.#observers.delete(observer); }
  notify(data) {
    for (const observer of this.#observers) observer.update(data);
  }
}

class Order extends Subject {
  constructor(id, items) { super(); this.id = id; this.items = items; }
  ship() {
    this.status = 'shipped';
    this.notify({ event: 'shipped', order: this }); // bas. kaun sun raha, pata nahi.
  }
}

// Har observer sirf ek contract follow karta hai: update(data)
const emailObserver = { update: ({ order }) => console.log(`✉ Email: ${order.id}`) };
const inventoryObserver = { update: ({ order }) => console.log(`📦 Inventory: -${order.items}`) };

const order = new Order('A1', 2);
order.subscribe(emailObserver);
const stopInventory = order.subscribe(inventoryObserver);

order.ship(); // dono chalte hain

console.log('--- inventory unsubscribe karne ke baad ---');
stopInventory(); // ab inventory nahi sunega

```

```
order.ship(); // sirf email chalega

// Naya reaction add karna = naya observer + subscribe. Order ko CHHUE BINA.
```

✓ 3-eventemitter.js — the pattern

```
// ✓ IDIOMATIC JS: Node ka EventEmitter = Observer pattern, built-in.
// Chalao: node 3-eventemitter.js

import { EventEmitter } from 'node:events';

class Order extends EventEmitter {
  constructor(id, items) { super(); this.id = id; this.items = items; }
  ship() {
    this.status = 'shipped';
    this.emit('shipped', this); // notify()
  }
}

const order = new Order('A1', 2);

order.on('shipped', (o) => console.log(`📧 Email: ${o.id}`)); // subscribe
order.on('shipped', (o) => console.log(`📦 Inventory: -${o.items}`)); // subscribe

const analytics = (o) => console.log(`📊 Analytics: ${o.id}`);
order.on('shipped', analytics);

order.ship();

console.log('--- analytics off karne ke baad ---');
order.off('shipped', analytics); // unsubscribe
order.ship();

// on = subscribe, emit = notify, off = unsubscribe. Naam alag, pattern wahi.
```

🔧 exercise.js — your turn

```
// 🎯 EXERCISE – ek StockPrice subject banao Observer pattern se.
// Chalao: node exercise.js
//
// Chahiye:
// - logger: har price change print kare ("📈 Price: 105")
// - alerter: sirf tab print kare jab price 100 ke UPAR cross kare ("⚠️ ALERT: 105")
//
// Do tarike se try karo (dono seekhne layak):
// A) khud Subject class banake (dekho 2-observer.js)
// B) EventEmitter extend karke (dekho 3-eventemitter.js)

// ---- yahan implement karo ----
class StockPrice /* extends ??? */ {
  constructor() {
    this.price = 0;
  }
}
```

```
}
setPrice(newPrice) {
  // TODO: purani price yaad rakho (cross detect karne ke liye)
  // TODO: price update karo
  // TODO: observers ko notify karo
}
}

// ---- test (implement karne ke baad uncomment karo) ----
// const stock = new StockPrice();
// stock.subscribe(/* logger */);
// stock.subscribe(/* alerter */);
// stock.setPrice(95); // logger: 95
// stock.setPrice(105); // logger: 105 + alerter: ⚠ ALERT (100 cross hua)
// stock.setPrice(110); // logger: 110 (koi alert nahi – pehle se upar tha)

console.log('Exercise file – apna code likho aur test uncomment karo.');
```

15. Command

Problem: ek "action" (button press, menu click, editor edit) ko turant execute karna hai, lekin baad mein **undo karna, queue mein daalna**, ya **log karke replay karna** bhi chahiye — aur caller (remote/UI) ko device ke internals nahi pata hone chahiye.

Idea: action ko khud ek object bana do — `execute()` aur `undo()` wala.

Invoker (remote/menu) sirf command ko **hold** karta hai aur call karta hai, uska implementation nahi jaanta.

Files — is order mein chalao

```
node 1-naive.js      # ❌ direct method calls, undo impossible
node 2-command.js    # ✅ Command objects + undo
node 3-command-queue.js # ✅ macro, queue, replay
node exercise.js     # 🎯 tumhara turn
```

Principles yahan dikh rahe hain

1. **Encapsulate a request as an object** — action + uska undo ek jagah.
2. **Decouple invoker from receiver** — RemoteControl ko Light ka pata nahi.
3. Ek baar action object ban jaaye to use **queue, log, ya batch** kar sakte ho.

When to skip

Agar undo/queue/log kabhi chahiye hi nahi, aur action seedha ek function call hai — seedha call karo. Command ki indirection tabhi kamaati hai jab in extra powers (undo/redo, macros, replay) ki zaroorat ho.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — remote directly calls device methods, no undo.
// Chalao: node 1-naive.js

const light = {
  on: () => console.log('💡 Light: ON'),
  off: () => console.log('💡 Light: OFF'),
};
```

```
function pressButton(action) {
  if (action === 'light-on') return light.on();
  if (action === 'light-off') return light.off();
  throw new Error(`Unknown action: ${action}`);
}

pressButton('light-on');
pressButton('light-off');

// PROBLEM: remote ko device ke internals pata hone chahiye. Undo impossible -
// Last action record nahi ho raha. Naya device (fan, AC) add karo to
// pressButton() dobara edit karna padega. Command pattern isse theek karta hai.
```

✅ 2-command.js — the pattern

```
// ✅ COMMAND PATTERN - action ko object bana do (execute + undo).
// Chalao: node 2-command.js

const light = {
  on: () => console.log('💡 Light: ON'),
  off: () => console.log('💡 Light: OFF'),
};

// Har command ek hi contract follow karta hai: execute() aur undo()
class LightOnCommand {
  execute() { light.on(); }
  undo() { light.off(); }
}
class LightOffCommand {
  execute() { light.off(); }
  undo() { light.on(); }
}

// Invoker - kisi command ke internals nahi jaanta, bas execute/undo call karta hai
class RemoteControl {
  #history = [];
  press(command) {
    command.execute();
    this.#history.push(command);
  }
  undoLast() {
    const command = this.#history.pop();
    if (command) command.undo();
  }
}

const remote = new RemoteControl();
remote.press(new LightOnCommand());
remote.press(new LightOffCommand());
remote.undoLast(); // Last action (off) undo -> light wapas ON

// Naya device (fan) add karna = naye Command classes, RemoteControl ko CHHUE BINA.
```

✅ 3-command-queue.js — the pattern

```
// ✅ COMMAND PATTERN – queue, macro aur replay (Log ke through).
// Chalao: node 3-command-queue.js

const light = { on: () => console.log('💡 Light: ON'), off: () => console.log('💡 Light: OFF')
};
const fan    = { on: () => console.log('🌀 Fan: ON'),   off: () => console.log('🌀 Fan: OFF') };

const cmd = (device, action) => ({
  execute: () => device[action](),
  label: `${action} ${device === light ? 'light' : 'fan'}`,
});

// Macro command = kai commands ka ek hi command
class MacroCommand {
  constructor(commands) { this.commands = commands; }
  execute() { this.commands.forEach((c) => c.execute()); }
}

const goodMorning = new MacroCommand([
  cmd(light, 'on'),
  cmd(fan, 'on'),
]);

console.log('-- Good morning macro --');
goodMorning.execute();

// Queue – commands ko turant execute karne ke bajaye baad ke liye rakho
const queue = [];
queue.push(cmd(light, 'off'));
queue.push(cmd(fan, 'off'));

console.log('-- Draining queue (e.g. batch job / scheduled task) --');
while (queue.length) queue.shift().execute();

// Replay – har executed command ka log rakho, baad mein wahi sequence dobara chalao
const log = [cmd(light, 'on'), cmd(fan, 'on'), cmd(light, 'off')];
console.log('-- Replaying log --');
log.forEach((c) => c.execute());
```

🔗 exercise.js — your turn

```
// 🔗 EXERCISE – text editor ke type/delete actions ko Command pattern mein
// refactor karo taaki undo() kaam kare.
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
let text = '';
function type(chars) { text += chars; console.log('text:', JSON.stringify(text)); }
function deleteLast(n) { text = text.slice(0, -n); console.log('text:', JSON.stringify(text)); }
```

```
type('Hello');
type(' ', world');
deleteLast(7); // 'undo' karna ho to yaad nahi ki kitna delete hua tha

// =====
// TODO 1: TypeCommand aur DeleteCommand classes banao, dono mein execute()
//         aur undo() ho. TypeCommand.undo() = wahi characters hata de jo
//         type kiye the. DeleteCommand.undo() = jo delete hua wapas jod de
//         (execute se pehle un characters ko save kar lo).
//
// TODO 2: Editor class banao jisme history array ho, type()/deleteLast()
//         commands create/execute/push karein, aur undo() history se pop
//         karke us command ka undo() chalaye.
//
// TODO 3 (bonus): redo() add karo – undo kiye hue commands ek "redo stack"
//         mein daalo, redo() unhe wapas execute kare.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

16. Chain of Responsibility

Problem: ek request ko kai possible handlers mein se koi ek handle kar sakta hai (approval levels, support ticket routing, middleware), aur "kaun handle karega" ka decision nested if/else mein tightly coupled hai.

Idea: har handler ko ek chain mein jodo. Har handler khud decide kare — "main isse handle kar sakta hoon?" Agar haan, kaam khatam. Agar nahi, request ko **agle handler** ko pass kar do.

Files — is order mein chalao

node 1-naive.js	# ❌ ek function, nested if/else levels
node 2-chain-of-responsibility.js	# ✅ handlers ki chain, har ek apna decision leta hai
node exercise.js	# 🎯 tumhara turn

Principles yahan dikh rahe hain

1. **Decouple sender from receiver** — sender ko pata nahi kaun final handle karega.
2. **Open/closed** — naya handler = chain mein ek naya link, existing handlers ko CHHUE BINA.

When to skip

Agar handlers ka order/set fixed hai aur kabhi badlega nahi, seedha if/else ya switch simpler hai — chain tabhi kamaati hai jab handlers ****dynamically configure/reorder**** hone chahiye.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — approval levels ek hi function mein nested if/else,
// naya level add karna is function ko dobara edit karna hai.
// Chalao: node 1-naive.js

function approveExpense(amount) {
  if (amount <= 1000) {
    console.log(`✅ Manager approved ₹${amount}`);
  } else if (amount <= 10000) {
    console.log(`✅ Director approved ₹${amount}`);
  } else if (amount <= 100000) {
```

```

    console.log(`✅ VP approved ₹${amount}`);
  } else {
    console.log(`❌ ₹${amount} needs CEO approval – not implemented here`);
  }
}

approveExpense(500);
approveExpense(5000);
approveExpense(50000);

// PROBLEM: naya approval level (CEO) ya level ke thresholds badalne ke liye
// is function ko dobara edit karna padega. Poora "kaun approve karta hai"
// logic ek hi jagah tightly coupled hai.

```

✅ 2-chain-of-responsibility.js — the pattern

```

// ✅ CHAIN OF RESPONSIBILITY – har approver apna "handle kar sakta hoon ya
// nahi" khud decide kare, nahi to REQUEST ko agle handler ko pass kar de.
// Chalao: node 2-chain-of-responsibility.js

class Approver {
  constructor(name, limit) { this.name = name; this.limit = limit; this.next = null; }
  setNext(approver) { this.next = approver; return approver; }
  handle(amount) {
    if (amount <= this.limit) {
      console.log(`✅ ${this.name} approved ₹${amount}`);
    } else if (this.next) {
      this.next.handle(amount); // apne bas ka nahi, agle ko pass kar do
    } else {
      console.log(`❌ ₹${amount} – koi bhi approve nahi kar saka`);
    }
  }
}

const manager = new Approver('Manager', 1000);
const director = new Approver('Director', 10000);
const vp = new Approver('VP', 100000);
manager.setNext(director).setNext(vp); // chain banao

manager.handle(500);
manager.handle(5000);
manager.handle(50000);
manager.handle(500000); // koi handle nahi kar sakta

// Naya Level (CEO, unlimited) add karna = ek naya Approver, chain mein jodo,
// baaki approvers ko CHHUE BINA.

```

🔧 exercise.js — your turn

```

// 🔧 EXERCISE – support ticket routing ko Chain of Responsibility se
// refactor karo (bot -> junior agent -> senior agent).
// Chalao: node exercise.js

```

```
// ----- naive version (isse refactor karna hai) -----
function routeTicket(ticket) {
  if (ticket.type === 'faq') console.log('🤖 Bot answered:', ticket.question);
  else if (ticket.priority === 'low') console.log('😓 Junior agent handling:', ticket.question);
  else console.log('👤 Senior agent handling:', ticket.question);
}
routeTicket({ type: 'faq', question: 'What are your hours?' });
routeTicket({ type: 'issue', priority: 'high', question: 'Server is down!' });

// =====
// TODO 1: `Handler` base class banao (jaisa Approver 2-chain-of-
//       responsibility.js mein tha) with setNext(), aur `handle(ticket)`
//       jo `canHandle(ticket)` check kare, warna next ko pass kare.
//
// TODO 2: `BotHandler` (canHandle: type === 'faq'), `JuniorHandler`
//       (canHandle: priority === 'low'), `SeniorHandler` (canHandle:
//       hamesha true – akhri fallback) banao.
//
// TODO 3 (bonus): chain build karo aur dono example tickets route karo,
//       verify karo ki sahi handler pick ho raha hai.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

17. Iterator

Problem: alag-alag collections (array, linked list, tree) ka internal structure alag hota hai. Agar caller ko har collection traverse karne ke liye uska internal shape jaanna pade, to generic "loop over anything" code likhna mushkil ho jaata hai.

Idea: har collection apna traversal khud implement kare, lekin bahar se ek **uniform interface** (`next()` / JS ka `Symbol.iterator`) dikhaye. Caller sirf `for...of` use karta hai — collection array hai ya linked list, usse fark nahi padta.

Files — is order mein chalao

```
node 1-naive.js      # ❌ har collection ka traversal alag, caller ko internals pata hone chahiye
node 2-iterator.js   # ✅ Symbol.iterator — uniform `for...of` sabke liye
node exercise.js     # 🔄 tumhara turn
```

Principles yahan dikh rahe hain

1. **Uniform traversal interface** — caller collection ke internals se decouple ho jaata hai.
2. JS mein `Symbol.iterator` implement karna matlab tumhara custom type `for...of` , `spread ( [...x] )` , `Array.from()` sab ke saath kaam karta hai.

When to skip

JS ke built-in collections (Array, Map, Set) already iterable hain — apna Iterator sirf tab likho jab tumhara **custom data structure** (linked list, tree, graph) ho jise `for...of` se traverse karna ho.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — caller ko har collection ke internal structure ka
// pata hona chahiye, taaki sahi tarike se loop kare.
// Chalao: node 1-naive.js

const arrayCollection = [1, 2, 3];
```

```

const linkedList = { head: { value: 'a', next: { value: 'b', next: { value: 'c', next: null } } }
};

// Array ke Liye ek tarika
for (const item of arrayCollection) console.log('array item:', item);

// Linked list ke liye BILKUL alag tarika – caller ko internal shape pata hona zaroori
let node = linkedList.head;
while (node) { console.log('list item:', node.value); node = node.next; }

// PROBLEM: caller ko har collection type ka internal structure yaad rakhna
// padta hai. Ek generic function jo "kisi bhi collection" pe kaam kare –
// Likhna mushkil hai kyunki traversal logic collection ke saath tightly
// coupled hai.

```

✓ 2-iterator.js — the pattern

```

// ✓ ITERATOR PATTERN – har collection apna traversal khud implement kare,
// lekin BAHAR se ek hi uniform interface (`next()`) dikhaye. Caller ko
// internal structure jaanne ki zaroorat nahi.
// Chalao: node 2-iterator.js

class LinkedList {
  constructor() { this.head = null; }
  push(value) {
    const node = { value, next: null };
    if (!this.head) { this.head = node; return this; }
    let cur = this.head;
    while (cur.next) cur = cur.next;
    cur.next = node;
    return this;
  }
  // Iterator protocol – JS ke `for...of` se compatible!
  [Symbol.iterator]() {
    let current = this.head;
    return {
      next: () => {
        if (!current) return { done: true, value: undefined };
        const value = current.value;
        current = current.next;
        return { done: false, value };
      },
    };
  }
}

const list = new LinkedList().push('a').push('b').push('c');

// Ab caller ko linked-list ke internals (head/next) ka pata hi nahi –
// array ki tarah hi `for...of` use karta hai:
for (const item of list) console.log('list item:', item);

function printAll(collection) {
  for (const item of collection) console.log(' ->', item);
}

```

```

}
printAll([1, 2, 3]); // array
printAll(list);      // linked list – SAME function, dono pe kaam karta hai!

```

exercise.js — your turn

```

// 🐛 EXERCISE – ek binary tree ko Iterator pattern se traverse-able banao,
// taaki `for...of` se depth-first traverse ho sake.
// Chalao: node exercise.js

// ----- given structure (isse badalna nahi hai) -----
const tree = {
  value: 1,
  left: { value: 2, left: { value: 4, left: null, right: null }, right: null },
  right: { value: 3, left: null, right: null },
};

// ----- naive version (isse refactor karna hai) -----
function printTree(node) {
  if (!node) return;
  console.log(node.value);
  printTree(node.left);
  printTree(node.right);
}
printTree(tree);

// =====
// TODO 1: `TreeNode` class banao (value, left, right) jismein
//         `[Symbol.iterator]()` method ho jo depth-first (pre-order:
//         self -> left -> right) traversal kare aur `{done, value}` objects
//         de (2-iterator.js jaisa pattern).
//         Hint: recursion se ek array collect karke uska iterator return
//         karna sabse simple hoga.
//
// TODO 2: upar wale plain-object `tree` ko `TreeNode` instances se dobara
//         banao, phir `for (const v of root) console.log(v)` se print karo.
//
// TODO 3 (bonus): ek "breadth-first" iterator variant socho – kya isse bhi
//                 `Symbol.iterator` ke through expose kiya jaa sakta hai? (Hint:
//                 alag method naam do, jaise `bfs()`, jo khud ek iterable return kare.)
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====

```

18. Mediator

Problem: components (UI fields, chat users, game objects) ek doosre ko DIRECTLY reference karte hain taaki interact kar sakein. Jitne components, wiring utni hi complex (many-to-many) ho jaati hai — naya component add karna sabko touch karwata hai.

Idea: components ek doosre ko na jaanein, sab sirf ek **central mediator** se baat karein. Mediator hi decide karta hai ki kisi event pe kaun kya karega.

Files — is order mein chalao

```
node 1-naive.js      # ❌ components ek doosre ko directly reference karte hain
node 2-mediator.js   # ✅ sab sirf mediator se baat karte hain
node exercise.js     # 🎯 tumhara turn
```

Principles yahan dikh rahe hain

1. **Decouple components from each other** — koi bhi component doosre ka naam tak nahi jaanta.
2. **Centralize interaction logic** — "kaun kisse affect hota hai" ka rule ek hi jagah (mediator) rehta hai.

When to skip

Agar sirf 2 components hain jinka relationship simple aur permanent hai, direct reference likhna hi simpler hai — Mediator tabhi kamaata hai jab **kai components ka wiring complex/changing** ho.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — form fields ek doosre ko DIRECTLY reference karte
// hain — many-to-many wiring, naya field add karna sabko edit karwata hai.
// Chalao: node 1-naive.js

class Checkbox {
  constructor() { this.checked = false; }
  toggle(submitButton) {
```

```

    this.checked = !this.checked;
    console.log('checkbox:', this.checked);
    submitButton.setEnabled(this.checked); // checkbox ko SubmitButton ka pata hai
  }
}

class SubmitButton {
  setEnabled(enabled) { console.log('submit button enabled:', enabled); }
}

const submitButton = new SubmitButton();
const checkbox = new Checkbox();
checkbox.toggle(submitButton);
checkbox.toggle(submitButton);

// PROBLEM: Checkbox ko SubmitButton ke baare mein directly pata hai. Ek
// teesra field (dropdown) add karo jo bhi submit ko affect kare – ab
// Checkbox aur Dropdown dono ko ek doosre + SubmitButton sabka pata hona
// padega. Components ka wiring  $O(n^2)$  ho jaata hai.

```

✓ 2-mediator.js — the pattern

```

// ✓ MEDIATOR PATTERN – components ek doosre ko directly nahi jaante, sab
// ek CENTRAL mediator se baat karte hain. Mediator decide karta hai "kisko
// kya karna hai".
// Chalao: node 2-mediator.js

class FormMediator {
  #components = {};
  register(name, component) { this.#components[name] = component; component.mediator = this; }
  notify(sender, event) {
    if (sender === 'checkbox' && event === 'toggled') {
      const checked = this.#components.checkbox.checked;
      this.#components.submitButton.setEnabled(checked);
    }
    // naya field/rule add karna = yahan ek naya `if`, components ko chhue bina
  }
}

class Checkbox {
  constructor() { this.checked = false; }
  toggle() {
    this.checked = !this.checked;
    console.log('checkbox:', this.checked);
    this.mediator.notify('checkbox', 'toggled'); // sirf mediator ko batata hai, SubmitButton ko
    // nahi jaanta
  }
}

class SubmitButton {
  setEnabled(enabled) { console.log('submit button enabled:', enabled); }
}

const mediator = new FormMediator();
const checkbox = new Checkbox();
const submitButton = new SubmitButton();

```

```
mediator.register('checkbox', checkbox);
mediator.register('submitButton', submitButton);

checkbox.toggle();
checkbox.toggle();

// Naya field (dropdown) add karna = mediator mein ek naya `if`/register call,
// Checkbox aur SubmitButton ko ek doosre ke baare mein KUCH pata nahi hona chahiye.
```

exercise.js — your turn

```
// 🐛 EXERCISE – chat users ko Mediator (ChatRoom) se connect karo taaki wo
// ek doosre ko directly reference na karein.
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
class User {
  constructor(name) { this.name = name; }
  sendDirect(otherUser, msg) { // har user ko doosre users ka direct reference chahiye
    console.log(`[${this.name} -> ${otherUser.name}]: ${msg}`);
  }
}

const alice = new User('Alice');
const bob = new User('Bob');
alice.sendDirect(bob, 'Hi Bob!');

// =====
// TODO 1: `ChatRoom` class banao with `register(user)` (users array mein
//         add + user.room = this set kare) aur `broadcast(sender, msg)`
//         (sender ke alawa sabko print kare: `[sender -> everyone]: msg`).
//
// TODO 2: `User` class ko refactor karo – `send(msg)` method ho jo
//         `this.room.broadcast(this, msg)` call kare, koi doosre User ka
//         direct reference na ho.
//
// TODO 3 (bonus): teesra user 'Carol' add karo, dikhao ki Alice ka message
//                 Bob aur Carol dono ko milta hai – Alice ko unka direct reference
//                 kabhi nahi pata.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

19. Memento

Problem: ek object ka state kabhi-kabhi save aur baad mein restore karna hai (undo, save-game, checkpoints) — lekin caller ko object ke internal fields khud copy/restore karne padte hain, jo encapsulation todta hai aur internals badalne par toot jaata hai.

Idea: object khud apna snapshot banaye (`save()`) aur khud restore kare (`restore()`). Caller sirf snapshot ko *opaque* tareeke se hold karta hai — uske andar kya hai, usse matlab nahi.

Files — is order mein chalao

```
node 1-naive.js      # ❌ caller directly internal fields copy/restore karta hai
node 2-memento.js    # ✅ object khud save()/restore() implement karta hai
node exercise.js     # 🎯 tumhara turn
```

Principles yahan dikh rahe hain

1. **Encapsulation preserved** — internals sirf object khud dekhta/badalta hai, caller nahi.
2. **Snapshot is opaque to the caller** — History/SaveSlot bas snapshot ko store karte hain, uske andar jhaankte nahi.

Related: Command se fark

Command "actions" ko undo karta hai (execute/undo pairs). Memento "state" ko undo karta hai (poora snapshot save/restore). Command tab better hai jab undo logic simple ho; Memento tab better hai jab state complex ho aur "reverse action" likhna mushkil ho.

When to skip

Agar state bohot chhota/simple hai (1-2 primitive fields), seedha ek variable mein copy rakhna kaafi hai — Memento tabhi kamaata hai jab state complex ho aur encapsulation todna avoid karna ho.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION – state ko bahar se directly copy/restore kiya ja
// raha hai – editor ke internal fields sabko expose karne padte hain.
// Chalao: node 1-naive.js

class Editor {
  constructor() { this.text = ''; }
  type(chars) { this.text += chars; }
}

const editor = new Editor();
editor.type('Hello');

// Undo ke liye caller khud state copy kar raha hai – editor ke internals
// (this.text) directly access ho rahe hain
const backup = editor.text; // agar Editor mein aur fields add ho (cursor, formatting)
// to yeh backup INCOMPLETE ho jaayega

editor.type(', world');
console.log('before undo:', editor.text);
editor.text = backup; // caller directly internal field ko overwrite kar raha hai
console.log('after undo:', editor.text);

// PROBLEM: caller ko Editor ke internal fields ka pata hona chahiye aur
// undo logic khud implement karna padta hai. Editor ke internals badle
// (naya field add ho) to yeh backup/restore code bhi TOOT sakta hai.
```

✅ 2-memento.js — the pattern

```
// ✅ MEMENTO PATTERN – object khud apna "snapshot" banaye aur khud restore
// kare. Caller ko internal fields ka pata nahi hona chahiye, sirf ek opaque
// snapshot object hold karta hai.
// Chalao: node 2-memento.js

class Editor {
  #text = '';
  type(chars) { this.#text += chars; }
  get text() { return this.#text; }

  save() {
    return { text: this.#text }; // memento – apna hi state, khud banaya
  }
  restore(memento) {
    this.#text = memento.text; // sirf Editor khud jaanta hai kaise restore kare
  }
}

class History {
  #snapshots = [];
  push(memento) { this.#snapshots.push(memento); }
  pop() { return this.#snapshots.pop(); }
}
```

```

const editor = new Editor();
const history = new History();

editor.type('Hello');
history.push(editor.save()); // undo point save

editor.type(', world');
console.log('before undo:', editor.text);

editor.restore(history.pop()); // caller ko Editor ke internals ka pata hi nahi
console.log('after undo:', editor.text);

// Editor mein naya field (cursor position) add karo – sirf save()/restore()
// badlenge, caller ka History code CHHUE BINA rahega.

```

exercise.js — your turn

```

// 🐛 EXERCISE – game character ke state ko Memento se save/restore karo.
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
class Character {
  constructor() { this.hp = 100; this.level = 1; }
  takeDamage(dmg) { this.hp -= dmg; }
  levelUp() { this.level++; this.hp = 100; }
}

const hero = new Character();
const backup = { hp: hero.hp, level: hero.level }; // manual, error-prone
hero.takeDamage(80);
hero.levelUp();
console.log('current:', hero.hp, hero.level);
// "backup" restore karna hai to caller ko khud `hero.hp = backup.hp` likhna padega

// =====
// TODO 1: `Character` class mein `save()` method add karo jo current state
//         ka snapshot object return kare, aur `restore(memento)` method jo
//         usse wapas apply kare (2-memento.js jaisa).
//
// TODO 2: `SaveSlot` class (ya simple array) banao jo mementos store kare
//         – `save(memento)` aur `load()` methods.
//
// TODO 3 (bonus): Level up hone se pehle auto-save karo, dikhao ki
//                 `takeDamage` + `levelUp` ke baad bhi purane save pe restore ho
//                 sakta hai.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====

```

20. State

Problem: ek object ka behavior uske **current status** pe depend karta hai (order: placed/paid/shipped; traffic light: red/green/yellow), aur har method mein "abhi is status mein kya allowed hai" wala if/else duplicate hota hai. Invalid transitions rokna bhi mushkil hai.

Idea: har status ko ek object bana do jo khud jaanta ho — "is state mein kaunsa action allowed hai, aur uske baad next state kya hoga". Main object (Order/Light) sirf current state ko delegate karta hai.

Files — is order mein chalao

```
node 1-naive.js    # ❌ har method mein status if/else duplicate
node 2-state.js    # ✅ har status apna hi object, khud transitions decide karta hai
node exercise.js   # 🎯 tumhara turn
```

Principles yahan dikh rahe hain

1. **Behavior depends on state, delegate it** — Strategy jaisa hi mechanism (delegation), lekin state **khud badalta hai** (self-transition), Strategy mein caller badalta hai.
2. **Invalid transitions naturally blocked** — jo state jis action ko support nahi karta, uska handler khud "not allowed" bol deta hai.

When to skip

Agar sirf 2 states hain ya transitions kabhi badhenge nahi, simple boolean flag + if/else kaafi hai — State pattern tabhi kamaata hai jab ***kai states aur transitions*** ho jo complex ho rahe hon.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — order status ek string hai, har jagah if/else se
// check hota hai, invalid transitions (delivered -> paid) rokta koi nahi.
// Chalao: node 1-naive.js

class Order {
  constructor() { this.status = 'placed'; }
  pay() {
```

```

    if (this.status === 'placed') { this.status = 'paid'; console.log('✅ paid'); }
    else console.log(`❌ cannot pay from status: ${this.status}`);
  }
  ship() {
    if (this.status === 'paid') { this.status = 'shipped'; console.log('✅ shipped'); }
    else console.log(`❌ cannot ship from status: ${this.status}`);
  }
  deliver() {
    if (this.status === 'shipped') { this.status = 'delivered'; console.log('✅ delivered'); }
    else console.log(`❌ cannot deliver from status: ${this.status}`);
  }
}

const order = new Order();
order.ship(); // invalid - abhi paid nahi hua
order.pay();
order.ship();
order.deliver();

// PROBLEM: har method mein "current status ke hisaab se kya allowed hai"
// wala if/else duplicate hai. Naya status (e.g. 'cancelled') add karo to
// HAR method dobara edit karna padega.

```

✅ 2-state.js — the pattern

```

// ✅ STATE PATTERN - har status ko ek object bana do jo khud decide kare
// "is status mein kya allowed hai". Order sirf current state ko delegate
// karta hai.
// Chalao: node 2-state.js

const placedState = {
  name: 'placed',
  pay: (order) => { console.log('✅ paid'); order.setState(paidState); },
  ship: () => console.log('❌ cannot ship - not paid yet'),
  deliver: () => console.log('❌ cannot deliver - not shipped yet'),
};

const paidState = {
  name: 'paid',
  pay: () => console.log('❌ already paid'),
  ship: (order) => { console.log('✅ shipped'); order.setState(shippedState); },
  deliver: () => console.log('❌ cannot deliver - not shipped yet'),
};

const shippedState = {
  name: 'shipped',
  pay: () => console.log('❌ already paid'),
  ship: () => console.log('❌ already shipped'),
  deliver: (order) => { console.log('✅ delivered'); order.setState(deliveredState); },
};

const deliveredState = {
  name: 'delivered',
  pay: () => console.log('❌ order already delivered'),
  ship: () => console.log('❌ order already delivered'),
  deliver: () => console.log('❌ already delivered'),
};

```

```

class Order {
  constructor() { this.state = placedState; }
  setState(state) { this.state = state; }
  pay() { this.state.pay(this); }
  ship() { this.state.ship(this); }
  deliver() { this.state.deliver(this); }
}

const order = new Order();
order.ship(); // invalid - placedState.ship() khud "not paid" bolta hai
order.pay();
order.ship();
order.deliver();

// Naya status (cancelled) add karna = ek naya state object, Order class
// ko CHHUE BINA.

```

exercise.js — your turn

```

// 🐛 EXERCISE - traffic light (red -> green -> yellow -> red) ko State
// pattern se refactor karo.
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
class TrafficLight {
  constructor() { this.color = 'red'; }
  next() {
    if (this.color === 'red') this.color = 'green';
    else if (this.color === 'green') this.color = 'yellow';
    else if (this.color === 'yellow') this.color = 'red';
    console.log('now:', this.color);
  }
}

const light = new TrafficLight();
light.next(); light.next(); light.next();

// =====
// TODO 1: `redState`, `greenState`, `yellowState` objects banao, har ek
// mein `name` aur `next(light)` method (jo light.setState(...))
// call kare next color ke state pe - 2-state.js jaisa).
//
//
// TODO 2: `TrafficLight` class ko refactor karo - constructor mein
// `this.state = redState`, `next()` method `this.state.next(this)`
// call kare, `setState(state)` method ho.
//
//
// TODO 3 (bonus): socho - is design mein naya rule "yellow se seedha red,
// par sirf agar pedestrian button dabaya ho" add karna kitna easy
// hai naive version ke mukable?
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====

```

21. Template Method

Problem: kai variants mein ek SAME skeleton/sequence hota hai (load -> process -> export; welcome -> setup -> notify), lekin beech ka 1-2 step alag hota hai. Naive tareeke mein poora skeleton har variant mein duplicate hota hai.

Idea: skeleton ko ek base class mein **ek baar** fix karo (template method). Jo steps variant-specific hain, unhe abstract/overridable methods banao — subclasses sirf unhi ko implement karein, skeleton ko nahi chhuein.

Files — is order mein chalao

node 1-naive.js	# ❌ poora skeleton har report mein duplicate
node 2-template-method.js	# ✅ skeleton ek jagah, sirf steps override hote hain
node exercise.js	# 🔄 tumhara turn

Principles yahan dikh rahe hain

1. **Don't repeat the skeleton** — sequence/order ek hi jagah likha jaata hai.
2. **Hollywood Principle** — "don't call us, we'll call you": base class subclass ke methods ko sahi order mein khud call karta hai.

When to skip

Agar variants ke beech skeleton hi alag hai (sirf 1-2 steps common hain, poora flow nahi), Template Method zabardasti fit karoge to rigid ho jaayega — tab Strategy (poora behavior swap) ya plain functions behtar hain.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — poora "load -> process -> export" skeleton har
// report type mein DUPLICATE hota hai, sirf beech ke steps alag hain.
// Chalao: node 1-naive.js

function generateCsvReport(data) {
  console.log('📁 loading data...');
  const rows = data.map((d) => `${d.name},${d.value}`);
  console.log('⚙️ processing...');
```

```

const processed = rows.join('\n');
console.log('📄 exporting...');
console.log('CSV OUTPUT:\n' + processed);
}

function generateJsonReport(data) {
  console.log('📁 loading data...'); // duplicate
  const processed = JSON.stringify(data);
  console.log('⚙️ processing...'); // duplicate
  console.log('📄 exporting...'); // duplicate
  console.log('JSON OUTPUT:\n' + processed);
}

generateCsvReport([{ name: 'A', value: 1 }]);
generateJsonReport([{ name: 'A', value: 1 }]);

// PROBLEM: "load -> process -> export" ka skeleton har report function mein
// copy-paste hua hai. Skeleton mein koi step add/change karna ho (e.g. ek
// validation step) to HAR report function edit karna padega.

```

✅ 2-template-method.js — the pattern

```

// ✅ TEMPLATE METHOD — algorithm ka SKELETON base class mein ek baar likho,
// sirf jo steps subtype-specific hain unhe subclass override kare.
// Chalao: node 2-template-method.js

class ReportGenerator {
  // Template method — skeleton FIXED hai, subclass isse override nahi karti
  generate(data) {
    console.log('📁 loading data...');
    const processed = this.process(data); // yahi step subclass-specific hai
    console.log('⚙️ processing done');
    console.log('📄 exporting...');
    console.log(`${this.format()} OUTPUT:\n${processed}`);
  }
  process(data) { throw new Error('subclass must implement process()'); }
  format() { throw new Error('subclass must implement format()'); }
}

class CsvReport extends ReportGenerator {
  format() { return 'CSV'; }
  process(data) { return data.map((d) => `${d.name},${d.value}`).join('\n'); }
}

class JsonReport extends ReportGenerator {
  format() { return 'JSON'; }
  process(data) { return JSON.stringify(data); }
}

new CsvReport().generate([{ name: 'A', value: 1 }]);
new JsonReport().generate([{ name: 'A', value: 1 }]);

// Skeleton mein naya step (validation) add karna = ek jagah, generate() mein.

```

```
// Naya format (XML) add karna = bas process()+format() override karne wali
// ek nayi subclass, skeleton ko CHHUE BINA.
```

exercise.js — your turn

```
// 🐛 EXERCISE – do onboarding flows (free user vs paid user) ke duplicate
// skeleton ko Template Method se nikalo.
// Chalao: node exercise.js

// ----- naive version (isse refactor karna hai) -----
function onboardFreeUser(name) {
  console.log(`📧 welcome email to ${name}`);
  console.log('📁 free plan setup');
  console.log(`📢 notifying team: ${name} joined (free)`);
}

function onboardPaidUser(name) {
  console.log(`📧 welcome email to ${name}`); // duplicate
  console.log('📁 paid plan setup + billing');
  console.log(`📢 notifying team: ${name} joined (paid)`); // near-duplicate
}

onboardFreeUser('Asha');
onboardPaidUser('Rohit');

// =====
// TODO 1: `Onboarding` base class banao with template method `run(name)`
//         jo: welcome email (fixed) -> `this.setup(name)` (subclass-specific)
//         -> `this.notifyTeam(name)` (fixed, lekin plan naam ke liye
//         `this.planName()` ko call kare).
//
//
// TODO 2: `FreeOnboarding` aur `PaidOnboarding` subclasses banao jo
//         `setup(name)` aur `planName()` override karein.
//
//
// TODO 3 (bonus): naya `TrialOnboarding` add karo – dikhaao ki base class
//         `run()` ko CHHUE BINA kaam karta hai.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

22. Visitor

Problem: classes ka ek fixed set hai (shapes, AST nodes), aur unpe naye **operations** add karte rehna hai (area, export, evaluate, print). Har naye operation ke liye har class ko edit karna padta hai — aur agar classes third-party se aayi ho to edit karna possible hi nahi.

Idea: naye operation ko class ke andar method ke bajaye ek alag "visitor" object mein rakho. Har class mein bas ek chhota, kabhi-na-badalne wala `accept(visitor)` method rehta hai jo sahi visitor-method ko call karta hai.

Files — is order mein chalao

```
node 1-naive.js      # ❌ naya operation = har shape class edit
node 2-visitor.js    # ✅ naya operation = naya visitor object, classes untouched
node exercise.js     # 🔄 tumhara turn
```

Principles yahan dikh rahe hain

1. **Add operations without modifying classes** — Open/Closed principle, operations ke perspective se.
2. **Double dispatch** — `shape.accept(visitor)` shape ka type decide karta hai, andar `visitor.visitX(this)` operation decide karta hai.

When to skip

Agar classes ka set frequently badalta hai (naya shape type baar-baar add hota hai) lekin operations stable hain, Visitor ulta pain hai — har naya shape ke liye HAR visitor ko edit karna padega. Visitor tab kamaata hai jab **classes stable hain, operations badalte rehte hain** (ulta scenario).

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — naya operation (e.g. "export to SVG") add karne ke
// liye HAR shape class ko edit karna padta hai.
// Chalao: node 1-naive.js

class Circle {
```

```

    constructor(r) { this.r = r; }
    area() { return Math.PI * this.r * this.r; }
    // naya operation chahiye? yahan naya method jodo... har shape mein.
  }
  class Square {
    constructor(s) { this.s = s; }
    area() { return this.s * this.s; }
  }

  const shapes = [new Circle(2), new Square(3)];
  shapes.forEach((s) => console.log(s.constructor.name, 'area:', s.area().toFixed(2)));

  // PROBLEM: agar kal "toSvg()" operation chahiye har shape ke liye, to Circle
  // AUR Square dono classes ko edit karna padega. Jitne shapes utni jagah edit.
  // Aur agar shapes third-party library se aayi hon (edit hi nahi kar sakte)?

```

✓ 2-visitor.js — the pattern

```

// ✓ VISITOR PATTERN – naye operations ko shape classes ke BAHAR, alag
// "visitor" objects mein rakho. Shapes sirf ek chhota `accept(visitor)`
// method rakhte hain – naya operation add karne ke liye unhe edit nahi
// karna padta.
// Chalao: node 2-visitor.js

class Circle {
  constructor(r) { this.r = r; }
  accept(visitor) { return visitor.visitCircle(this); } // fixed, kabhi nahi badalta
}
class Square {
  constructor(s) { this.s = s; }
  accept(visitor) { return visitor.visitSquare(this); }
}

// Operation #1: area calculation – ek visitor object
const areaVisitor = {
  visitCircle: (c) => Math.PI * c.r * c.r,
  visitSquare: (s) => s.s * s.s,
};

// Operation #2 (naya!): SVG export – shapes ko CHHUE BINA add hua
const svgVisitor = {
  visitCircle: (c) => `<circle r="${c.r}" />`,
  visitSquare: (s) => `<rect width="${s.s}" height="${s.s}" />`,
};

const shapes = [new Circle(2), new Square(3)];
shapes.forEach((s) => console.log('area:', s.accept(areaVisitor).toFixed(2)));
shapes.forEach((s) => console.log('svg:', s.accept(svgVisitor)));

// Naya operation (e.g. perimeter) add karna = ek naya visitor object,
// Circle/Square classes ko CHHUE BINA.

```

exercise.js — your turn

```
// 🐛 EXERCISE – simple expression tree (Number, Add nodes) ke Liye
// evaluate() aur print() operations ko Visitor se add karo, node classes ko
// chhue bina.
// Chalao: node exercise.js

// ----- given node classes (accept() already fixed hai, inhe mat badlo) -----
class NumberNode {
  constructor(value) { this.value = value; }
  accept(visitor) { return visitor.visitNumber(this); }
}
class AddNode {
  constructor(left, right) { this.left = left; this.right = right; }
  accept(visitor) { return visitor.visitAdd(this); }
}

// (2 + 3)
const tree = new AddNode(new NumberNode(2), new NumberNode(3));

// =====
// TODO 1: `evaluateVisitor` object banao with `visitNumber(node)` (return
//         node.value) aur `visitAdd(node)` (return node.left.accept(this) +
//         node.right.accept(this) – recursive!).
//
// TODO 2: `tree.accept(evaluateVisitor)` call karke result print karo
//         (chahiye: 5).
//
// TODO 3 (bonus): `printVisitor` banao jo expression ko string mein convert
//         kare (`"(2 + 3)"`), NumberNode/AddNode classes ko CHHUE BINA.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

23. Interpreter

Problem: ek chhoti "language"/grammar hai (math expressions, boolean rules, filter queries) jise evaluate karna hai. String-parsing + if/else se karo to nested/composed expressions handle karna mushkil ho jaata hai.

Idea: grammar ke har rule (number, operator) ko ek chhoti class banao jismein `interpret()` method ho. Complex expressions in chhote nodes ko compose karke (tree ki tarah) banti hain — Composite pattern ka special use-case.

Files — is order mein chalao

```
node 1-naive.js      # ❌ string-parsing + if/else, sirf flat expressions
node 2-interpreter.js # ✅ chhoti composable classes, nested expressions natural
node exercise.js     # 🎯 tumhara turn
```

Principles yahan dikh rahe hain

1. **Grammar as objects** — har rule apna node-type, evaluation logic uske andar.
2. **Composable via recursion** — Composite jaisa hi idea, yahan operation ka naam `interpret()` hai.

When to skip

Agar grammar simple/fixed hai ya rarely change hoti hai, ek proper parser library (ya even `eval` -jaisa safe alternative) use karna behtar hai. Interpreter tab kamaata hai jab grammar **chhoti ho aur custom rules frequently badalte/badhte** hon.

❌ 1-naive.js — the naive approach

```
// ❌ THE NAIVE VERSION — simple math expressions ("3 + 4") ko ek hi function
// mein string-parsing + hardcoded if/else se evaluate kiya ja raha hai.
// Chalao: node 1-naive.js

function evaluate(expr) {
  const parts = expr.split(' ');
  if (parts.length === 3 && parts[1] === '+') return Number(parts[0]) + Number(parts[2]);
  if (parts.length === 3 && parts[1] === '-') return Number(parts[0]) - Number(parts[2]);
}
```

```

    throw new Error(`cannot evaluate: ${expr}`);
  }

  console.log(evaluate('3 + 4'));
  console.log(evaluate('10 - 6'));

  // PROBLEM: sirf DO numbers wale simple expressions handle hote hain.
  // Nested expressions ("3 + 4 - 2") ya naya operator add karna is function
  // ko bade if/else mein aur uljha dega – grammar aur evaluation mixed hain.

```

✓ 2-interpreter.js — the pattern

```

// ✓ INTERPRETER PATTERN – grammar ke har "rule" ko ek chhoti class banao
// jismein `interpret()` method ho. Complex expressions in chhote nodes ko
// COMPOSE karke banti hain (jaisa Composite pattern).
// Chalao: node 2-interpreter.js

class NumberExpr {
  constructor(value) { this.value = value; }
  interpret() { return this.value; }
}

class AddExpr {
  constructor(left, right) { this.left = left; this.right = right; }
  interpret() { return this.left.interpret() + this.right.interpret(); }
}

class SubtractExpr {
  constructor(left, right) { this.left = left; this.right = right; }
  interpret() { return this.left.interpret() - this.right.interpret(); }
}

// "3 + 4 - 2" ko tree se build karo – har node grammar ka ek rule hai
const expr = new SubtractExpr(
  new AddExpr(new NumberExpr(3), new NumberExpr(4)),
  new NumberExpr(2),
);
console.log('3 + 4 - 2 =', expr.interpret());

// Naya operator (multiply) add karna = ek nayi chhoti class, existing
// classes ko CHHUE BINA. Nested expressions naturally kaam karte hain
// kyunki har node bas .interpret() call karta hai apne children pe.

```

🔧 exercise.js — your turn

```

// 🔧 EXERCISE – feature-flag rules ("isAdmin AND isBeta") ko Interpreter
// pattern se evaluate-able banao.
// Chalao: node exercise.js

const context = { isAdmin: true, isBeta: false, isPremium: true };

// ----- naive version (isse refactor karna hai) -----
function evalRule(rule, ctx) {
  // sirf yeh ek hardcoded case handle karta hai:

```

```
    if (rule === 'isAdmin AND isBeta') return ctx.isAdmin && ctx.isBeta;
    throw new Error(`cannot evaluate: ${rule}`);
}
console.log(evalRule('isAdmin AND isBeta', context));

// =====
// TODO 1: `VarExpr` class banao (constructor: name) with `interpret(ctx)`
//          jo `ctx[this.name]` return kare.
//
//
// TODO 2: `AndExpr` aur `OrExpr` classes banao (constructor: left, right)
//          with `interpret(ctx)` jo `left.interpret(ctx) && right.interpret(ctx)`
//          (ya `||`) return karein.
//
//
// TODO 3 (bonus): `NotExpr` class banao (constructor: expr) with
//                  `interpret(ctx)` jo `!expr.interpret(ctx)` return kare. Phir
//                  "isAdmin AND NOT isBeta" jaisa tree bana kar evaluate karo.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

PART II — SOLID PRINCIPLES

SOLID Principles

23 patterns sikhne ke baad ab "WHY" samajhna hai — SOLID woh 5 principles hain jo batate hain

ki achha object-oriented design **kyun** achha hota hai. Har pattern in principles mein se kisi na kisi ko satisfy karne ke liye exist karta hai.

Order matters — is sequence mein padho

#	Principle	One-line rule	Directly powers
1	Single Responsibility	Ek class, ek reason to change	Facade, Command
2	Open/Closed	Extension ke liye open, modification ke liye closed	Strategy, Factory Method
3	Liskov Substitution	Subtype, base type ki jagah safely use ho sake	Template Method, Composite
4	Interface Segregation	Chhote, focused interfaces — fat interface mat banao	Adapter, Bridge
5	Dependency Inversion	High-level aur low-level dono abstraction pe depend karein	Adapter, Abstract Factory

Files — har principle mein same flow

```
node 1-violation.js    # ❌ principle todta hua code
node 2-fixed.js        # ✅ principle follow karta hua code
node exercise.js       # 🔄 tumhara turn
```

Yeh patterns se pehle kyun nahi padhaya?

Honestly — dono order sahi hain. Patterns pehle padhna tumhe "shape" dikhata hai (yeh kaisa dikhta hai), SOLID baad mein "WHY" deta hai (yeh isliye achha hai). Interview mein dono

directions se sawaal aate hain: "yeh pattern kaunsa principle satisfy karta hai?" utna hi common hai jitna "yeh code SOLID kyun nahi hai, fix karo?"

When to skip

SOLID rules nahi, **guidelines** hain. Chhote scripts, prototypes, ya code jo kabhi grow nahi karega — in principles ki indirection cost mat uthao. Yeh tab kamaate hain jab codebase genuinely evolve/scale karni ho.

24. Single Responsibility Principle (SRP)

Rule: ek class ka sirf **EK "reason to change"** hona chahiye.

Problem: jab ek class business logic + persistence + presentation + notification sab karti hai, to koi bhi EK concern badalne se poori class touch hoti hai — aur baaki concerns accidentally break ho sakte hain.

Idea: har concern ko apni khud ki class mein nikaalo. "Reason to change" = kaun sa external factor (DB engine, email provider, report format, pricing rule) is class ko badalne ke liye force karega.

Files — is order mein chalao

```
node 1-violation.js    # ❌ ek class, 4 reasons to change
node 2-fixed.js        # ✅ har concern apni class mein
node exercise.js       # 🎯 tumhara turn
```

Yeh design patterns se kaise judta hai

SRP hi wo foundation hai jisme [Facade](#) aur [Command](#) jaise patterns khade hote hain — "encapsulate what varies" ka pehla kadam yeh decide karna hai ki **KYA** varies, aur SRP tumhe woh dikhata hai.

When to skip

Har chhoti class ko 5 sub-classes mein todna zaroori nahi — agar do concerns HAMESHA saath

badalte hain (genuinely coupled), unhe alag karna sirf indirection badhata hai. SRP tab lagao jab concerns ke liye alag-alag, **independent** reasons-to-change ho.

❌ 1-violation.js — the violation

```
// ❌ VIOLATION — ek hi class business logic, persistence, AUR presentation sab kar rahi hai.
// Chalao: node 1-violation.js

class Invoice {
  constructor(items) { this.items = items; }
```

```

calculateTotal() {
  return this.items.reduce((sum, item) => sum + item.price * item.qty, 0);
}

saveToDatabase() {
  console.log('💾 saving invoice to database...'); // pretend yeh DB write hai
}

printReceipt() {
  console.log('--- RECEIPT ---');
  this.items.forEach((i) => console.log(`${i.name} x${i.qty} = ₹${i.price * i.qty}`));
  console.log(`TOTAL: ₹${this.calculateTotal()}`);
}

emailToCustomer(email) {
  console.log(`✉️ emailing receipt to ${email}...`);
}
}

const invoice = new Invoice([ { name: 'Widget', price: 100, qty: 2 } ]);
invoice.printReceipt();
invoice.saveToDatabase();
invoice.emailToCustomer('customer@example.com');

// PROBLEM: is class ko badalne ke 4 alag reasons hain -
// 1. Pricing rule badle (calculateTotal)
// 2. DB engine badle (saveToDatabase)
// 3. Receipt format badle (printReceipt)
// 4. Email provider badle (emailToCustomer)
// Har reason ek ALAG concern hai, Lekin sab ek hi class mein bandhe hain. Koi bhi change
// poori class ko touch karta hai aur baaki teeno cheezon ko accidentally break kar sakta hai.

```

✅ 2-fixed.js — the fix

```

// ✅ FIXED — har concern apni khud ki class mein, ek hi "reason to change" ke saath.
// Chalao: node 2-fixed.js

class Invoice {
  constructor(items) { this.items = items; }
  calculateTotal() {
    return this.items.reduce((sum, item) => sum + item.price * item.qty, 0);
  }
}

class InvoiceRepository {
  save(invoice) { console.log('💾 saving invoice to database...'); }
}

class InvoicePrinter {
  print(invoice) {
    console.log('--- RECEIPT ---');
    invoice.items.forEach((i) => console.log(`${i.name} x${i.qty} = ₹${i.price * i.qty}`));
    console.log(`TOTAL: ₹${invoice.calculateTotal()}`);
  }
}

```

```

}

class InvoiceMailer {
  send(invoice, email) { console.log(`📧 emailing receipt to ${email}...`); }
}

const invoice = new Invoice([ { name: 'Widget', price: 100, qty: 2 } ]);
new InvoicePrinter().print(invoice);
new InvoiceRepository().save(invoice);
new InvoiceMailer().send(invoice, 'customer@example.com');

// Ab DB engine badalna sirf InvoiceRepository ko touch karta hai. Receipt format badalna
// sirf InvoicePrinter ko. Invoice class ko badalne ka SIRF EK reason hai: pricing/data
// shape khud badle. Yehi Single Responsibility hai – "ek class, ek reason to change."

```

exercise.js — your turn

```

// 🛠️ EXERCISE – is UserManager class mein kitne "reasons to change" hain? Unhe alag classes
// mein split karo.
// Chalao: node exercise.js

// ----- violation (isse refactor karna hai) -----
class UserManager {
  constructor() { this.users = []; }
  registerUser(name, email) {
    if (!email.includes('@')) throw new Error('invalid email'); // validation
    this.users.push({ name, email }); // storage
    console.log(`💾 saved user ${name}`);
  }
  sendWelcomeEmail(email) {
    console.log(`📧 welcome email sent to ${email}`); // notification
  }
  generateUserReport() {
    console.log('--- USER REPORT ---'); // reporting
    this.users.forEach((u) => console.log(`${u.name} <${u.email}>`));
  }
}

const manager = new UserManager();
manager.registerUser('Asha', 'asha@example.com');
manager.sendWelcomeEmail('asha@example.com');
manager.generateUserReport();

// =====
// TODO 1: count the distinct "reasons to change" hiding in UserManager (hint: there are at
//         least 4 – validation, storage, notification, reporting).
//
//
// TODO 2: split it into focused classes (e.g. UserValidator, UserRepository, WelcomeEmailer,
//         UserReportGenerator) – each with ONE job.
//
//
// TODO 3 (bonus): which of your new classes would change if you swapped the database? Which
//                 would change if marketing wanted a different welcome email template? If the answer
//                 to both is "more than one class," you haven't fully separated the concerns yet.
//

```

```
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.  
// =====
```

25. Open/Closed Principle (OCP)

Rule: classes/functions **extension ke liye open** hone chahiye, ****modification ke liye closed****.

Problem: jab naya "type" (discount tier, shipping method, notification channel) add karne ke liye tumhe EXISTING, already-tested function ko edit karna padta hai, to har change purani functionality ko risk mein daalta hai.

Idea: variation points ko ek registry/lookup mein nikaalo. Naya type add karna ab ek NAYI entry hai (extension), purane code ko touch karna nahi (modification).

Files — is order mein chalao

```
node 1-violation.js    # ❌ naya type = existing function edit karo
node 2-fixed.js        # ✅ naya type = registry mein ek entry
node exercise.js       # 🔄 tumhara turn
```

Yeh design patterns se kaise judta hai

Yeh literally [Strategy](#) aur [Factory Method](#)

ka "why" hai — dono patterns EXACTLY OCP ko satisfy karne ke liye exist karte hain. Agar tumne

Lesson 1 (Strategy) already kiya hai, `2-fixed.js` yahan familiar lagega.

When to skip

Har if/else ko turant registry mein convert mat karo — agar sirf 2 cases hain jo KABHI badhenge nahi, plain `if` behtar hai. OCP tab lagao jab naye types/variants **regularly add** hone ki expectation ho.

❌ 1-violation.js — the violation

```
// ❌ VIOLATION - naya discount type add karne ke liye is function ko HAR baar edit karna
// padta hai (modification), extend nahi kar sakte bina chhue.
// Chalao: node 1-violation.js
```

```
function calculateDiscount(type, amount) {
  if (type === 'regular') return amount * 0.05;
```

```

    if (type === 'premium') return amount * 0.10;
    if (type === 'vip') return amount * 0.20;
    throw new Error(`Unknown type: ${type}`);
  }

  console.log(calculateDiscount('regular', 1000));
  console.log(calculateDiscount('vip', 1000));

  // PROBLEM: naya customer tier (e.g. 'platinum') add karne ke liye is EXISTING function ko
  // dobara edit karna padega – jo already-tested code ko risk mein daalta hai. Yeh "closed
  // for modification" ko todta hai.

```

✓ 2-fixed.js — the fix

```

// ✓ FIXED – naya discount type add karna ab EXTENSION hai (naya entry), MODIFICATION nahi
// (existing code chhua nahi).
// Chalao: node 2-fixed.js

const discountStrategies = {
  regular: (amount) => amount * 0.05,
  premium: (amount) => amount * 0.10,
  vip: (amount) => amount * 0.20,
};

function calculateDiscount(type, amount) {
  const strategy = discountStrategies[type];
  if (!strategy) throw new Error(`Unknown type: ${type}`);
  return strategy(amount);
}

console.log(calculateDiscount('regular', 1000));
console.log(calculateDiscount('vip', 1000));

// Naya tier add karna:
discountStrategies.platinum = (amount) => amount * 0.30;
console.log(calculateDiscount('platinum', 1000));

// calculateDiscount() ka body EK baar bhi nahi badla. Yehi Open/Closed hai – aur yeh
// literally Strategy pattern hai (Lesson 1). OCP hi wo "why" hai jiske liye Strategy
// exist karta hai.

```

🔧 exercise.js — your turn

```

// 🧑‍🔧 EXERCISE – shipping cost calculator ko OCP-compliant banao.
// Chalao: node exercise.js

// ----- violation (isse refactor karna hai) -----
function shippingCost(method, weightKg) {
  if (method === 'standard') return 50;
  if (method === 'express') return 50 + 10 * weightKg;
  if (method === 'overnight') return 200;
  throw new Error(`Unknown method: ${method}`);
}

```

```
}
console.log(shippingCost('express', 3));

// =====
// TODO 1: convert this into a `shippingStrategies` registry object (like discountStrategies
//         in 2-fixed.js), and a thin `shippingCost()` that just looks up + calls.
//
// TODO 2: add a new 'international' method (flat ₹500 + ₹50/kg) WITHOUT touching the
//         shippingCost() function body – only add a new registry entry.
//
// TODO 3 (bonus): what's the difference between this exercise and Lesson 1's Strategy
//         exercise? (Hint: there isn't one – that's the point. OCP is the PRINCIPLE, Strategy
//         is the PATTERN that satisfies it. Same code, different Lens.)
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

26. Liskov Substitution Principle (LSP)

Rule: agar **B** **A** ko extend karta hai, to jahan bhi **A** expected hai, **B** ko wahan **safely substitute** kiya ja sakna chahiye — bina behavior galat kiye.

Problem: `extends` sirf syntax hai, guarantee nahi. Agar subclass base class ka contract (implicit assumptions) todta hai, to code jo base class ke against likha gaya tha, subclass ke saath silently galat behave karega.

Idea: agar "is-a" relationship genuinely FIT nahi karta (Square technically Rectangle nahi hai, uski apni constraint hai), to inheritance mat force karo — dono ko ek common, honest interface follow karwao.

Files — is order mein chalao

```
node 1-violation.js    # ❌ Square, Rectangle ka contract todta hai
node 2-fixed.js        # ✅ dono independent Shape implementations
node exercise.js       # 🔄 tumhara turn (classic Bird/Penguin problem)
```

Yeh design patterns se kaise judta hai

[Template Method](#) aur [Composite](#) dono LSP

pe depend karte hain — dono mein subclasses ko base class ki jagah interchangeably use kiya jaata hai. Agar LSP toot jaaye, in patterns ka poora point khatam ho jaata hai.

When to skip

Yeh koi "kabhi skip karo" wali cheez nahi hai — LSP todna almost hamesha ek design bug hai, optimization nahi. Agar lag raha hai ki tumhe LSP todna PAD raha hai, usually iska matlab hai ki `extends` relationship hi galat hai — jaisa is lesson mein Square/Rectangle ka tha.

❌ 1-violation.js — the violation

```
// ❌ VIOLATION — Square, Rectangle ko "extends" karta hai, lekin base class ke contract
// (setWidth aur setHeight INDEPENDENT hain) ko todta hai.
// Chalao: node 1-violation.js

class Rectangle {
  setWidth(w) { this.width = w; }
```

```

    setHeight(h) { this.height = h; }
    area() { return this.width * this.height; }
  }

  class Square extends Rectangle {
    setWidth(w) { this.width = w; this.height = w; } // side-effect: height bhi badal gaya!
    setHeight(h) { this.width = h; this.height = h; }
  }

  function resizeAndCheck(rect) {
    rect.setWidth(4);
    rect.setHeight(5);
    console.log(`expected area 20, got ${rect.area()}`);
  }

  resizeAndCheck(new Rectangle()); // 20 - sahi
  resizeAndCheck(new Square());    // 25 - GALAT! Square base class ka contract nahi follow karta

  // PROBLEM: Square TECHNICALLY Rectangle "extends" karta hai, Lekin jahan bhi Rectangle
  // expected hai, Square substitute karne se code galat behave karta hai. Yeh Liskov
  // Substitution ko todta hai - "subtype ko base type ki jagah use karna SAFE hona chahiye."

```

✅ 2-fixed.js — the fix

```

// ✅ FIXED - Square, Rectangle nahi hai (is-a relationship galat thi). Dono ek common
// Shape interface follow karte hain, koi inheritance-via-force nahi.
// Chalao: node 2-fixed.js

class Shape {
  area() { throw new Error('subclass must implement area()'); }
}

class Rectangle extends Shape {
  constructor(width, height) { super(); this.width = width; this.height = height; }
  area() { return this.width * this.height; }
}

class Square extends Shape {
  constructor(side) { super(); this.side = side; }
  area() { return this.side * this.side; }
}

function printArea(shape) { console.log(`area: ${shape.area()}`); }

printArea(new Rectangle(4, 5)); // 20
printArea(new Square(4));       // 16 - apna khud ka correct contract, koi surprise nahi

// Ab Square, Rectangle ki jagah "substitute" hone ka dawa hi nahi karta - dono independent
// Shape implementations hain. Jahan bhi Shape expected hai, dono safely use ho sakte hain
// kyunki koi hidden extra constraint (width==height) base contract se Leak nahi ho raha.

```

🔗 exercise.js — your turn

```
// 🐦 EXERCISE – classic Bird/Penguin problem. Fix the LSP violation.
// Chalao: node exercise.js

// ----- violation (isse refactor karna hai) -----
class Bird {
  fly() { console.log('🐦 flying high'); }
}
class Sparrow extends Bird {}
class Penguin extends Bird {
  fly() { throw new Error('penguins cannot fly!'); } // breaks the base contract
}

function makeBirdFly(bird) { bird.fly(); } // written assuming ALL Birds can fly()
makeBirdFly(new Sparrow());
try { makeBirdFly(new Penguin()); } catch (e) { console.log(`❌ ${e.message}`); }

// =====
// TODO 1: `Penguin extends Bird` is the LSP violation – Penguin cannot honestly promise
//         everything Bird promises. Split `Bird` into a base with only what's TRUE for every
//         bird (e.g. `eat()`, `layEggs()`), and a separate `FlyingBird` (extends Bird, adds
//         `fly()`) that only flying birds extend.
//
// TODO 2: make Sparrow extend FlyingBird, and Penguin extend Bird (NOT FlyingBird). Rewrite
//         makeBirdFly() to only accept FlyingBird – this should now be a compile-time-shaped
//         guarantee (well, JS doesn't compile-check, but the TYPE relationship should make
//         it obviously wrong to pass a Penguin there).
//
// TODO 3 (bonus): where else in the wild have you seen a subclass override a method just to
//         throw/no-op? That's almost always an LSP smell – name one example from a codebase
//         you've worked in (or a hypothetical one) in a comment.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====
```

27. Interface Segregation Principle (ISP)

Rule: clients ko un methods pe depend karne ke liye **force nahi** karna chahiye jo woh use hi nahi karte. Chhote, focused interfaces banao — ek "fat" interface mat banao.

Problem: ek badi class/interface jo sab kuch ek saath expect karti hai, kuch implementations ko methods implement karne padte hain jo unpe apply hi nahi hote (aur usually sirf ek error throw karte hain).

Idea: interface ko split karo taaki har implementer sirf wahi depend kare jo usko GENUINELY chahiye.

Files — is order mein chalao

```
node 1-violation.js    # ❌ ek fat Worker interface, RobotWorker ko eat() force
node 2-fixed.js       # ✅ do chhoti, focused interfaces
node exercise.js      # 🎯 tumhara turn
```

Yeh design patterns se kaise judta hai

[Adapter](#) aur [Bridge](#) dono ISP ko respect karte hain —

dono chhote, single-purpose contracts (`pay()` , `deliver()`) define karte hain instead of ek bada do-everything interface. Fat interfaces banate hi Adapter/Bridge likhna mushkil ho jaata hai kyunki wrap karne ke liye bohot saare irrelevant methods aa jaate hain.

When to skip

Agar sab implementers GENUINELY sab methods use karte hain (koi bhi kisi method ko fake/throw

nahi kar raha), interface split karna sirf extra files banayega bina fayde ke. ISP ka signal hai: koi implementation ek method ko empty/throw/no-op kar rahi hai sirf "interface follow karne" ke liye.

❌ 1-violation.js — the violation

```
// ❌ VIOLATION – ek "fat" interface (base class jo contract set karti hai) sabko FORCE
// karta hai methods implement karne ke liye jo unpe apply hi nahi hote.
// Chalao: node 1-violation.js

class Worker {
  work() { throw new Error('must implement work()'); }
  eat() { throw new Error('must implement eat()'); }
}

class HumanWorker extends Worker {
  work() { console.log('🧑 human working'); }
  eat() { console.log('🍽️ human eating lunch'); }
}

class RobotWorker extends Worker {
  work() { console.log('🤖 robot working'); }
  eat() { throw new Error('robots dont eat!'); } // forced to implement, sirf error throw karne
  // ke liye
}

const workers = [new HumanWorker(), new RobotWorker()];
workers.forEach(w => w.work());
workers.forEach(w => {
  try { w.eat(); } catch (e) { console.log(`❌ ${e.message}`); }
});

// PROBLEM: RobotWorker ko eat() implement karna PADTA hai sirf isliye kyunki Worker
// interface mein hai – chahe uska koi matlab na ho. Yeh RobotWorker ko ek method pe
// "depend" karwata hai jo use kabhi chahiye hi nahi.
```

✅ 2-fixed.js — the fix

```
// ✅ FIXED – do chhoti, FOCUSED interfaces. Har worker sirf woh implement karta hai jo
// usko genuinely chahiye.
// Chalao: node 2-fixed.js

// JS mein multiple inheritance nahi hai, so hum "interface" ko sirf convention (duck typing)
// ke through split karte hain – jisko jo method chahiye wahi implement karo:

class HumanWorker {
  work() { console.log('🧑 human working'); }
  eat() { console.log('🍽️ human eating lunch'); }
}

class RobotWorker {
  work() { console.log('🤖 robot working'); } // eat() ka koi zikar tak nahi – zaroorat hi nahi
}

const workers = [new HumanWorker(), new RobotWorker()];
workers.forEach(w => w.work());

// Sirf woh workers jo genuinely "eat" karte hain unhi ko eat() call karo:
const eaters = workers.filter(w => typeof w.eat === 'function');
```

```
eaters.forEach((w) => w.eat());
```

```
// RobotWorker ab kisi aise method pe depend hi nahi karta jo usse relevant nahi. Interface  
// (yahan: "jo methods class pe expect kiye jaate hain") ko SPLIT kiya, "ek size fits all"  
// force nahi kiya.
```

exercise.js — your turn

```
// 🧑‍🔬 EXERCISE – ek fat Printer interface ko segregate karo.  
// Chalao: node exercise.js  
  
// ----- violation (isse refactor karna hai) -----  
class MultiFunctionPrinter {  
  print(doc) { console.log(`🖨️ printing ${doc}`); }  
  scan(doc) { console.log(`📄 scanning ${doc}`); }  
  fax(doc) { console.log(`📠 faxing ${doc}`); }  
}  
class BasicPrinter {  
  print(doc) { console.log(`🖨️ printing ${doc}`); }  
  scan() { throw new Error('this printer cannot scan!'); } // forced, unused  
  fax() { throw new Error('this printer cannot fax!'); } // forced, unused  
}  
  
new MultiFunctionPrinter().print('report.pdf');  
try { new BasicPrinter().scan('report.pdf'); } catch (e) { console.log(`❌ ${e.message}`); }  
  
// =====  
// TODO 1: BasicPrinter should only need to implement `print()`. Restructure so it's not  
//         forced to define scan()/fax() at all (delete those methods from it entirely –  
//         nothing should call them on a BasicPrinter, so nothing should break).  
//  
// TODO 2: write a function `printIfPossible(device, doc)` that prints via `device.print(doc)`  
//         – it should work for BOTH BasicPrinter and MultiFunctionPrinter without either  
//         needing a shared "fat" base class.  
//  
// TODO 3 (bonus): write a function `scanIfPossible(device, doc)` that checks  
//         `typeof device.scan === 'function'` before calling it (like the `eaters` filter in  
//         2-fixed.js) – call it with both printer types and confirm it doesn't crash on  
//         BasicPrinter.  
//  
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.  
// =====
```

28. Dependency Inversion Principle (DIP)

Rule: high-level modules (business logic) ko low-level modules (DB, email provider, file system) pe DIRECTLY depend nahi karna chahiye. **Dono** ko ek abstraction (contract) pe depend karna chahiye.

Problem: jab high-level class kisi concrete low-level class ko khud `new` karti hai, to woh usi implementation se hamesha ke liye tightly coupled ho jaati hai — DB badalna, ya unit test mein fake dependency use karna, mushkil ho jaata hai.

Idea: dependency ko `new` karne ke bajaye BAHAR se inject karo (constructor param). High-level class sirf itna jaanti hai ki "mujhe kuch chahiye jiska `save()` method ho" — kaunsi concrete class hai, usse fark nahi padta.

Files — is order mein chalao

```
node 1-violation.js    # ❌ OrderService khud MySQLDatabase "new" karta hai
node 2-fixed.js        # ✅ database inject hoti hai, kaunsi hai fark nahi padta
node exercise.js       # 🎯 tumhara turn
```

Yeh design patterns se kaise judta hai

Yeh technique — dependency ko constructor se inject karna — **Dependency Injection** kehlata

hai, aur yeh wahi mechanism hai jo [Adapter](#) aur

[Abstract Factory](#) ko kaam karne laayak banata hai. Yeh bhi wahi

wajah hai ki `2-fixed.js` mein `InMemoryDatabase` se **testing** itni easy ho jaati hai —

DIP hi unit-testable code ki foundation hai.

When to skip

Agar koi dependency GENUINELY kabhi nahi badlegi aur test mein fake karne ki zaroorat nahi

(e.g. `Math`, `console`), inject karna sirf ceremony hai. DIP tab lagao jab dependency swap-able ya mock-able hona genuinely valuable ho.

❌ 1-violation.js — the violation

```
// ❌ VIOLATION – high-level OrderService, low-level MySQLDatabase pe DIRECTLY depend karta
// hai. DB badalna ya test karna mushkil hai.
// Chalao: node 1-violation.js

class MySQLDatabase {
  save(order) { console.log(`💾 [MySQL] saved order for ₹${order.amount}`); }
}

class OrderService {
  constructor() {
    this.db = new MySQLDatabase(); // hardcoded – OrderService ko pata hai KAUNSA DB hai
  }
  placeOrder(amount) {
    const order = { amount };
    this.db.save(order);
  }
}

const service = new OrderService();
service.placeOrder(999);

// PROBLEM: OrderService (high-level business logic) MySQLDatabase (low-level detail) ko
// directly "new" kar raha hai. Kal ko Postgres pe switch karna ho, ya unit test mein fake
// DB use karna ho – OrderService ko khud edit karna padega. High-level policy, low-level
// detail se tightly coupled hai.
```

✅ 2-fixed.js — the fix

```
// ✅ FIXED – OrderService kisi bhi "database jaisi cheez" pe depend karta hai (jiska sirf
// save() method ho), concrete MySQLDatabase pe nahi. Dependency BAHAR se inject hoti hai.
// Chalao: node 2-fixed.js

class MySQLDatabase {
  save(order) { console.log(`💾 [MySQL] saved order for ₹${order.amount}`); }
}

class PostgresDatabase {
  save(order) { console.log(`💾 [Postgres] saved order for ₹${order.amount}`); }
}

class InMemoryDatabase { // tests ke liye – real DB ki zaroorat nahi
  constructor() { this.orders = []; }
  save(order) { this.orders.push(order); console.log(`💾 [InMemory] saved order for ₹
  ${order.amount}`); }
}

class OrderService {
  constructor(database) { this.db = database; } // dependency INJECTED, hardcoded nahi
  placeOrder(amount) {
    const order = { amount };
    this.db.save(order);
  }
}
```

```

}

new OrderService(new MySQLDatabase()).placeOrder(999);
new OrderService(new PostgresDatabase()).placeOrder(499);

const testDb = new InMemoryDatabase();
new OrderService(testDb).placeOrder(199);
console.log('test db captured:', testDb.orders);

// OrderService ka code EK baar bhi nahi badla DB switch karne ke liye. Dono (high-level
// OrderService, low-level *Database classes) ab ek "abstraction" (save(order) contract) pe
// depend karte hain, ek doosre pe nahi. Yehi Dependency Inversion hai.

```

exercise.js — your turn

```

// 🧑‍💻 EXERCISE – NotificationService ko DIP-compliant banao.
// Chalao: node exercise.js

// ----- violation (isse refactor karna hai) -----
class EmailSender {
  send(to, msg) { console.log(`📧 emailing ${to}: ${msg}`); }
}

class NotificationService {
  constructor() {
    this.sender = new EmailSender(); // hardcoded – sirf email kabhi bhi hardcoded ho gaya
  }
  notify(to, msg) { this.sender.send(to, msg); }
}

new NotificationService().notify('alice@example.com', 'Your order shipped');

// =====
// TODO 1: change NotificationService's constructor to accept a `sender` parameter instead of
//         hardcoding `new EmailSender()`.
//
// TODO 2: write an `SmsSender` class with the same `send(to, msg)` contract, and show
//         NotificationService working with BOTH EmailSender and SmsSender without any change
//         to NotificationService itself.
//
// TODO 3 (bonus): write a `FakeSender` (captures messages in an array instead of really
//                 sending) and show how it would let you unit-test NotificationService.notify()
//                 without actually sending an email or SMS – assert the fake captured the right
//                 message.
//
// Neeche apna solution likho, phir `node exercise.js` chala kar verify karo.
// =====

```

LLD Practice — Machine Coding Interview Problems

The 23 lessons one level up teach you the *vocabulary* (Strategy, State, Decorator...). This folder is where that vocabulary gets used for real: classic "design X" machine-coding questions, each solved with the smallest set of patterns that actually earns its keep — not pattern soup.

How to use this module

1. Open a problem's `README.md` — read ONLY the Functional Requirements section first.
2. Before looking at `design.js`, spend 10–15 minutes sketching your own classes on paper — this is what the actual interview feels like.
3. Compare against `design.js`, run it (`node design.js`), then read "What an interviewer is watching for" — that's the part most people skip and it's the highest-signal section.
4. Do `exercise.js` — extending a design cleanly is what actually proves you understand *why* the pattern was chosen, not just that you can name it.

Problems

#	Problem	Core pattern(s)	Also asked at
1	Parking Lot	Singleton, Strategy	Amazon, Uber, Ola
2	Splitwise	Strategy, Facade	Uber, Flipkart, Swiggy
3	Elevator System	State	Google, Microsoft, Amazon
4	Rate Limiter	Strategy	Stripe, Uber, API platforms
5	Movie Ticket Booking	State, Facade	Amazon, BookMyShow
6	Notification Service	Factory, Decorator	Uber, Swiggy, fintechs

The habit that actually matters

Every `design.js` in this folder was written the "wrong way round" on purpose — read the naive instinct in each README's requirements, then notice how few patterns were actually needed. Real interviews penalize pattern-stuffing as much as no-pattern spaghetti. The question is never "which patterns do I know" — it's "what in THIS design might change, and what's the cheapest way to isolate that change."

29. Parking Lot System

Asked at: Amazon, Uber, Ola, most product companies (extremely common opener).

Functional Requirements

- Multiple vehicle types: motorcycle, car, truck (each needs a spot of matching-or-larger size)
- Multiple levels, each with a fixed set of spots
- Park a vehicle → find an available, correctly-sized spot → issue a ticket
- Unpark a vehicle → free the spot → calculate the fee based on duration and vehicle type

Patterns used (and why)

- **Singleton** — a physical building has exactly one `ParkingLot`. No reason two instances should ever disagree about which spots are free.
- **Strategy** — fee calculation differs per vehicle type. Kept as a swappable registry so a pricing change never touches `ParkingLot`.
- **Composite-ish containment** — `ParkingLot` → `Level` → `ParkingSpot` is a simple hierarchy; not full Composite (Lesson 11) since spots and levels don't share one interface, but the same "contain and delegate" instinct applies.

Files

```
node design.js      # the full working design + a runnable demo
node exercise.js    # 🚗 extend it – add a vehicle type + a decorator-based discount
```

What an interviewer is watching for

1. Do you ask clarifying questions first (how many levels? can a bigger spot hold a smaller vehicle? is pricing flat or per-hour?) before writing a class?
2. Do you separate "what can vary" (pricing, spot allocation) from "what's structural" (`ParkingLot`/`Level`/`Spot`)? That separation is exactly what Strategy buys you.
3. Can you extend the design live (new vehicle type, new pricing rule) without a rewrite? That's the actual exercise below.

✅ **design.js — the full design**

```
// P PARKING LOT SYSTEM – classic LLD interview question
// Chalao: node design.js
//
// See README.md for functional requirements and which patterns are used.

const SIZE = { MOTORCYCLE: 1, CAR: 2, TRUCK: 3 };

class Vehicle {
  constructor(licensePlate, type) {
    this.licensePlate = licensePlate;
    this.type = type; // 'motorcycle' | 'car' | 'truck'
    this.size = SIZE[type.toUpperCase()];
  }
}

class ParkingSpot {
  constructor(id, size) {
    this.id = id;
    this.size = size; // a spot fits vehicles of this size OR SMALLER (a car can use a truck spot)
    this.vehicle = null;
  }
  isAvailable() { return this.vehicle === null; }
  canFit(vehicle) { return this.isAvailable() && vehicle.size <= this.size; }
  park(vehicle) { this.vehicle = vehicle; }
  remove() { const v = this.vehicle; this.vehicle = null; return v; }
}

class Level {
  constructor(levelNumber, spots) {
    this.levelNumber = levelNumber;
    this.spots = spots;
  }
  findSpotFor(vehicle) {
    return this.spots.find((s) => s.canFit(vehicle)) ?? null;
  }
}

class Ticket {
  constructor(vehicle, spot) {
    this.vehicle = vehicle;
    this.spot = spot;
    this.entryTime = Date.now();
  }
}

// ---- Strategy: pricing per vehicle type, per hour ----
const pricingStrategies = {
  motorcycle: (hours) => hours * 10,
  car: (hours) => hours * 20,
  truck: (hours) => hours * 35,
};

// ---- Singleton: the parking lot itself ----
```

```

class ParkingLot {
  static #instance;
  constructor() {
    if (ParkingLot.#instance) return ParkingLot.#instance;
    this.levels = [];
    this.activeTickets = new Map(); // licensePlate -> Ticket
    ParkingLot.#instance = this;
  }
  static getInstance() { return ParkingLot.#instance ?? new ParkingLot(); }

  addLevel(level) { this.levels.push(level); }

  parkVehicle(vehicle) {
    for (const level of this.levels) {
      const spot = level.findSpotFor(vehicle);
      if (spot) {
        spot.park(vehicle);
        const ticket = new Ticket(vehicle, spot);
        this.activeTickets.set(vehicle.licensePlate, ticket);
        console.log(`✅ Parked ${vehicle.type} [${vehicle.licensePlate}] at Level
${level.levelNumber}, Spot ${spot.id}`);
        return ticket;
      }
    }
    console.log(`❌ Parking lot full – no spot for ${vehicle.type} [${vehicle.licensePlate}]`);
    return null;
  }

  unparkVehicle(licensePlate) {
    const ticket = this.activeTickets.get(licensePlate);
    if (!ticket) { console.log(`❌ No active ticket for ${licensePlate}`); return; }
    const hours = Math.max(1, (Date.now() - ticket.entryTime) / 3600000);
    const fee = pricingStrategies[ticket.vehicle.type](hours);
    ticket.spot.remove();
    this.activeTickets.delete(licensePlate);
    console.log(`🚗 Unparked ${licensePlate} – fee: ₹${fee.toFixed(2)}`);
    return fee;
  }
}

// ----- DEMO -----
const lot = ParkingLot.getInstance();
lot.addLevel(new Level(1, [
  new ParkingSpot('L1-S1', SIZE.MOTORCYCLE),
  new ParkingSpot('L1-S2', SIZE.CAR),
  new ParkingSpot('L1-S3', SIZE.TRUCK),
]));

const bike = new Vehicle('MH12AB1234', 'motorcycle');
const car = new Vehicle('MH12CD5678', 'car');
const truck = new Vehicle('MH12EF9999', 'truck');

lot.parkVehicle(bike);
lot.parkVehicle(car);
lot.parkVehicle(truck);

```

```
lot.parkVehicle(new Vehicle('MH12ZZ0000', 'car')); // no spot left that fits -> rejected

lot.unparkVehicle('MH12AB1234');
lot.unparkVehicle('MH12CD5678');

console.log('same singleton instance?', ParkingLot.getInstance() === lot);
```

exercise.js — your turn

```
// 🐛 EXERCISE – extend the parking lot design without rewriting it.
// Copy design.js above this comment (or just edit design.js directly), then:
//
// TODO 1: Add a 'van' vehicle type – size between car and truck – to the SIZE map and add a
//         matching entry to pricingStrategies. Confirm you did NOT have to touch ParkingLot,
//         Level, or ParkingSpot – only the two registries. That's Open/Closed in action.
//
// TODO 2: Add an "EV discount" – give Vehicle an `isEV` flag, and make EVs get 20% off
//         whatever pricing strategy applies. Hint: this is the Decorator pattern (Lesson 12)
//         applied to a Strategy FUNCTION instead of an object – write a
//         `withEvDiscount(pricingFn)` wrapper instead of editing each strategy.
//
// TODO 3 (bonus): today, if two vehicles try to park in the exact same millisecond, both could
//         call findSpotFor() before either calls park(), and both might get told the same spot
//         is free. Where would a real (multi-request, concurrent) system need a lock or
//         transaction that this single-threaded demo doesn't need? You don't need to implement
//         it – just identify the exact race window in the code.
```

30. Splitwise (Expense Sharing)

Asked at: Uber, Flipkart, Swiggy, and any company that wants to see you handle a non-trivial data model (not just objects — actual running balances).

Functional Requirements

- Add users to a group
- Add an expense paid by one user, split among others: **equal**, **exact amounts**, or **percentage**
- Track running balances: who owes whom, and how much
- Query the balance between any two users

Patterns used (and why)

- **Strategy** — the split calculation (`equal` / `exact` / `percent`) is the one piece that genuinely varies. Kept as a registry so a new split type never touches the ledger.
- **Facade** — `ExpenseManager` is the one entry point callers use; it hides the interaction between split strategies and the `Ledger`.

Files

```
node design.js      # the full working design + a runnable demo
node exercise.js    # 🚀 extend it – settlements + an Observer hook
```

What an interviewer is watching for

1. Do you net balances (if A owes B ₹50 and B owes A ₹20, the ledger should just say "A owes B ₹30"), or do you let debts pile up unnormalized? The `Ledger.addDebt` method here nets them — that's usually the detail that separates a strong answer from a mediocre one.
2. Do you validate `exact` / `percent` splits sum to the right total, or silently accept garbage?
3. Can you add a 4th split type (e.g. "by shares", like 2 shares vs 1 share) without touching `Ledger` or `ExpenseManager`?

✓ design.js — the full design

```
// 🧑 SPLITWISE-LITE – expense sharing system, classic LLD question
// Chalao: node design.js
//
```

```
// See README.md for functional requirements and which patterns are used.

class User {
  constructor(id, name) { this.id = id; this.name = name; }
}

// ---- Strategy: how to split an expense among participants ----
const splitStrategies = {
  equal: (amount, users) => {
    const share = amount / users.length;
    return users.map((u) => ({ user: u, amount: share }));
  },
  exact: (amount, users, extra) => {
    const total = extra.amounts.reduce((a, b) => a + b, 0);
    if (Math.abs(total - amount) > 0.01) throw new Error('exact amounts must sum to total');
    return users.map((u, i) => ({ user: u, amount: extra.amounts[i] }));
  },
  percent: (amount, users, extra) => {
    const total = extra.percents.reduce((a, b) => a + b, 0);
    if (Math.abs(total - 100) > 0.01) throw new Error('percents must sum to 100');
    return users.map((u, i) => ({ user: u, amount: (amount * extra.percents[i]) / 100 }));
  },
};

// ---- Ledger: tracks NET pairwise balances (never lets both directions grow at once) ----
class Ledger {
  #balances = new Map(); // key `${owesId}->${owedToId}` = amount owed

  #key(a, b) { return `${a}->${b}`; }

  addDebt(owesId, owedToId, amount) {
    if (owesId === owedToId || amount === 0) return;
    const opposite = this.#balances.get(this.#key(owedToId, owesId)) ?? 0;
    if (opposite >= amount) {
      this.#balances.set(this.#key(owedToId, owesId), opposite - amount);
    } else {
      this.#balances.delete(this.#key(owedToId, owesId));
      const existing = this.#balances.get(this.#key(owesId, owedToId)) ?? 0;
      this.#balances.set(this.#key(owesId, owedToId), existing + (amount - opposite));
    }
  }

  getBalance(aId, bId) {
    const aOwesB = this.#balances.get(this.#key(aId, bId)) ?? 0;
    const bOwesA = this.#balances.get(this.#key(bId, aId)) ?? 0;
    return aOwesB - bOwesA; // positive => a owes b
  }

  printAll(users) {
    for (const [key, amount] of this.#balances) {
      if (amount <= 0.001) continue;
      const [owesId, owedToId] = key.split('->');
      const owes = users.find((u) => u.id === owesId);
      const owedTo = users.find((u) => u.id === owedToId);
      console.log(`  ${owes.name} owes ${owedTo.name}: ₹${amount.toFixed(2)}`);
    }
  }
}
```

```

    }
  }
}

// ---- Facade over split strategies + Ledger ----
class ExpenseManager {
  constructor(users) { this.users = users; this.ledger = new Ledger(); }

  addExpense(paidBy, amount, splitType, participants, extra = {}) {
    const splits = splitStrategies[splitType](amount, participants, extra);
    for (const { user, amount: share } of splits) {
      if (user.id === paidBy.id) continue;
      this.ledger.addDebt(user.id, paidBy.id, share);
    }
    console.log(`✅ ${paidBy.name} paid ₹${amount} (${splitType} split among
    ${participants.length})`);
  }

  balanceBetween(a, b) {
    const bal = this.ledger.getBalance(a.id, b.id);
    if (bal === 0) { console.log(`${a.name} and ${b.name} are settled up`); return; }
    console.log(bal > 0 ? `${a.name} owes ${b.name} ₹${bal.toFixed(2)}` : `${b.name} owes
    ${a.name} ₹${(-bal).toFixed(2)}`);
  }

  printBalances() {
    console.log('-- balances --');
    this.ledger.printAll(this.users);
  }
}

// ----- DEMO -----
const alice = new User('u1', 'Alice');
const bob = new User('u2', 'Bob');
const carol = new User('u3', 'Carol');
const manager = new ExpenseManager([alice, bob, carol]);

manager.addExpense(alice, 300, 'equal', [alice, bob, carol]);
manager.addExpense(bob, 100, 'exact', [alice, bob], { amounts: [60, 40] });
manager.addExpense(carol, 500, 'percent', [alice, carol], { percents: [30, 70] });

manager.printBalances();
manager.balanceBetween(alice, bob);

```

exercise.js — your turn

```

// 🌟 EXERCISE – extend the Splitwise design without rewriting it.
// Copy design.js above this comment (or just edit design.js directly), then:
//
// TODO 1: Add a `settle(a, b)` method to ExpenseManager that ACTUALLY clears the balance
//         between two users (not just prints it) – it should call `ledger.addDebt()` with
//         whatever amount zeroes out the current balance, then log a "settled up" message.
//
// TODO 2: Add a 4th split type: 'shares' – participants split by relative shares instead of

```

```
//      equal/exact/percent (e.g. Alice=2 shares, Bob=1 share means Alice pays 2/3, Bob 1/3).  
//      Add it to splitStrategies ONLY – confirm ExpenseManager needs zero changes.  
//  
// TODO 3 (bonus): if you wanted every participant to get notified (console.log a message to  
//      them) whenever a new expense involving them is added, which pattern from the main  
//      course would you reach for, and where exactly would you hook it in? (You don't have  
//      to implement it – name the pattern and the hook point.)
```

31. Elevator System

Asked at: Google, Microsoft, Amazon — a favorite because it's a real state machine, not just a bag of objects.

Functional Requirements

- An elevator is Idle, Moving Up, Moving Down, or Doors Open at any moment
- External requests (someone presses a floor button) get queued
- The elevator processes requests one floor-step at a time, stopping when it reaches a requested floor

Patterns used (and why)

- **State** — this is the textbook State pattern use case. What `step()` and `onRequest()` do depends ENTIRELY on the elevator's current state. No `if (this.status === 'idle')` anywhere.
- (implicit) **Strategy** — "which floor to service next" is scheduling logic that could be swapped independently of the state machine itself (see the exercise).

Files

```
node design.js      # the full working design + a runnable demo (single elevator)
node exercise.js    # 🚀 extend it - multiple elevators + a dispatch strategy
```

What an interviewer is watching for

1. Do you model states as OBJECTS (like here) instead of a string + switch statement? A string `status` field with `if/else` everywhere is exactly the naive version Lesson 20 warns about.
2. Do transitions live inside the states themselves, not in the Elevator class?
3. Can you add multiple elevators + a dispatch algorithm without touching the state objects?

✓ design.js — the full design

```
// 📖 ELEVATOR SYSTEM - classic LLD question, State pattern in action
// Chalao: node design.js
//
// See README.md for functional requirements and which patterns are used.
```

```

class Elevator {
  constructor(id) {
    this.id = id;
    this.currentFloor = 0;
    this.requests = new Set(); // floors we still need to stop at
    this.state = idleState;
  }
  setState(state) {
    console.log(` elevator ${this.id}: ${this.state.name} -> ${state.name}`);
    this.state = state;
  }
  requestFloor(floor) {
    this.requests.add(floor);
    this.state.onRequest(this);
  }
  step() { this.state.step(this); }
}

const idleState = {
  name: 'IDLE',
  onRequest(elevator) {
    if (elevator.requests.size === 0) return;
    const target = [...elevator.requests][0];
    if (target > elevator.currentFloor) elevator.setState(movingUpState);
    else if (target < elevator.currentFloor) elevator.setState(movingDownState);
    else elevator.setState(doorsOpenState);
  },
  step() {},
};

const movingUpState = {
  name: 'MOVING_UP',
  onRequest() {}, // already moving - new requests just sit in the set until we pass them
  step(elevator) {
    elevator.currentFloor++;
    console.log(` elevator ${elevator.id} passing floor ${elevator.currentFloor}`);
    if (elevator.requests.has(elevator.currentFloor)) {
      elevator.requests.delete(elevator.currentFloor);
      elevator.setState(doorsOpenState);
    }
  },
};

const movingDownState = {
  name: 'MOVING_DOWN',
  onRequest() {},
  step(elevator) {
    elevator.currentFloor--;
    console.log(` elevator ${elevator.id} passing floor ${elevator.currentFloor}`);
    if (elevator.requests.has(elevator.currentFloor)) {
      elevator.requests.delete(elevator.currentFloor);
      elevator.setState(doorsOpenState);
    }
  },
};

```

```

};

const doorsOpenState = {
  name: 'DOORS_OPEN',
  onRequest() {},
  step(elevator) {
    console.log(` elevator ${elevator.id} doors closing at floor ${elevator.currentFloor}`);
    elevator.setState(idleState);
    elevator.state.onRequest(elevator); // immediately re-evaluate: anything else pending?
  },
};

// ----- DEMO -----
const elevator = new Elevator('E1');
elevator.requestFloor(3);
elevator.requestFloor(5);

let ticks = 0;
while ((elevator.requests.size > 0 || elevator.state !== idleState) && ticks < 20) {
  elevator.step();
  ticks++;
}
console.log(`done in ${ticks} steps, resting at floor ${elevator.currentFloor}`);

```

exercise.js — your turn

```

//  EXERCISE – extend the elevator design without touching the state objects.
// Copy design.js above this comment (or just edit design.js directly), then:
//
// TODO 1: Build an `ElevatorSystem` class that holds multiple Elevator instances and a
//         `dispatch(floor)` method – it should pick the IDLE elevator whose currentFloor is
//         closest to the requested floor (fall back to any elevator if none are idle), then
//         call `.requestFloor(floor)` on it. You should NOT need to change idleState,
//         movingUpState, movingDownState, or doorsOpenState at all.
//
// TODO 2: Add a MAINTENANCE state – an elevator in this state ignores all requests and
//         doesn't move. Add a `setMaintenanceMode(elevator, on)` helper that transitions in
//         and out of it. What has to happen to `elevator.requests` when maintenance mode ends?
//
// TODO 3 (bonus): the current design services requests in the order they were queued
//                 ([...requests][0]), not by "elevator's current direction" (the real SCAN/elevator
//                 algorithm). Sketch – in a comment, no need to fully implement – how you'd change
//                 `idleState.onRequest` to pick the nearest request in the current direction instead.

```

32. Rate Limiter

Asked at: Uber, Stripe, most API-platform companies — often phrased as "design an API rate limiter" in a system-design round, but the algorithm core is pure LLD/Strategy.

Functional Requirements

- Limit how many requests a client can make within a time window
- Support multiple algorithms interchangeably: fixed window, sliding window log, token bucket

Patterns used (and why)

- **Strategy** — each algorithm is a self-contained object with the same `allow(clientId)` contract. `RateLimiter` (the context) never knows which one is plugged in.

Files

```
node design.js      # three algorithms implemented + a runnable demo of each
node exercise.js    # 🚀 extend it – a 4th algorithm + per-client overrides
```

What an interviewer is watching for

1. Do you know the tradeoffs between algorithms? Fixed window is simple but allows a burst of 2x the limit right at the window boundary. Sliding window log is accurate but memory-heavy (stores every timestamp). Token bucket allows controlled bursts and is what most real systems (AWS, Stripe) actually use.
2. Is the algorithm truly swappable — same `allow()` contract, zero `if (algo === 'x')` in the caller?
3. Do you reason about per-client state (a `Map` keyed by `clientId`) vs. one global counter?

✅ design.js — the full design

```
// 🚀 RATE LIMITER – classic system-design/LLD question, Strategy pattern
// Chalao: node design.js
//
// See README.md for functional requirements, algorithms, and their tradeoffs.

class FixedWindowStrategy {
  constructor(limit, windowMs) { this.limit = limit; this.windowMs = windowMs; this.counters =
```

```

new Map(); }
allow(clientId) {
  const now = Date.now();
  const entry = this.counters.get(clientId);
  if (!entry || now - entry.windowStart >= this.windowMs) {
    this.counters.set(clientId, { windowStart: now, count: 1 });
    return true;
  }
  if (entry.count < this.limit) { entry.count++; return true; }
  return false;
}
}

class SlidingWindowLogStrategy {
  constructor(limit, windowMs) { this.limit = limit; this.windowMs = windowMs; this.log = new
Map(); }
  allow(clientId) {
    const now = Date.now();
    const timestamps = (this.log.get(clientId) ?? []).filter((t) => now - t < this.windowMs);
    if (timestamps.length < this.limit) {
      timestamps.push(now);
      this.log.set(clientId, timestamps);
      return true;
    }
    this.log.set(clientId, timestamps);
    return false;
  }
}

class TokenBucketStrategy {
  constructor(capacity, refillPerSec) { this.capacity = capacity; this.refillPerSec =
refillPerSec; this.buckets = new Map(); }
  allow(clientId) {
    const now = Date.now();
    const bucket = this.buckets.get(clientId) ?? { tokens: this.capacity, last: now };
    const elapsedSec = (now - bucket.last) / 1000;
    bucket.tokens = Math.min(this.capacity, bucket.tokens + elapsedSec * this.refillPerSec);
    bucket.last = now;
    if (bucket.tokens >= 1) { bucket.tokens -= 1; this.buckets.set(clientId, bucket); return
true; }
    this.buckets.set(clientId, bucket);
    return false;
  }
}

// ---- Context: doesn't know or care which algorithm is plugged in ----
class RateLimiter {
  constructor(strategy) { this.strategy = strategy; }
  setStrategy(strategy) { this.strategy = strategy; }
  allowRequest(clientId) { return this.strategy.allow(clientId); }
}

// ----- DEMO -----
const limiter = new RateLimiter(new FixedWindowStrategy(3, 1000)); // 3 req/sec
console.log('-- fixed window (3/sec) --');

```

```

for (let i = 1; i <= 5; i++) {
  console.log(`req ${i}:`, limiter.allowRequest('user-1') ? '✅ allowed' : '❌ blocked');
}

limiter.setStrategy(new TokenBucketStrategy(3, 1)); // capacity 3, refill 1/sec
console.log('-- token bucket (capacity 3, refill 1/sec) --');
for (let i = 1; i <= 5; i++) {
  console.log(`req ${i}:`, limiter.allowRequest('user-2') ? '✅ allowed' : '❌ blocked');
}

```

exercise.js — your turn

```

// 🛠️ EXERCISE – extend the rate limiter design without touching RateLimiter itself.
// Copy design.js above this comment (or just edit design.js directly), then:
//
// TODO 1: Implement a 4th strategy – SlidingWindowCounterStrategy – the hybrid approach real
//          systems favor: it splits time into fixed buckets (like FixedWindowStrategy) but
//          weights the previous bucket's count by how far into the current bucket we are,
//          approximating a true sliding window without storing every timestamp. Same `allow()`
//          contract as the others.
//
// TODO 2: Add per-client overrides – some clients (e.g. paid tier) should get a higher limit
//          than the default. Do this WITHOUT adding an `if (clientId === ...)` inside any
//          strategy – think about where a "limit lookup" belongs instead (hint: it's still the
//          strategy's job, just parameterized differently per client at construction time, or
//          via a small wrapper/decorator around any strategy).
//
// TODO 3 (bonus): which of the three original strategies would you pick for a public API that
//                  must run across multiple servers (not just one process)? What breaks about storing
//                  `Map`s in memory once you have more than one server instance?

```

33. Movie Ticket Booking (BookMyShow-lite)

Asked at: Amazon, BookMyShow, most consumer-app companies — good for testing whether you

handle CONCURRENCY-shaped problems (double booking) even in a single-threaded demo.

Functional Requirements

- A show has a fixed set of seats
- A user selects seats → they get **locked** (held) for a short time
- User confirms payment → seats become **booked**
- Locking is all-or-nothing: if any requested seat can't be locked, none are

Patterns used (and why)

- **State** — a seat's allowed transitions depend entirely on its current state (Available → Locked → Booked). A booked seat structurally CANNOT be locked again by someone else — there's no `if` checking that anywhere, the state object itself refuses.
- **Facade** — `BookingService` is the one entry point over seat-locking + confirmation.

Files

```
node design.js      # the full working design + a runnable demo
node exercise.js    # 🔄 extend it – cancellation as an explicit transition
```

What an interviewer is watching for

1. Do you make double-booking a seat structurally impossible (state objects), or just "check a boolean and hope nobody races you"?
2. Is seat-locking all-or-nothing? (Lock A1+A2, but A2 is taken → A1 must be released too.)
3. Do you treat "booked → available" as ONE transition (`release`) or do you correctly see that a hold timing out and a user cancelling a paid booking are different business events that probably need different rules (refunds, etc.)? See the exercise.

✅ design.js — the full design

```
// 🎬 MOVIE TICKET BOOKING – BookMyShow-Lite, classic LLD question
// Chalao: node design.js
//
// See README.md for functional requirements and which patterns are used.

const availableState = {
  name: 'AVAILABLE',
  lock(seat) { seat.setState(lockedState); return true; },
  book() { return false; },
  release() { return false; },
};

const lockedState = {
  name: 'LOCKED',
  lock() { return false; }, // already locked by someone else
  book(seat) { seat.setState(bookedState); return true; },
  release(seat) { seat.setState(availableState); return true; }, // hold timed out / user backed
  out
};

const bookedState = {
  name: 'BOOKED',
  lock() { return false; },
  book() { return false; },
  release() { return false; }, // booked seats need an explicit cancellation flow, not a plain
  release
};

class Seat {
  constructor(id) { this.id = id; this.state = availableState; }
  setState(state) { this.state = state; }
  lock() { const ok = this.state.lock(this); console.log(` seat ${this.id}: lock -> ${ok ?
this.state.name : 'DENIED'}`); return ok; }
  book() { const ok = this.state.book(this); console.log(` seat ${this.id}: book -> ${ok ?
this.state.name : 'DENIED'}`); return ok; }
  release() { const ok = this.state.release(this); console.log(` seat ${this.id}: release ->
${ok ? this.state.name : 'DENIED'}`); return ok; }
}

class Show {
  constructor(id, movieName, seatIds) {
    this.id = id;
    this.movieName = movieName;
    this.seats = new Map(seatIds.map((sid) => [sid, new Seat(sid)]));
  }
}

// ---- Facade ----
class BookingService {
  lockSeats(show, seatIds) {
    const seats = seatIds.map((id) => show.seats.get(id));
    const locked = [];
    for (const seat of seats) {
      if (seat.lock()) locked.push(seat);
      else { locked.forEach((s) => s.release()); return null; } // all-or-nothing
    }
  }
}
```

```

    return locked;
  }
  confirmBooking(seats) {
    const allBooked = seats.every((s) => s.book());
    console.log(allBooked ? '✅ booking confirmed' : '❌ booking failed');
    return allBooked;
  }
}

// ----- DEMO -----
const show = new Show('S1', 'Inception', ['A1', 'A2', 'A3']);
const service = new BookingService();

const locked = service.lockSeats(show, ['A1', 'A2']);
if (locked) service.confirmBooking(locked);

// someone else tries to lock an already-booked seat – structurally denied
service.lockSeats(show, ['A1']);

```

exercise.js — your turn

```

// ⚙️ EXERCISE – extend the booking design without weakening the state machine.
// Copy design.js above this comment (or just edit design.js directly), then:
//
// TODO 1: bookedState.release() deliberately returns false – a plain "release" shouldn't undo
//         a paid booking. Add a SEPARATE `cancel()` transition (distinct from `release()`) that
//         moves BOOKED -> AVAILABLE, and log a "refund initiated" message when it happens.
//         LockedState and availableState should NOT support `cancel()` (only booked seats can
//         be cancelled – you can't cancel something that was never booked).
//
// TODO 2: Add a `lockedAt` timestamp on Seat, and a `releaseExpiredLocks(show, maxAgeMs)`
//         function on BookingService that walks all seats and releases any LOCKED seat older
//         than maxAgeMs. This simulates what a real background job would do for abandoned
//         carts.
//
// TODO 3 (bonus): why is "all-or-nothing" Locking (see LockSeats) important here? Write a
//                 2-sentence explanation of what would go wrong for a user if Locking were NOT
//                 all-or-nothing.

```

34. Notification Service

Asked at: most companies with any kind of alerting/messaging feature (Uber, Swiggy, fintechs).

Functional Requirements

- Send notifications over multiple channels: Email, SMS, Push
- Some notifications need retry-on-failure and delivery logging; others don't — and that extra behavior shouldn't be baked into the channels themselves

Patterns used (and why)

- **Factory** — picks the right channel object from a type string; adding a channel is a one-line registry entry.
- **Decorator** — `RetryDecorator` and `LoggingDecorator` wrap a channel with extra behavior WITHOUT touching the channel's own code, and can be stacked in any combination (same idea as Lesson 12's coffee add-ons, applied to a "send" behavior instead of a drink).

Files

```
node design.js      # the full working design + a runnable demo
node exercise.js    # 🚀 extend it – a new channel + a new decorator
```

What an interviewer is watching for

1. Do "retry" and "logging" live INSIDE each channel (duplicated 3x), or are they cross-cutting concerns applied once, generically, to any channel? Decorator is the whole point here.
2. Is adding a channel (Slack, WhatsApp) a one-line registry addition, or does it require editing a big switch statement?
3. Can decorators be composed in different orders, and do you understand what changes about behavior when you do (e.g. `Logging(Retry(channel))` logs once per attempt vs. `Retry(Logging(channel))` logs each individual attempt)?

✅ design.js — the full design

```
// 📌 NOTIFICATION SERVICE – multi-channel notifications, classic LLD question
// Chalao: node design.js
```

```
//
// See README.md for functional requirements and which patterns are used.

class EmailChannel { send(to, msg) { console.log(`✉ Email to ${to}: ${msg}`); return true; } }
class SmsChannel { send(to, msg) { console.log(`📱 SMS to ${to}: ${msg}`); return true; } }
class PushChannel {
  send(to, msg) {
    const ok = Math.random() > 0.4; // pretend push occasionally fails (device offline etc.)
    console.log(ok ? `🔔 Push to ${to}: ${msg}` : `🔔 Push to ${to}: FAILED (device offline)`);
    return ok;
  }
}

// ---- Factory ----
function createChannel(type) {
  const channels = { email: EmailChannel, sms: SmsChannel, push: PushChannel };
  const ChannelClass = channels[type];
  if (!ChannelClass) throw new Error(`Unknown channel: ${type}`);
  return new ChannelClass();
}

// ---- Decorators: same `send()` interface as a plain channel ----
class RetryDecorator {
  constructor(channel, maxAttempts = 3) { this.channel = channel; this.maxAttempts = maxAttempts; }
  send(to, msg) {
    for (let attempt = 1; attempt <= this.maxAttempts; attempt++) {
      if (this.channel.send(to, msg)) return true;
      if (attempt < this.maxAttempts) console.log(` retrying... (attempt ${attempt} + 1/${this.maxAttempts})`);
    }
    return false;
  }
}

class LoggingDecorator {
  constructor(channel) { this.channel = channel; }
  send(to, msg) {
    console.log(` [log] sending to ${to} at ${new Date().toISOString()}`);
    const ok = this.channel.send(to, msg);
    console.log(` [log] result: ${ok ? 'delivered' : 'failed'}`);
    return ok;
  }
}

// ----- DEMO -----
const plainEmail = createChannel('email');
plainEmail.send('alice@example.com', 'Your order shipped');

let reliablePush = createChannel('push');
reliablePush = new LoggingDecorator(new RetryDecorator(reliablePush, 3));
reliablePush.send('device-123', 'Flash sale starts now!');
```

```
// 🌀 EXERCISE – extend the notification design without touching existing channels/decorators.
// Copy design.js above this comment (or just edit design.js directly), then:
//
// TODO 1: Add a 'slack' channel to the createChannel() registry (a SlackChannel class with the
//         same `send(to, msg)` contract). Confirm you didn't need to touch EmailChannel,
//         SmsChannel, or PushChannel.
//
// TODO 2: Write a TimeoutDecorator – it should call the wrapped channel's send(), and if it
//         doesn't resolve fast enough, log a timeout message and return false. (Since our
//         `send()` is synchronous here, simulate "slow" with a random chance instead of a real
//         timer – the point is the DECORATOR SHAPE, not real async timing.)
//
// TODO 3 (bonus): wrap the same push channel two different ways –
//         `new LoggingDecorator(new RetryDecorator(channel))` vs.
//         `new RetryDecorator(new LoggingDecorator(channel))` – run both and describe in a
//         comment what's different about the log output between the two orderings.
```

PART IV — SYSTEM DESIGN (HLD)

System Design (HLD) — Primer

Patterns and SOLID teach you to design a single service well. LLD practice teaches you to design one class hierarchy well. This is the layer above both: how you design a system that runs across MANY machines, where the failure modes are different (networks partition, servers die, data has to live somewhere and get copied around).

Unlike the other modules, most of these topics can't be fully "solved" with a clean class — they're fundamentally about **tradeoffs**. So each lesson pairs a runnable simulation (to make the mechanism concrete) with a README that's honest about "it depends."

How to use this module

1. Read the README's "Core idea(s)" section first.
2. Run `simulate.js` and actually read the output — several of these are designed to show you a concrete, measurable number (e.g. "only 2/20 keys moved") rather than just asserting a claim.
3. Do `exercise.js` — several of these ask you to extend toward what a REAL system (SQS, DynamoDB, Redis) actually does, past the simplified demo.

Topics

#	Topic	Also asked at
1	Load Balancing	every company with >1 server
2	Caching Strategies (LRU)	Amazon, Google, everyone
3	Consistent Hashing	Amazon, Uber, distributed-systems teams
4	DB Sharding & Replication	any company past 1 DB server
5	CAP Theorem & Consistency	senior/staff system-design rounds

#	Topic	Also asked at
6	Message Queues & Async	Uber, Swiggy, background-jobs-heavy systems

How this connects to the rest of the course

- Load Balancing's health-check exercise reuses **Decorator** (Lesson 12).
- Pub-Sub in the Message Queues lesson IS **Observer** (Lesson 2), just across services.
- Consistent Hashing is the real mechanism behind sticky-session **Strategy** routing (Lesson 1 / HLD Lesson 1).
- CAP's replication-lag demo is the exact same mechanism as the Sharding & Replication lesson's `ReplicatedStore` — same code idea, viewed from a different angle.

Real system-design interviews expect you to move fluidly between these layers: "I'd shard by hash for even load (HLD 4), use consistent hashing so resharding is cheap (HLD 3), and cache hot reads with an LRU eviction policy (HLD 2)" is a sentence that uses three of these lessons at once — that fluency is the actual goal, not memorizing each topic in isolation.

35. Load Balancing

Asked at: every company with more than one server. Often the FIRST thing you draw in a system-design round, and interviewers probe whether you know the algorithm tradeoffs, not just the box-and-arrow diagram.

Core idea

A load balancer sits in front of a pool of servers and decides which one handles each incoming request. The algorithm you pick has real consequences:

- **Round Robin** — dead simple, cycles through servers evenly. Bad if servers have different capacity, or if some requests are much heavier than others.
- **Weighted Round Robin** — like Round Robin, but bigger/faster servers get proportionally more requests.
- **Least Connections** — send the request to whichever server currently has the fewest active connections. Adapts to uneven request duration, unlike Round Robin.
- **Consistent Hashing / IP Hash** — the same client (or key) always lands on the same server, which matters for **sticky sessions** or when a server holds local cache state. (Full implementation in [Lesson 3 — Consistent Hashing](#).)

Files

```
node simulate.js    # all 3 basic algorithms, run against the same server pool
node exercise.js    # 🚩 extend it — health checks + a hybrid strategy
```

What an interviewer is watching for

1. Do you know Round Robin can overload a slow/small server just as much as a fast one?
2. Do you know WHY sticky sessions matter (stateful in-memory session data) and what breaks without them (a user's session "disappears" if their next request hits a different server)?
3. Real load balancers (Nginx, AWS ELB, HAProxy) combine health checks with the algorithm — a server that's failing health checks should be pulled out of rotation. That's the exercise.

```
// 🐛 EXERCISE – extend the load balancer design without breaking existing strategies.
// Copy simulate.js above this comment (or just edit simulate.js directly), then:
//
// TODO 1: add a `healthy` flag (default true) to Server. Write a `HealthAwareBalancer`
//         wrapper that takes ANY of the three existing balancers and skips unhealthy
//         servers when picking – you should NOT need to modify RoundRobinBalancer,
//         WeightedRoundRobinBalancer, or LeastConnectionsBalancer themselves (this is the
//         Decorator pattern from Lesson 12, applied to a load balancer instead of a coffee).
//
// TODO 2: simulate server C going down mid-run (set its `healthy = false` partway through
//         a loop of pick() calls) and show that HealthAwareBalancer stops routing to it.
//
// TODO 3 (bonus): what would go wrong if a load balancer used Round Robin for a service
//                 where each server keeps user sessions in local memory (not a shared session
//                 store)? Write a 2-sentence explanation of the failure a user would actually see.
```

✅ simulate.js — the simulation

```
// 🏗️ LOAD BALANCING – simulate 3 balancing strategies against the same server pool
// Chalao: node simulate.js
//
// See README.md for what each algorithm is good/bad at.

class Server {
  constructor(id, weight = 1) { this.id = id; this.weight = weight; this.activeConnections = 0; }
}

// ---- Strategy: Round Robin ----
class RoundRobinBalancer {
  constructor(servers) { this.servers = servers; this.index = 0; }
  pick() {
    const server = this.servers[this.index % this.servers.length];
    this.index++;
    return server;
  }
}

// ---- Strategy: Weighted Round Robin ----
class WeightedRoundRobinBalancer {
  constructor(servers) {
    this.pool = servers.flatMap((s) => Array(s.weight).fill(s)); // heavier servers repeat more often
    this.index = 0;
  }
  pick() {
    const server = this.pool[this.index % this.pool.length];
    this.index++;
    return server;
  }
}

// ---- Strategy: Least Connections ----
class LeastConnectionsBalancer {
```

```

    constructor(servers) { this.servers = servers; }
    pick() {
        return this.servers.reduce((min, s) => (s.activeConnections < min.activeConnections ? s :
min));
    }
}

function simulateRequests(balancer, servers, count, label) {
    console.log(`-- ${label} --`);
    const hits = Object.fromEntries(servers.map((s) => [s.id, 0]));
    for (let i = 0; i < count; i++) {
        const server = balancer.pick();
        hits[server.id]++;
        server.activeConnections++;
        if (i % 3 === 0) server.activeConnections = Math.max(0, server.activeConnections - 1); //
some finish early
    }
    console.log('  requests per server:', hits);
}

const servers = [new Server('A', 1), new Server('B', 1), new Server('C', 2)]; // C is a bigger
machine

simulateRequests(new RoundRobinBalancer(servers), servers, 9, 'Round Robin');
servers.forEach((s) => (s.activeConnections = 0));

simulateRequests(new WeightedRoundRobinBalancer(servers), servers, 8, 'Weighted Round Robin (C
has weight 2)');
servers.forEach((s) => (s.activeConnections = 0));

simulateRequests(new LeastConnectionsBalancer(servers), servers, 9, 'Least Connections');

```

36. Caching Strategies

Asked at: literally everywhere. "Design an LRU cache" is one of the most common data-structure-meets-system-design questions there is, asked standalone AND as a building block in bigger system-design answers (CDN, DB query cache, session store).

Core ideas

Where the cache sits (the pattern, not the eviction policy):

- **Cache-aside (lazy loading)** — app checks cache first; on a miss, reads from the DB and populates the cache. Most common pattern. Cache can go stale if the DB changes elsewhere.
- **Write-through** — every write goes to the cache AND the DB synchronously. Cache is always fresh, but writes are slower.
- **Write-back (write-behind)** — writes go to the cache immediately, DB is updated asynchronously later. Fast writes, but risks data loss if the cache dies before flushing.

Eviction policy (what gets removed when the cache is full):

- **LRU (Least Recently Used)** — evict whatever hasn't been touched in the longest time. The default almost everywhere (Redis, most CDNs) because access patterns are usually "recently used = likely to be used again."
- **LFU (Least Frequently Used)** — evict whatever's been accessed the fewest times. Better for some workloads, but needs extra bookkeeping (a frequency counter) and can keep old "one-time popular" items around too long.

Files

```
node simulate.js    # LRUCache with O(1) get/put, using a JS idiom worth knowing
node exercise.js    # 🌀 extend it - TTL expiry + a cache-aside wrapper
```

What an interviewer is watching for

1. Can you get $O(1)$ `get` / `put` ? The classic answer is a hash map + doubly linked list. In JS specifically, a `Map` **already preserves insertion order** and has $O(1)$ delete+re-insert — `simulate.js` uses that instead of hand-rolling a linked list. Knowing this JS-specific shortcut is a good signal you're not just reciting a memorized answer.
2. Do you know the difference between cache-aside/write-through/write-back, and can you say which one fits a given scenario (e.g. a read-heavy product catalog vs. a write-heavy

counter)?

3. Do you think about cache INVALIDATION — the other classic hard problem ("there are only two hard things in computer science: cache invalidation and naming things")?

exercise.js — your turn

```
// 🐛 EXERCISE — extend the LRU cache without breaking its O(1) contract.
// Copy simulate.js above this comment (or just edit simulate.js directly), then:
//
// TODO 1: add TTL (time-to-live) expiry — `put(key, value, ttlMs)` should store an expiry
//         timestamp alongside the value, and `get(key)` should treat an expired entry as a
//         miss (return -1 AND actually delete it, don't leave dead weight in the map).
//
// TODO 2: write a `getOrCompute(cache, db, key)` function implementing the CACHE-ASIDE
//         pattern from the README: check the cache first; on a miss, call `db.fetch(key)`,
//         `cache.put()` the result, then return it. Use a fake `db` object with a `fetch()`
//         that just logs "hit the database" so you can see cache hits vs misses in the
//         console output.
//
// TODO 3 (bonus): implement an `LFUCache` (Least Frequently Used) — same get/put contract as
//                 LRUcache, but evicts whichever key has the LOWEST access count (track counts in a
//                 second Map). What's a realistic access pattern where LFU would outperform LRU?
```

simulate.js — the simulation

```
// 📖 CACHING STRATEGIES — LRU (Least Recently Used) cache, O(1) get/put
// Chalao: node simulate.js
//
// See README.md for cache-aside vs write-through vs write-back, and why LRU is the
// default eviction policy almost everywhere.

class LRUCache {
  #capacity;
  #map; // JS Map preserves INSERTION order -- we exploit that for O(1) "recency" tracking
  constructor(capacity) { this.#capacity = capacity; this.#map = new Map(); }

  get(key) {
    if (!this.#map.has(key)) return -1;
    const value = this.#map.get(key);
    this.#map.delete(key);
    this.#map.set(key, value); // re-insert -> now "most recently used"
    return value;
  }

  put(key, value) {
    if (this.#map.has(key)) this.#map.delete(key);
    else if (this.#map.size >= this.#capacity) {
      const oldestKey = this.#map.keys().next().value; // first inserted = least recently used
      this.#map.delete(oldestKey);
      console.log(`  evicted (LRU): ${oldestKey}`);
    }
    this.#map.set(key, value);
  }
}
```

```
}

peekOrder() { return [...this.#map.keys()]; } // oldest -> newest
}

// ----- DEMO -----
const cache = new LRUCache(3);
cache.put('a', 1);
cache.put('b', 2);
cache.put('c', 3);
console.log('order:', cache.peekOrder()); // [a, b, c]

cache.get('a'); // touches 'a' -> moves to most-recent
console.log('order after get(a):', cache.peekOrder()); // [b, c, a]

cache.put('d', 4); // capacity full -> evicts LEAST recently used = 'b'
console.log('order after put(d):', cache.peekOrder()); // [c, a, d]

console.log('get(b):', cache.get('b')); // -1, evicted
```

37. Consistent Hashing

Asked at: Amazon, Uber, any company running distributed caches/DBs. A favorite "implement it

live" question because it's genuinely non-obvious until you've seen it once, and it's the mechanism behind DynamoDB, Cassandra, and most CDN/memcached client routing.

The problem it solves

Naïve routing does `hash(key) % serverCount`. That works fine — until you add or remove a server. `serverCount` changes, and almost **every** key's `% serverCount` result changes with it, meaning almost every key suddenly maps to a different server. For a cache, that's a mass cache-miss stampede. For a database, that's a massive, unnecessary data migration.

The idea

Place servers AND keys on a conceptual ring (hash values from 0 to $2^{32}-1$). A key belongs to the first server clockwise from its position. Adding/removing one server only affects the keys between it and its neighbor — roughly `1/N` of all keys, not nearly all of them.

Virtual nodes: a real server gets placed on the ring many times (under different hashes) instead of once, so the keys it owns are spread evenly around the ring instead of one big contiguous chunk. Without virtual nodes, you can get very uneven load by bad luck.

Files

```
node simulate.js      # naive modulo hashing vs. consistent hashing, side by side
node exercise.js      # 🌀 extend it – removeServer() + measure virtual-node impact
```

What an interviewer is watching for

1. Can you explain WHY modulo hashing breaks on scale-up/down, concretely (not just "it's bad")?
2. Do you know virtual nodes exist and why (even distribution, not just "more nodes")?
3. Can you reason about the actual number of keys that move — "roughly `1/N`" is the answer they're listening for, and `simulate.js` proves it with real numbers.

exercise.js — your turn

```
// 🔗 EXERCISE – extend the consistent hashing simulation.
// Copy simulate.js above this comment (or just edit simulate.js directly), then:
//
// TODO 1: demonstrate `removeServer()` the same way the demo demonstrates `addServer()` –
//         build a ring with 4 servers, snapshot every key's assigned server, remove one
//         server, snapshot again, and print how many keys moved. It should again be roughly
//         1/N, not nearly all of them.
//
// TODO 2: measure the IMPACT of virtual node count. Build two rings – one with
//         `virtualNodesPerServer = 1`, one with the default 100 – assign 1000 keys
//         (`user-0` through `user-999`) on each, and print how many keys land on each
//         server. Compute the difference between the busiest and least-busy server for
//         both rings. The low-virtual-node ring should show a much bigger imbalance.
//
// TODO 3 (bonus): why does adding virtual nodes fix the imbalance instead of just placing
//         each real server on the ring once but at a "smarter" position? Write a 2-sentence
//         answer – think about what happens when a 4th server needs to be added later.
```

simulate.js — the simulation

```
// 🔗 CONSISTENT HASHING – why it beats naive modulo hashing when servers scale in/out
// Chalao: node simulate.js
//
// See README.md for the problem this solves and how virtual nodes fit in.

// MD5 as a diffusion hash – this is a Node built-in (no npm install), and it's what real
// consistent-hashing implementations (e.g. Libketama) actually use: not for security, but
// because it spreads similar-looking keys (`user-0`, `user-1`, `user-2`...) evenly across
// the full 32-bit range. A cheaper hand-rolled hash (djb2, FNV) sounds appealing but often
// clusters near-identical short strings together instead of spreading them – which silently
// breaks this whole demo by making one virtual node "own" a huge cluster of keys at once.
import { createHash } from 'node:crypto';
function hash(str) {
  return createHash('md5').update(str).digest().readUInt32BE(0);
}

// ---- Naive: modulo hashing ----
function naiveAssign(key, serverCount) {
  return hash(key) % serverCount;
}

// ---- Consistent hashing: a ring with virtual nodes for even distribution ----
class ConsistentHashRing {
  #ring = new Map(); // ringHash -> serverId
  #sortedHashes = [];
  #virtualNodesPerServer;

  constructor(virtualNodesPerServer = 100) { this.#virtualNodesPerServer = virtualNodesPerServer;
}
```

```

addServer(serverId) {
  for (let i = 0; i < this.#virtualNodesPerServer; i++) {
    this.#ring.set(hash(`${serverId}#${i}`), serverId);
  }
  this.#resort();
}

removeServer(serverId) {
  for (const [h, id] of this.#ring) if (id === serverId) this.#ring.delete(h);
  this.#resort();
}

#resort() { this.#sortedHashes = [...this.#ring.keys()].sort((a, b) => a - b); }

getServer(key) {
  const h = hash(key);
  // first server hash >= key hash, walking clockwise; wrap around at the end of the ring
  const idx = this.#sortedHashes.findIndex((serverHash) => serverHash >= h);
  const ringHash = idx === -1 ? this.#sortedHashes[0] : this.#sortedHashes[idx];
  return this.#ring.get(ringHash);
}
}

// ----- DEMO -----
const keys = Array.from({ length: 20 }, (_, i) => `user-${i}`);

console.log('-- naive modulo hashing --');
const before3 = keys.map((k) => naiveAssign(k, 3));
const after4 = keys.map((k) => naiveAssign(k, 4)); // one server added
const naiveMoved = before3.filter((_, i) => before3[i] !== after4[i]).length;
console.log(` keys that moved when going 3 -> 4 servers: ${naiveMoved}/${keys.length}`);

console.log('-- consistent hashing --');
const ring = new ConsistentHashRing();
['server-A', 'server-B', 'server-C'].forEach((s) => ring.addServer(s));
const beforeRing = keys.map((k) => ring.getServer(k));
ring.addServer('server-D'); // one server added
const afterRing = keys.map((k) => ring.getServer(k));
const ringMoved = beforeRing.filter((_, i) => beforeRing[i] !== afterRing[i]).length;
console.log(` keys that moved when going 3 -> 4 servers: ${ringMoved}/${keys.length}`);

console.log('\nThis is THE reason distributed caches/DBs use consistent hashing: adding a');
console.log('server should only reshuffle ~1/N of the keys, not almost all of them.');
```

38. Database Sharding & Replication

Asked at: any company past "single database" scale — Amazon, Uber, most fintechs.

Interviewers use this to see if you understand the REAL consequences of scaling data, not just "add more servers."

Sharding (horizontal partitioning) — spreading data ACROSS machines

- **Range-based** — e.g. user IDs 0-999 on shard 1, 1000-1999 on shard 2. Simple, but can create **hot shards**: if new users get sequential IDs, all new traffic hits the newest (and often least-provisioned) shard.
- **Hash-based** — `hash(userId) % shardCount`, or better, consistent hashing (see [Lesson 3](#)) so resharding doesn't move almost everything. Spreads load evenly but makes range queries ("all users created this week") expensive — they now have to fan out across every shard.
- **Geo-based** — shard by region, so EU user data stays in an EU shard. Often driven by data-residency LAW (GDPR), not just performance.

Replication — copies of the SAME data for availability/read-scaling

- **Primary-replica (leader-follower)** — writes go to one primary; replicas copy the primary's changes asynchronously and serve reads. Scales read throughput, but replicas can lag behind — this is exactly the "eventual consistency" tradeoff from CAP theorem (see [Lesson 5](#)).
- **Replication lag** creates a real bug class: a user writes something, then immediately reads it back from a lagging replica and doesn't see their own write. `simulate.js` shows this exact failure.

Files

```
node simulate.js      # range-based sharding + primary-replica with simulated lag
node exercise.js      # 🎯 extend it – hash-based sharding + "read your own writes"
```

What an interviewer is watching for

1. Do you know sharding (spreading DIFFERENT data across machines) and replication (copying the SAME data across machines) are solving different problems, and most real

systems use BOTH together (each shard has its own primary + replicas)?

2. Can you name a concrete failure mode of range sharding (hot shards) and hash sharding (expensive range queries)?
3. Do you understand replication lag well enough to propose a fix (route a user's reads to the primary right after their own write) rather than just naming the problem?

exercise.js — your turn

```
// 🛠 EXERCISE – extend the sharding/replication design.
// Copy simulate.js above this comment (or just edit simulate.js directly), then:
//
// TODO 1: implement `HashShardRouter` – same `route(key)` contract as RangeShardRouter, but
//         uses `hash(key) % shardCount` (reuse the MD5-based `hash()` from Lesson 3 – it's
//         what gives an even spread; a cheap hand-rolled hash can silently cluster similar
//         keys together). Route the same 20 keys through both routers and eyeball how much
//         more evenly the hash router spreads them.
//
// TODO 2: fix the "read your own writes" problem from simulate.js – add a
//         `writeThenReadOwn(store, key, value)` helper that: writes the value, then reads it
//         back from the PRIMARY (not the replica) immediately after, so the caller always
//         sees their own write regardless of replication lag.
//
// TODO 3 (bonus): ReplicatedStore currently has exactly one replica. Extend it to support
//         MULTIPLE replicas (an array instead of a single Map), and add a
//         `readFromAnyReplica(key)` that round-robins across them. What new problem does
//         this introduce that a single replica didn't have? (Hint: think about what happens
//         if replicas lag by DIFFERENT amounts.)
```

simulate.js — the simulation

```
// 🗄 DATABASE SHARDING & REPLICATION – routing writes/reads across a distributed DB layer
// Chalao: node simulate.js
//
// See README.md for sharding strategies and what replication lag actually breaks.

// ---- Sharding: range-based ----
class RangeShardRouter {
  constructor(ranges) { this.ranges = ranges; } // [{ shard, min, max }]
  route(userId) {
    const range = this.ranges.find((r) => userId >= r.min && userId <= r.max);
    if (!range) throw new Error(`no shard for userId ${userId}`);
    return range.shard;
  }
}

const router = new RangeShardRouter([
  { shard: 'shard-1', min: 0, max: 999 },
  { shard: 'shard-2', min: 1000, max: 1999 },
  { shard: 'shard-3', min: 2000, max: 2999 },
]);
console.log('-- range-based sharding --');
```

```

console.log('user 42 ->', router.route(42));
console.log('user 1500 ->', router.route(1500));
console.log('user 2800 ->', router.route(2800));
console.log("range sharding is simple but can create HOT shards if user IDs cluster (e.g. all");
console.log("new signups land in the newest range) -- that's why hash-based sharding (see the");
console.log('consistent-hashing lesson) is often preferred for even load.');
```

// ---- Replication: primary-replica with simulated lag ----

```

class ReplicatedStore {
  #primary = new Map();
  #replica = new Map();
  #pendingReplication = [];

  write(key, value) {
    this.#primary.set(key, value);
    this.#pendingReplication.push({ key, value }); // "arrives" at the replica only after a tick
    console.log(` 📝 write ${key}=${value} -> PRIMARY`);
  }

  readFromPrimary(key) { return this.#primary.get(key); }
  readFromReplica(key) { return this.#replica.get(key); }

  tickReplication() { // simulate the async replication delay finally catching up
    const batch = this.#pendingReplication.splice(0, this.#pendingReplication.length);
    for (const { key, value } of batch) {
      this.#replica.set(key, value);
      console.log(` 📄 replicated ${key}=${value} -> REPLICA`);
    }
  }
}

console.log('\n-- replication lag demo --');
const store = new ReplicatedStore();
store.write('balance', 100);
console.log(' read from replica immediately:', store.readFromReplica('balance')); // undefined!
console.log(' read from primary immediately:', store.readFromPrimary('balance')); // 100
store.tickReplication();
console.log(' read from replica after replication:', store.readFromReplica('balance')); // 100

console.log("\nThis is why \"read your own writes\" is a real problem: right after a write, a");
console.log('read hitting a REPLICA can see stale data. Common fixes: route read-after-write');
console.log("reads to the primary, or have the client track a \"read your writes\" token.");

```

39. CAP Theorem & Consistency Models

Asked at: nearly every senior/staff system-design round, usually as a follow-up to "what happens if a server in your design goes down / the network splits?"

The theorem

In a **distributed** system, during a **network partition** (some nodes can't talk to others), you must choose between:

- **Consistency (C)** — every read gets the most recent write, or an error. Refuse to serve possibly-stale data.
- **Availability (A)** — every request gets a response, even if it might be stale.

You can't have both DURING a partition (Partition tolerance, P, isn't really optional for any real distributed system — networks fail, so P is a given, and CAP is really "C vs A when P happens"). When there's no partition, you can have both C and A.

Real systems, real choices

- **CP** (favor consistency) — MongoDB (default config), Zookeeper, etcd, HBase. Good for: bank balances, inventory counts, leader election — anywhere stale data causes real harm (double-spending, overselling).
- **AP** (favor availability) — Cassandra, DynamoDB, CouchDB. Good for: social feeds, product catalogs, shopping carts — anywhere "show something, even if slightly stale" beats "show an error."

Consistency models (the more nuanced answer beyond "CP or AP")

- **Strong consistency** — every read sees the latest write, always.
- **Eventual consistency** — replicas converge to the same value EVENTUALLY, if writes stop. (This is exactly what [Lesson 4](#)'s replication lag demo showed.)
- **Causal consistency** — writes that are causally related (a reply always appears after the comment it replies to) are seen in order; unrelated writes can be seen in any order.

Files

```
node simulate.js      # a CP store and an AP store behaving differently during a partition
node exercise.js      # 🧪 extend it – quorum-based tunable consistency
```

What an interviewer is watching for

1. Do you know P isn't a choice (networks partition whether you like it or not) — the real choice is C vs A, and only DURING a partition?
2. Can you map a REAL requirement ("users must never see a negative balance" vs "the feed can be a few seconds stale") to CP or AP, with a reason?
3. Do you know quorum systems (Dynamo-style $W + R > N$) let you tune consistency PER OPERATION instead of being locked into pure CP or pure AP? That's the exercise.

🧪 exercise.js — your turn

```
// 🧪 EXERCISE – go beyond "pure CP or pure AP" with quorum-based consistency.
// Copy simulate.js above this comment (or just edit simulate.js directly), then:
//
// TODO 1: implement a `QuorumStore` with N nodes, a write quorum W, and a read quorum R.
//         `write(key, value)` succeeds once at least W nodes have the value.
//         `read(key)` reads from R nodes and returns the value with the HIGHEST version
//         number seen (track a version counter per key). This is the Dynamo-style approach:
//         you tune W and R per your consistency needs instead of being locked into CP or AP.
//
// TODO 2: show that when  $W + R > N$  (e.g. N=3, W=2, R=2), a read is GUARANTEED to overlap
//         with the most recent write's quorum – so it always sees the latest value. Then
//         show that when  $W + R \leq N$  (e.g. W=1, R=1), a read CAN miss the latest write
//         (simulate this by writing to nodes [0] only, then reading from node [2] only).
//
// TODO 3 (bonus): what W/R combination would you pick for a "Like" counter on a social post
//                 (where losing a stray like under heavy load is fine) vs. a bank account balance
//                 (where it's never fine)? Justify each in one sentence.
```

✅ simulate.js — the simulation

```
// 🧑‍🔬 CAP THEOREM – during a network partition, pick Consistency OR Availability (not both)
// Chalao: node simulate.js
//
// See README.md for the full theorem, real systems, and consistency models.

class Node {
  constructor(id) { this.id = id; this.data = new Map(); }
}

// ---- CP system: during a partition, refuse requests rather than risk stale data ----
class CPStore {
  constructor(nodes) { this.nodes = nodes; this.partitioned = false; }
  write(key, value) {
    if (this.partitioned) { console.log(` ❌ [CP] write REJECTED – partition detected, can't`
```

```

guarantee consistency`); return false; }
    this.nodes.forEach((n) => n.data.set(key, value));
    console.log(`  ✅ [CP] write succeeded on all ${this.nodes.length} nodes`);
    return true;
  }
  read(key) {
    if (this.partitioned) { console.log(`  ❌ [CP] read REJECTED – can't guarantee it's not stale`); return null; }
    return this.nodes[0].data.get(key);
  }
}

// ---- AP system: during a partition, keep serving, accept possible staleness ----
class APStore {
  constructor(nodes) { this.nodes = nodes; this.partitioned = false; }
  write(key, value) {
    const reachable = this.partitioned ? [this.nodes[0]] : this.nodes; // only reach some nodes during a partition
    reachable.forEach((n) => n.data.set(key, value));
    console.log(`  ✅ [AP] write succeeded on ${reachable.length}/${this.nodes.length} nodes${this.partitioned ? ' (partitioned!)' : ''}`);
    return true;
  }
  read(key, fromNodeIndex = 0) {
    const value = this.nodes[fromNodeIndex].data.get(key);
    console.log(`  ✅ [AP] read from node ${fromNodeIndex}: ${value}${this.partitioned ? ' (might be stale!)' : ''}`);
    return value;
  }
}

// ----- DEMO -----
console.log('-- normal operation (no partition) --');
const cp = new CPStore([new Node('n1'), new Node('n2'), new Node('n3')]);
cp.write('x', 1);
cp.read('x');

console.log('\n-- network partition happens --');
cp.partitioned = true;
cp.write('x', 2); // rejected -- CP chooses Consistency over Availability
cp.read('x');    // rejected

const ap = new APStore([new Node('n1'), new Node('n2'), new Node('n3')]);
ap.write('y', 1);
ap.partitioned = true;
ap.write('y', 2); // "succeeds" but only reaches 1 node -- AP chooses Availability
ap.read('y', 0);  // sees the new value
ap.read('y', 1);  // sees the OLD value -- stale read, but the system stayed UP

console.log('\nNeither choice is "wrong" -- it depends on the product. A payments ledger');
console.log('usually wants CP (better to reject than double-spend). A social media feed');
console.log('usually wants AP (better to show slightly stale data than an error page).');

```

40. Message Queues & Async Processing

Asked at: Uber, Swiggy, any company with background jobs (emails, payments, notifications).

Comes up whenever a system-design answer needs to avoid making the user WAIT for slow work.

Why queues exist

Some work shouldn't block the request that triggered it — sending a confirmation email, resizing an uploaded image, charging a payment. A queue lets a producer say "do this later" and move on, while one or more consumers process the backlog independently. This DECOUPLES

producers from consumers: neither needs to know the other exists, or be up at the same time.

Two shapes

- **Point-to-point queue** — each message is processed by exactly ONE consumer (e.g. "charge this order" should happen once, not once per service that's listening).
- **Pub-Sub (publish-subscribe)** — each message is broadcast to EVERY subscriber independently. This is the [Observer pattern](#) at the scale of independent services instead of objects in one process — an `order.placed` event might need to reach inventory, email, AND analytics, and none of them should know the others exist.

Delivery guarantees (know these cold)

- **At-most-once** — message might be lost, never processed twice. Fast, simplest, riskiest.
- **At-least-once** — message is guaranteed to be processed, but MIGHT be processed twice (e.g. consumer crashes after processing but before acknowledging). Requires consumers to be **idempotent** (processing the same message twice has the same effect as once).
- **Exactly-once** — the ideal, genuinely hard to guarantee across a network; most "exactly once" systems are really at-least-once + deduplication (an idempotency key).

Files

```
node simulate.js      # a retrying point-to-point queue with a dead-letter queue, plus
pub-sub
node exercise.js      # 🔄 extend it - visibility timeout + idempotent processing
```

What an interviewer is watching for

1. Do you reach for a queue specifically to decouple slow/unreliable work from the user-facing request path?
2. Do you know what a **dead-letter queue** is for (messages that fail repeatedly get set aside for manual inspection, instead of blocking/retrying forever)?
3. Do you understand why "at-least-once + idempotency" is the realistic target, not "exactly-once", and can explain what idempotency means concretely (a `messageId` you've already processed gets skipped, not reprocessed)?

exercise.js — your turn

```
// ⚙️ EXERCISE – extend the queue design toward what a real message broker does.
// Copy simulate.js above this comment (or just edit simulate.js directly), then:
//
// TODO 1: add a VISIBILITY TIMEOUT (Like AWS SQS) – when a consumer dequeues a message, it
//          should become invisible to OTHER consumers for some duration instead of being
//          removed immediately. If the consumer doesn't explicitly "delete" it (call a new
//          `ack(messageId)` method) within that window, it should become visible again for
//          someone else to pick up. This is what makes "at-least-once" delivery work when a
//          consumer crashes mid-processing.
//
// TODO 2: make message processing IDEMPOTENT – add a `processedIds` Set to the consumer
//          side (not the Queue itself), and have the handler skip (Log "already processed,
//          skipping") any payload whose `messageId` it's seen before. Enqueue the SAME
//          message twice on purpose and show it only has its real effect once.
//
// TODO 3 (bonus): PubSub here delivers synchronously and in-process. What would have to
//          change for subscribers to run as genuinely separate services (e.g. different
//          Node processes, possibly on different machines)? Name 2 concrete things (in a
//          comment) that this in-memory PubSub is hiding from you.
```

simulate.js — the simulation

```
// 📬 MESSAGE QUEUES & ASYNC PROCESSING – decoupling producers from consumers
// Chalao: node simulate.js
//
// See README.md for point-to-point vs pub-sub, and delivery guarantees.
//
// ---- Point-to-point queue with retry + dead-letter queue ----
class Queue {
  #messages = [];
  #deadLetter = [];
  constructor(maxRetries = 3) { this.maxRetries = maxRetries; }

  enqueue(payload) { this.#messages.push({ payload, attempts: 0 }); }

  process(handler) {
    while (this.#messages.length) {
      const msg = this.#messages.shift();
```

```

    try {
      handler(msg.payload);
      console.log(` ✅ processed: ${JSON.stringify(msg.payload)}`);
    } catch (err) {
      msg.attempts++;
      if (msg.attempts >= this.maxRetries) {
        this.#deadLetter.push(msg);
        console.log(` 🗑️ moved to dead-letter after ${msg.attempts} attempts:
${JSON.stringify(msg.payload)}`);
      } else {
        console.log(` 🔄 retrying (attempt ${msg.attempts}/${this.maxRetries}):
${JSON.stringify(msg.payload)}`);
        this.#messages.push(msg); // requeue
      }
    }
  }
}

deadLetterCount() { return this.#deadLetter.length; }
}

console.log('-- point-to-point queue with retries --');
const orderQueue = new Queue(3);
orderQueue.enqueue({ orderId: 1 });
orderQueue.enqueue({ orderId: 2, forceFail: true }); // always fails -> ends up in the DLQ
orderQueue.process((payload) => {
  if (payload.forceFail) throw new Error('processing failed');
  console.log(` 💰 charging payment for order ${payload.orderId}`);
});
console.log(` dead-letter queue size: ${orderQueue.deadLetterCount()}`);

// ---- Pub-Sub: one event, many independent subscribers ----
class PubSub {
  #subscribers = new Map(); // topic -> [handlers]
  subscribe(topic, handler) {
    if (!this.#subscribers.has(topic)) this.#subscribers.set(topic, []);
    this.#subscribers.get(topic).push(handler);
  }
  publish(topic, event) {
    const handlers = this.#subscribers.get(topic) ?? [];
    console.log(` 📢 publishing to '${topic}' (${handlers.length} subscribers)`);
    handlers.forEach((h) => h(event));
  }
}

console.log('\n-- pub-sub: one event, independent subscribers --');
const bus = new PubSub();
bus.subscribe('order.placed', (e) => console.log(` [inventory-service] reserving stock for
order ${e.orderId}`));
bus.subscribe('order.placed', (e) => console.log(` [email-service] sending confirmation for
order ${e.orderId}`));
bus.subscribe('order.placed', (e) => console.log(` [analytics-service] logging order
${e.orderId}`));
bus.publish('order.placed', { orderId: 99 });

```

```
console.log('\nNotice inventory/email/analytics services never call each other directly --');  
console.log('exactly the decoupling the Observer pattern (Lesson 2) teaches, just at the');  
console.log('scale of independent SERVICES instead of objects in one process.');
```